



Assignment Cover Sheet

Subject Code: CSIT321
Subject Name: Project
Submission Type: Report
Assignment Title: Requirement Document Report | EduAgentX
Student Name: Raniya Habachi, Pranav Sujith Nambiar, Timothy Kyle R. Valle, Neeraj Santosh, Djuraev Timurmalik, Luis Vargas, Salman Akram
Student Number: 8316703, 8097367, 8538797, 8329345, 8562763, 7655538, 8770268

Student Email: _rh225@uowmail.edu.au, _psn993@uowmail.edu.au, _tkv250@uowmail.edu.au, _ns769@uowmail.edu.au, _td164@uowmail.edu.au, _lev936@uowmail.edu.au, _sa109@uowmail.edu.au

Lecturer Name: Dr Patrick Mukala
Due Date: 06/12/2026
Date Submitted: 06/10/2026

PLAGIARISM:

The penalty for deliberate plagiarism is FAILURE in the subject. Plagiarism is cheating by using the written ideas or submitted work of someone else. UOWD has a strong policy against plagiarism. The University of Wollongong in Dubai also endorses a policy of non-discriminatory language practice and presentation.

PLEASE NOTE: STUDENTS MUST RETAIN A COPY OF ANY WORK SUBMITTED

DECLARATION:

I/We certify that this is entirely my/our own work, except where I/we have given fully-documented references to the work of others, and that the material contained in this document has not previously been submitted for assessment in any formal course of study. I/we understand the definition and consequences of plagiarism.

Signature of Student:

Optional Marks:

Comments:

Lecturer Assignment Receipt (To be filled in by student and retained by Lecturer upon return of assignment)

Subject: CSIT321
Student Name:
Due Date: 06/12/2026
Signature of Student:

Assignment Title: Requirement Document Report | EduAgentX
Student Number:
Date Submitted: 05/06/2026

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Requirement Document Report | EduAgentX

Student Name: Raniya Habachi

Student Number: 8316703

Due Date: 06/12/2026

Date Submitted: 06/10/2026

Signature of Lecturer

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Requirement Document Report | EduAgentX

Student Name: Pranav Sujith Nambiar

Student Number: 8097367

Due Date: 06/12/2026

Date Submitted: 06/10/2026

Signature of Lecturer

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Requirement Document Report | EduAgentX

Student Name: Timothy Kyle R. Valle

Student Number: 8538797

Due Date: 06/12/2026

Date Submitted: 06/10/2026

Signature of Lecturer

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Requirement Document Report | EduAgentX

Student Name: Neeraj Santosh

Student Number: 8329345

Due Date: 06/12/2026

Date Submitted: 06/10/2026

Signature of Lecturer

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Requirement Document Report | EduAgentX

Student Name: Djuraev Timurmalik

Student Number: 8562763

Due Date: 06/12/2026

Date Submitted: 06/10/2026

Signature of Lecturer

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Requirement Document Report | EduAgentX

Student Name: Luis Vargas

Student Number: 7655538

Due Date: 06/12/2026

Date Submitted: 06/10/2026

Signature of Lecturer

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSIT321

Assignment Title: Requirement Document Report | EduAgentX

Student Name: Salman Akram

Student Number: 8770268

Due Date: 06/12/2026

Date Submitted: 06/10/2026

Signature of Lecturer

Table of Contents

1. Introduction.....	7
1.1. Purpose	7
1.2. Scope.....	8
1.2.1 Capabilities in Scope for Trimester 1	9
1.2.2. Capabilities in Scope for Trimester 2	10
1.2.3 Permanent Exclusions.....	10
1.3 Business Context	10
1.3.1 The Problem We Are Solving.....	10
1.3.2 Why EduAgentX Is Different.....	11
1.3.3 Who This Is For	12
1.3.4 How We Stack Up Against Existing Tools	13
1.4 Ideas for Solutions	13
1.5 Stakeholders.....	17
1.6 Notation	18
1.7 Document Overview	19
2. General Description and System Scope.....	20
2.2. Scope of the system	21
2.2. High-Level System Interfaces	22
2.3 Functional Requirements Master List.....	25
2.4 Data Requirements.....	26
3. Graphical Systems Modeling.....	28
3.1. Context Diagram.....	29
3.2. Use Case Diagram	30
3.3. Use Case Description.....	31
3.4. Class Diagram.....	33
3.5. Activity Diagram	33
3.6. Sequence Diagram	35
3.7. Architecture Diagram	35
4. System Constraints and Technical Interfaces	36
4.1. Data Interfaces	36
4.2. User Interfaces	39
4.3. Other Interfaces	40
4.4. Other Constraints	42
4.5. Software Requirements.....	43

4.6. Hardware Requirements	45
5. Functional Requirements	46
5.1. FR-01: Agent Registration.....	46
5.2. FR-02: Curriculum Track Assignment.....	47
5.3. FR-03: Autonomous Curriculum Generation	48
5.4. FR-04: Curriculum Version Control.....	49
5.5. FR-05: Training Session Delivery	52
5.6. FR-06: Adaptive Difficulty Adjustment.....	54
5.7. FR-07: Audit Log Maintenance.....	56
5.8. FR-08: Dynamic Examination Generation	57
5.9.FR-09: Automated Exam Evaluation.....	58
5.10. FR-10: Certification Issuance	59
5.11. FR-11: Re-Certification Trigger.....	61
5.12. FR-12: Skill Gap Identification	64
5.13. FR-13: Remedial Training Trigger.....	66
5.14. FR-14: Automated Marketplace Matching.....	68
5.15. FR-15: Task Posting Registry.....	70
5.16. FR-16: System-Wide Health Monitoring	72
5.17. FR-17: Admin Dashboard.....	73
5.18. FR-18: Alert Generation	74
6. Security Requirements.....	75
6.1. Supporting Security Decisions	77
7. Non-Functional Requirements.....	78
7.1. Usability Non-Functional Requirements	78
7.2. Operational and Reliability Non-Functional Requirements	78
7.3. Legal Non-Functional Requirements.....	79
7.4. Other Non-Functional requirements	79
8. Project Management	80
8.1. Open Issues	80
8.2. Preliminary Schedule.....	81
8.3. Preliminary Budget.....	82
9. Pilot Testing.....	82
9.1 Pilot Testing Objective	82
9.2. Scope and Samples	82
9.3. Timeline	83

9.4. Pilot Test Cases.....	83
9.5. Risks and Mitigation.....	84
10. References.....	84
11. Appendix.....	87
Appendix A: Glossary	87
Appendix B: Figures and Tables	89
Figures:	89
Tables.....	94

Table of Contributions

Student ID	Name	Role	Contribution to This Report
8316703	Raniya Habachi	Team Leader	Sec 1.1 Purpose, Sec 1.2 Scope, Sec 1.3 Business Context, FR-10 Certification Issuance, FR-11 Re-certification Trigger, Sec 3.3 Security Requirements (SEC-01 to SEC-08), Final document merge
7655538	Luis Vargas	Requirements Engineer	FR Master List (Table 1, all 18 FRs), FR-01 Agent Registration, FR-02 Curriculum Track Assignment, FR-03 Autonomous Curriculum Generation, FR-04 Curriculum Version Control, FR-08 Dynamic Examination Generation, FR-09 Automated Exam Evaluation, Sec 2.1 Scope of the System
8562763	Djuraev Timurmali	Scribe & Documentation Lead	Sec 1.4 Ideas for Solutions, Sec 1.5 Stakeholders Table, Sec 1.6 Notation, Sec 1.7 Document Overview, Appendix A Glossary, Appendix B References, Proofreading
8538797	Timothy Kyle R. Valle	Market Analysis & Scope	Figure 1 Use Case Diagram, Figure 4 Context Diagram, Table 8 Use Case Summary (UC-01 to UC-12), FR-14 Automated Marketplace Matching, FR-15 Task Posting Registry
8329345	Neeraj Santosh	Resources & Critical Appraisal	Sec 3.1.1 Data Interfaces, Sec 3.1.2 User Interfaces, Sec 3.1.3 Other Interfaces, Sec 3.6 Other Constraints, Table 8 Software Requirements, Table 9 Hardware Requirements, FR-05 Training Session Delivery, FR-06 Adaptive Difficulty Adjustment, FR-07 Audit Log Maintenance
8097367	Pranav Sujith Nambiar	Project Management Lead	Sec 6.1 Open Issues, Sec 6.2 Preliminary Schedule, Sec 6.3 Preliminary Budget, Sec 7.1 Pilot Testing Objective, Sec 7.2 Scope and Samples, Sec 7.3 Timeline, Table 10 Pilot Test Cases, Sec 7.5 Risks and Mitigation, Tables 2/5/6/7 NFRs, FR-16 Health Monitoring, FR-17 Admin Dashboard, FR-18 Alert Generation
8770268	Salman Akram	Technical Architect	Figure 2 Class Diagram, Figure 3 Activity Diagram, Figure 5 Sequence Diagram, Figure 6 Architecture Diagram, Sec 2.4 Data Requirements Table, FR-12 Skill Gap Identification, FR-13 Remedial Training Trigger

1. Introduction

1.1. Purpose

This System Requirements Specification (SRS) will describe what actions should be performed by EduAgentX, what performance criteria should be met, and in what constraints the application should operate. EduAgentX is a platform created by our team which automates training of AI agents, conducting of adversarial examinations, and granting digital certificates to those agents who managed to pass the test; all of that without any human intervention.

SRS is a reference document which is used by all participants of the team when creating this system. Without such document, each individual developer could understand project differently which will inevitably lead to chaos and failure to develop a consistent and coherent system. Thus, according to Sommerville (2016), the goal of an efficient SRS is avoiding re-design by establishing requirements before starting implementation, and this is our goal. All the requirements stated in this specification can be traced back to a particular microservice, a use case, or a database entity.

There are many people who would read this document. The development team will use it as a guide on how the system is supposed to be implemented, from each API endpoint to each database schema and each event structure mentioned. For the testing team, this document will help formulate tests, as any mention of something having to happen in less than 30 seconds is a test that is either passed or failed. Our supervisors and our academic evaluators would use this document to determine whether we have done what we set out to do in creating this program.

The SRS document comprises seven sections. Section one entails the introduction and scope of the project including description of the system, purpose of the system and its stakeholders. Functional requirements comprise section two and include all the 18 functional requirements listed in full with their input, processes, outputs, errors and data processing. System constraints entail section three and

include interface constraints, performance constraints, security constraints, operational constraints and legal constraints. Section four encompasses the UML diagrams. Non-functional requirements include section five which provides measurable acceptance criteria for non-functional requirements. Project considerations including schedule and budget make up section six.

1.2. Scope

EduAgentX is an AI agent learning platform, not a tool for human learners. This is critical because every single element of the process – creation of curriculum, conducting exams, evaluation of results – is designed to work on a premise of the object of study not being a human but an artificial intelligence agent. This is a full-cycle platform, as the agent first registers on it, learns its designated curriculum, takes an exam, earns a certificate if they pass or undergoes targeted remedial training if they fail. They are matched with job postings via an automatically functioning marketplace that doesn't require any human input at all.

System architecture includes eight separate microservices along with an API gateway, both implemented using Docker containers that may be started using a single command. Five layers in terms of their functionality are described in the following table.

Functional Layer	Microservices Included	Key Technologies
Entry Point	API Gateway & Auth Service	FastAPI, PyJWT, Docker
Learning Pipeline	Agent Registry, Knowledge Engine, Training Session Service, Examination Service	FastAPI, LangChain, ChromaDB, PostgreSQL
Post-Certification	Certification & Credential Service, Talent Marketplace Service	FastAPI, PostgreSQL, Redis pub/sub
Observability	Monitoring & Observability Service, Skill Gap Analysis Service	FastAPI, Prometheus, PostgreSQL

Data Layer	PostgreSQL, ChromaDB, Redis	PostgreSQL 16, ChromaDB 1.5.9, Redis 7.2
-------------------	-----------------------------	--

Table 1. — System Architecture

1.2.1 Capabilities in Scope for Trimester 1

The capabilities we are aiming to provide you with by Week 11 and demonstrate during the First Panel

Presentation at Week 9:

- Eight microservices plus the API Gateway which can be easily deployed on an empty Ubuntu 24.04 VM running docker-compose up.
- A simulation pipeline for generating more than 1,000 agent interaction logs of three different types of agents: those who go through the process in one attempt, those who need more than one attempt and those who fail to reach the 80% mark. This will enable us to test all the branches of our pipeline using synthetic rather than commercial agents.
- At least one demonstration curriculum domain, we currently consider AI Safety Fundamentals and Cybersecurity Threat Analysis, along with seed data stored in ChromaDB.
- Self-curated content creation via LangChain for scheduling API calls on behalf of OpenAI, resulting in versioned JSON files stored in PostgreSQL.
- Adversarial-style assessment creation featuring three cognitive levels of assessment: memory (40%), application reasoning (40%), and adversarial reasoning (20%). No two agents will receive the same exam.
- Assessment scoring and evaluation via LLMs where passing is defined as 80%. Upon passing, a cert.issued event fires in Redis; otherwise, a skills assessment occurs.
- Creation of immutable digital credentials by applying SHA-256 to all credentials. These credentials are stored in an append-only PostgreSQL table, never UPDATE nor DELETE.
- Marketplace automated match within 30 seconds after certification by creating dummy task posting of 10 to 20 in number.
- Gap assessment will help determine where exactly the agent needs to be trained and what sub-domain they have trouble with.
- Remedial training would follow, rather than re-teaching everything from scratch.

- Admin React dashboard showing service health status, pipeline performance, curriculum validity, and audit trail.
- health endpoints on all services being polled every 30 seconds and reporting to the metrics endpoint in Prometheus.
- Append only auditing enabled on all services with SHA-256 tampering detection.

1.2.2. Capabilities in Scope for Trimester 2

- Connection with commercial AI agents by means of third-party APIs - T1 utilizes only fake AI agents.
- Real-time job board connection and enterprise webhooks for matching on the marketplace.
- Credential anchoring in a blockchain or CT log.
- Write functionality for the admin dashboard - the T1 dashboard is read-only.
- Curriculum expansion across multiple domains rather than only the two demonstrated T1 domains.
- In-house LLM running with GPU capability - T1 utilizes the OpenAI API.
- Mobile-optimized admin dashboard with multiple language options.
- Clustering of PostgreSQL database with replication of Redis database - T1 runs single nodes of each.

1.2.3 Permanent Exclusions

Things that EduAgentX will never do, come first or second trimester:

- Teach human students, nor give any sort of certificates. It's strictly an AI Agent platform.
- Payment processing, subscription handling, nor commerce operations.
- Gather any sort of personally identifiable information. Everything collected about the agent in the first trimester is synthetic data, where the only identifiable thing is the unique identifier generated through UUID v4.

1.3 Business Context

1.3.1 The Problem We Are Solving

In today's business world, artificial intelligence is being adopted at an unprecedented rate across all sorts of critical applications, from financial analysis to law, medicine, logistics, and cybersecurity. And yet these

enterprises have no way of knowing whether their agents know what they are doing or whether their reasoning holds up under pressure. This problem is precisely why EduAgentX was invented.

This issue is of considerable importance. As per estimates by Gartner (2025), by the year 2026, 40 percent of enterprise applications will feature task-specific AI agents, compared to only five percent in the previous year (2025). McKinsey & Company (2025) estimated that 62 percent of firms were already implementing agentic systems through pilot projects and full deployment programs. According to Institute for Law & AI (2026), implementing agentic systems in highly-regulated industries without competency certification can lead to legal liabilities.

Existing solutions fail to address this issue:

- Platforms such as Coursera and LinkedIn Learning have been designed for humans; video materials, quiz activities, and peer forums make no sense for the machine.
- Internal assessment is carried out by the development team, introducing a conflict of interest into the process and lacking independence.
- Research benchmarks like MMLU, HELM, and BIG-Bench (Liang et al., 2023) are fixed datasets. It has been demonstrated that models can be trained on them in such a way that improves the result in terms of the dataset but not the targeted domain (Dong et al., 2024). Besides, such solutions generate no verifiable credentials; rather, they deliver a high score on the leaderboard that no one outside academia is concerned about (Katzman et al., 2025).

1.3.2 Why EduAgentX Is Different

This platform is not an adaptation of any previous one. EduAgentX represents an entirely novel approach to AI certification that equates to how a professional board certifies humans but automatically. Table 1 below provides an overview of its distinguishing features.

What Makes It Different	What That Actually Means in Practice
-------------------------	--------------------------------------

Zero-human-loop operation	All this takes place automatically with no need for any human action: from registration to generating the curriculum, training, testing, grading, certification, and finally matching on the market. This makes the price for certifying another agent close to zero.
Adversarial, agent-native exams	Each test is created anew for each agent based on three different cognitive levels, with questions that can be used to trick someone who uses pattern matching into failing. One will not be able to take advantage of having learned from past tests.
Tamper-evident credentials	Each credential certificate is hashed by SHA-256 using agent UUID, domain, curriculum version, score, and timestamp. The credentials table is immutable at the database layer level. Changing a credential results in detecting the hash mismatch.
Curriculum currency guarantee	As the knowledge database within the domain gets revised, agents with expired certificates will be automatically informed to recertify. Only the agents that have been certified based on the latest version of the curriculum appear on the marketplace.
Research-grade audit trail	The training prompt given, answer provided during the exam, grade given, as well as any certification conducted, will all be recorded using SHA-256 to ensure that tampering cannot occur. This is also very useful from a learning perspective.

Table 2. — Distinctive System Capabilities

1.3.3 Who This Is For

The audience for EduAgentX can be identified in three groups:

- Teams developing AI agents or corporations working with specialized agents who require independent confirmation of the quality of their product, for internal processes, commercial purposes, or regulation compliance.
- Academicians who work on evaluating AI agents, building benchmarks, or conducting adversarial testing who require flexible and easily repeatable evaluation procedures.
- Platform managers interested in having an independent quality assurance step before adding agents to the talent market.

From the point of view of business model, zero-human-loop approach offers excellent unit economics. When the platform is built, adding another agent to certify requires no other expense but LLM API calls, which is a far cry from any alternative solution.

1.3.4 How We Stack Up Against Existing Tools

This competitor analysis was part of our feasibility study and has been included in this paper since it directly affects how EduAgentX was scoped.

Competitor / Tool	What They Actually Do	Why EduAgentX Is Different
Coursera / LinkedIn Learning	Human-based e-learning - video courses and certificates for professional development.	We credential artificial intelligence agents, not humans. They have completely different architectures and logic behind evaluation.
Stanford HELM (Liang et al., 2023)	Holistic static analysis of LLMs against benchmark data sets.	We create adversarial assessments that are customized to each agent. HELM creates a leaderboard scoring, and not verifiable credentialing in the marketplace.
Internal red-teaming / unit testing	The organization assesses its agents by its standards.	We are a third-party independent entity. We have no conflict of interest here. Third-party verifiability.
Scale AI / Turing	Humans rate the performance of the AI in terms of quality and safety.	Our process requires zero humans for assessment. This is faster and more cost-effective. Scalable to a thousand or more agents at the same time.
Arize AI / WhyLabs	Watch out for drift in production ML models.	Domain competence certification takes place before deployment, not after.

Table 3. — Competitor Analysis

1.4 Ideas for Solutions

Before selecting EduAgentX as the final project direction, the team considered several related solution ideas. The comparison below shows how VisionVoice, FocusLens AI, and EduAgentX differ

in purpose, technical fit, risk, and alignment with supervisor feedback. The key design movement was from a human-assistance tool toward a more original autonomous system for AI-agent learning, testing, and certification (ISO/IEC/IEEE,2018).

Solution Idea	Main Concept	Strengths	Weaknesses / Risks	Prof. Farhad's Feedback Applied	Decision
VisionVoice	A multimodal learning-support concept that combines visual input with voice interaction. The idea is to help users understand the information through spoken guidance, visual prompts and accessibility-style interaction.	Good for inclusive learning and human-computer interaction. The concept is easy to explain and can be demonstrated with voice recognition, text-to-speech, and simple computer-vision functions.	It remains strongly human-user focused. It does not fully address the project direction of autonomous AI-agent learning, certification, and marketplace readiness. It also overlaps with existing accessibility and assistant tools unless the scope is made more original.	Prof. Farhad's feedback was treated as a signal that the project should avoid becoming another generic human-assistance tool. VisionVoice would need a stronger systems angle and clearer novelty to be suitable as the final project.	Not selected as the final direction. It helped the team think about multimodal interaction, but it was less aligned with the final AI-agent certification platform.

FocusLens AI	A computer-vision and learning-analytics concept tracking attention and engagement with video, gaze, orientation, and retention signals. It is an EdTech platform using FastAPI, TensorFlow, MediaPipe, SHAP, React, PostgreSQL, Redis, Docker, and eight microservices (FastAPI, n.d.; TensorFlow, n.d.; Google AI Edge, n.d.; SHAP Developers, n.d.; React, n.d.; PostgreSQL Global Development Group, n.d.; Redis, n.d.; Docker, n.d.), as described in the uploaded topic sheet.	Technically strong with a clear microservice structure, measurable attention scores, explainable low-attention alerts and realistic teacher dashboard. The proposal sheet also registered it as a high-scoring topic idea.	The main problem is that the original concept still depends on human classrooms, human teachers and student monitoring. That creates privacy, consent, and ethical risk, even if synthetic video is used during development. It also makes the system more like a monitoring dashboard than an autonomous learning platform.	Prof. Farhad's feedback was that the idea should not simply assist human professors or teaching staff. The useful part of FocusLens AI should be redirected toward improving AI tutoring and autonomous learning rather than monitoring human students.	Partially selected and transformed. Its "detect learning gaps" idea became the Skill Gap Analysis concept in EduAgentX, but the human classroom monitoring part was removed.
--------------	--	--	--	---	--

EduAgentX	A full-cycle platform where AI agents, not human learners, register, receive curriculum, complete training, sit dynamic adversarial examinations, earn tamper-evident digital credentials (NIST, 2015), and are matched to task postings. The report defines it as a zero-human-loop AI-agent learning and certification platform built using eight Dockerised microservices, an API Gateway, PostgreSQL, Redis, ChromaDB, and monitoring services (Docker, n.d.; PostgreSQL Global Development Group, n.d.; Redis, n.d.; ChromaDB, n.d.; Prometheus Authors, n.d.) .	Most aligned with the final requirements because it is autonomous, agent-native, assessable through functional requirements, and suitable for a microservice architecture. It also avoids collecting personally identifiable information because the agents are represented by UUIDs (Davis, Peabody and Leach, 2024).	The scope is more complex than the earlier ideas. It needs careful boundaries, clear functional requirements, realistic T1/T2 separation, and strong explanation of how certification, re-certification, audit logs, and marketplace matching work.	Prof. Farhad's feedback was applied by reframing the project away from human classroom assistance and toward an autonomous AI-agent learning system. This made the final idea more original, more technical, and easier to justify as a software engineering project.	Selected as the final project direction. EduAgentX keeps the strongest part of FocusLens AI, gap detection, but applies it to AI-agent training, remediation, certification, and marketplace readiness.
-----------	---	--	---	---	---

Table 4. — Alternative Solution Concepts and Project Selection Logic

The selected direction is EduAgentX because it provides the strongest software engineering scope. Unlike VisionVoice and the original FocusLens AI concept, EduAgentX is not mainly a

human-support product. It is a system-of-services where AI agents move through enrolment, curriculum assignment, training, examination, certification, re-certification, and marketplace matching. This direction also fits the existing SRS because the report already defines eight microservices, event-driven Redis communication, append-only credentials, ChromaDB knowledge storage, and a monitoring dashboard (Redis, n.d.; NIST, 2015; ChromaDB, n.d.; Prometheus Authors, n.d.).

1.5 Stakeholders

The stakeholders below are the main parties affected by EduAgentX. They are included because the system is not a normal student learning system; its primary clients are AI agents, while human users mainly supervise, monitor, evaluate, or consume the certification results (ISO/IEC/IEEE, 2018).

Stakeholder	Role in the System	Needs / Concerns	Related Requirements
AI Agent	Primary system user/client. It registers with a target domain, receives training, sits examinations, obtains credentials, and may be matched to task postings.	Requires reliable API access, accurate curriculum assignment, fair examination generation, clear pass/fail output, diagnostic feedback after failure, and credential retrieval.	FR-01 to FR-13, especially registration, training, examination, certification, skill-gap analysis, and remedial training.
Platform Admin	Human administrator responsible for monitoring the platform rather than manually training or grading agents.	Requires dashboard visibility over service health, curriculum status, certification results, re-certification queue, audit logs, marketplace matches, and alerts.	FR-14 to FR-18, security requirements, dashboard and monitoring requirements.
Enterprises	Organisations that need certified AI agents for specialised tasks or internal deployment.	Require trustworthy credentials, current curriculum versions, verifiable competence, marketplace matching, and assurance that agents were assessed independently.	FR-10, FR-11, FR-14, FR-15 and security/audit requirements.

Research Institutions	Academic or research groups interested in repeatable AI-agent evaluation, benchmark-style testing, and skill-gap data.	Require structured logs, reproducible examinations, domain-specific diagnostic outputs, and research-grade audit trails without personally identifiable information.	FR-08, FR-09, FR-12, FR-16 and data requirements.
Supervisors	Project supervisors and academic evaluators who check whether the project is feasible, properly scoped, documented, and testable.	Require clear requirements, traceability between diagrams and FRs, a realistic T1 scope, correct notation, measurable acceptance criteria, and consistent terminology.	All sections of the SRS, especially Sections 1, 2, 4, 6, 7 and Appendix A.
Dev Team	The student development team that designs, implements, tests, integrates, and documents the system.	Requires clear service responsibilities, event names, database entities, interface definitions, constraints, risk controls, and consistent glossary terms.	All functional requirements, diagrams, interfaces, data requirements, NFRs and implementation constraints.

Table 5. — Stakeholders

1.6 Notation

This SRS uses several diagram types to describe EduAgentX from different viewpoints. The notation is kept consistent with standard UML-style documentation, while the context and architecture diagrams are used as supporting system-boundary and deployment views (Object Management Group, 2017; ISO/IEC/IEEE, 2022).

Diagram Type	What the Diagram Shows	Purpose in This SRS
Use Case Diagram	Shows external actors and the main services/use cases they interact with. In this report it should show AI Agent, Platform Admin, and System AI, together with the main use cases UC-01 to UC-12.	Used to define the system boundary at a user-goal level before the detailed FRs are described. It supports stakeholder understanding and traceability from actors to functional requirements (Object Management Group, 2017).

Class Diagram	Shows the main domain entities and their relationships e.g. Agent, Curriculum, Module, TrainingSession, Examination, Question, SkillGap, Credential, JobPosting, AgentMatch and AuditEvent.	Used to elucidate the data model and object relationships behind the functional requirements and the PostgreSQL tables (Object Management Group, 2017).
Activity Diagram	Illustrates the flow of activities from enrolment, training, examination, pass/fail decision, certification, skill gap analysis, remedial training, re-exam trigger and marketplace matching.	Describing logic of end-to-end process and alternative paths after examination results (Object Management Group, 2017).
Sequence Diagram	Shows time-ordered messages between the AI Agent, API Gateway, Examination Service, Knowledge Engine, LLM API, Certification Service, Marketplace Service, and Redis Pub/Sub.	Used to explain how services talk to each other, how events are published, asynchronous processing, and how an exam result causes the certification or failure processing (Object Management Group, 2017).
Context Diagram	EduAgentX as one system boundary connected to external entities such as AI Agent, System Administrator, LLM Provider and External Talent Marketplace.	It used to show inputs and outputs at the highest level without internal service detail. It helps to confirm what is in scope and out of scope (ISO/IEC/IEEE, 2022).
Architecture Diagram	Shows the internal microservice structure, API Gateway, databases, Redis event bus, ChromaDB, monitoring, and dashboard components.	Used to connect requirements to implementation structure, deployment decisions, and service ownership (ISO/IEC/IEEE, 2022; Docker, n.d).

Table 6. — Diagram Types

1.7 Document Overview

The Requirement Document Report is organised so that readers can move from high-level project purpose to detailed requirements, diagrams, constraints, testing, and project considerations (ISO/IEC/IEEE, 2018; Somerville, 2016). The sections are summarised below.

Section 1 - Introduction: outlines the purpose of SRS, project scope, business viewpoint, comparison of solutions, stakeholders, notation and document structure.

Section 2 - Functional Requirements and System Scope: describes the functional requirements and defines the core workflows covering agent registration, curriculum assignment, training,

examination, certification, skill gap analysis, re-certification, marketplace matching, monitoring, dashboard and alert generation.

Section 2.1 Data Requirements: In this section, we explain the main data entities used in the platform: agents, curricula, modules, training_sessions, tests, questions, skill_gaps, credentials, job_postings, agent_matches, audit_events, and system_health.

Section 2.2 / Diagram Section: Diagrams to communicate the system structure and behavior such as use case, class, activity, sequence, context and architectural diagrams.

Section 3 - Security Requirements: describes rules for JWT authentication, API rate limiting, prompt-injection protection, append-only credentials, TLS, no-PII storage, dashboard session expiration, and audit-event integrity (OWASP, 2023; NIST, 2015).

Section 4 - Requirements Engineer Content: The detailed explanations of the functional requirements, such as inputs, processes, outputs, constraints, data processing and error handling.

Section 5 - Interfaces and Constraints: defines data interfaces, user interfaces, other technical interfaces, software requirements, hardware requirements and implementation constraints.

Section 6 - PROJECT CONSIDERATIONS: Discusses open issues, assumptions on schedule, budget estimates and areas which may need supervisor review or later design decisions.

Section 7 - Pilot Testing: describes the goal, scope, scheduling, pilot testing scenarios and risk mitigation strategy for the system validation prior to final delivery.

Appendix A - Glossary: defines the main project terms and ensures they match how the terms are used in the functional requirements.

Appendix B - References: contains the sources used to justify the project background, software engineering approach, security controls, and modelling notation.

2. General Description and System Scope

2.2. Scope of the system

EduAgentX is an independent, full-service AI-agent certification platform, offering end-to-end learning, testing, certification, and marketplace-matching solutions for AI agents. The system boundary includes the API Gateway, 8 microservices running in Docker containers, the databases (PostgreSQL, Redis, ChromaDB) and the associated monitoring components. All education, examination, grading, certification and matching activities are automatically performed without the need to involve human beings. Only human users interact, via the administrative dashboard for monitoring and configuration. The platform expressly denies that it includes human learning, payment transactions or storing of personally identifiable information.

External Interfaces

1. AI Agent Interface
 - REST API endpoints exposed through API Gateway
 - JWT-based authentication
 - Registration, training, examination, and credential retrieval
2. OpenAI API
 - Curriculum generation
 - Examination generation
 - Automated grading
 - Skill-gap analysis
3. ChromaDB Vector Database
 - Knowledge retrieval
 - Curriculum grounding
 - Semantic similarity searches
4. Redis Pub/Sub
 - Event-driven communication
 - agent.registered
 - agent.track_assigned

- training.complete
 - exam.passed
 - exam.failed
 - cert.issued
 - recert.required
5. PostgreSQL Database
- Persistent storage for agents, curricula, examinations, credentials, audit logs, and marketplace data
6. Prometheus Monitoring Interface
- Collection of service metrics
 - Health monitoring
 - Alert generation
7. Administrative Dashboard
- React SPA
 - System health monitoring
 - Audit review
 - Curriculum status monitoring
 - Marketplace analytics

2.2. High-Level System Interfaces

The system is a microservice architecture that interacts with a few data interfaces using EduAgentX. The system uses different communication means for its services. These include synchronous REST APIs and asynchronous Redis Publish / Subscribe for communicating system events. This works for eduAgentX since some operations should be done right away while others should simply notify other services about state changes.

Synchronous REST API is used for external and internal operations where the client/service needs to receive a clear response. That means if the operation involves getting the response back from the server, e.g., an agent registration process, checking the state of credentials, requesting the current training module or sending a training response, the API Gateway will forward the request to the relevant backend microservice. REST is good in situations when a particular action must produce a certain result; the possible results might include success, failure, validation errors, JSON object responses, etc. This is a great choice because FastAPI is used for creating API endpoints and it makes OpenAPI integration, request/response validation, and security easy for this project (FastAPI, 2026). It is important for us because we'll be able to compare the API implementation with the specification created in the SRS, and the testing and development teams can also benefit.

Asynchronous events communication is provided using Redis Publish/Subscribe. One shouldn't expect all of the system services to wait for each other to finish their job directly. For example, the training session ends so the Training Session service publishes training.complete event. This way, the Examination Service doesn't have to call the Training Session Service to know that the training session is over; the latter sends the event so the former could start preparing the examination. Similar events are sent out including agent.registered, agent.track_assigned, exam.passed, exam.failed, cert.issued and recert.required. Such events allow tracking an agent through the whole life-cycle without the need for all the services to interact tightly with each other. Redis Pub/Sub allows services to exchange messages between themselves using channels (Redis, 2026). One service publishes a message to a channel while other services that subscribe to that channel receive it. The problem here is that this technology provides at-most-once delivery so if a service receives a message but is unable to process it, it won't retransmit it. Therefore, this method is not to be used for transmitting important state changes in EduAgentX; PostgreSQL would be used instead.

The interface used to communicate between the AI engine and the platform utilizes the OpenAI API. It provides means for curricula generation, responses' assessments, and examinations

evaluations. In Trimester 1, the team of EduAgentX does not intend to launch its own local LLM because of the requirement for GPU resources which cannot be afforded in the project timeframe and project budget. Instead, the Knowledge Research and Curriculum Engine communicates with the OpenAI API using structured prompts and receives back JSON objects as response. The OpenAI API relies on HTTP-based API calls and API-key based authentication and thus it is crucial to protect the security of the API key by storing it in Docker environment variables (OpenAI, 2026). Such interface is strong yet the limitations are significant. The system depends on third-party services, API rate limits, costs, and validation of output, and hence all LLM's outputs must be validated before storing or communicating them further.

ChromaDB is applied as vector database interface for the knowledge retrieval. Before making new curriculum modules, the Knowledge Research and Curriculum Engine retrieves the relevant domain content from ChromaDB. The ChromaDB interface is applicable here due to the ability to work in the client-server mode, when Chroma client sends requests to a Chroma server through HTTP (ChromaDB, 2026). That means the deployment of the separate ChromaDB container as well as usage of the database independently of the Knowledge Engine without having it embedded in the source code as is in EduAgentX's project. Note that the data retrieved from ChromaDB will be used as basis for grounding the created curriculums, therefore, the curriculum modules will not be generated based on the LLM's raw outputs.

PostgreSQL acts as main persistence interface for the structured data. The following types of data will be kept in it including agents, curriculum version information, training session details, examination scores, credentials, matches in the marketplace, gaps, system health logs, alerts, and audits. The use of PostgreSQL database is justified by the use of structured queries and transaction integrity (PostgreSQL Global Development Group, 2026). Some functions of the platform can be executed through Redis and ChromaDB but the primary data store will still be PostgreSQL.

The interface design for accessing the data is based on a clear segregation of tasks. In this regard, they apply REST APIs for controlling request-response operations, Redis Pub/Sub for handling event-based communication, OpenAI API for generating and evaluating LLMs, ChromaDB for semantic knowledge acquisition and PostgreSQL for persistently storing records of the system.

2.3 Functional Requirements Master List

FR ID	Requirement Name	Description	Responsible Microservice
FR-01	Agent Registration	Register AI agents and create unique learning profiles	Agent Registry & Enrolment Service
FR-02	Curriculum Track Assignment	Automatically assign agents to curriculum tracks	Agent Registry & Enrolment Service
FR-03	Autonomous Curriculum Generation	Generate structured curricula using LLMs and knowledge retrieval	Knowledge Research & Curriculum Engine
FR-04	Curriculum Version Control	Maintain version history and activation status of curricula	Knowledge Research & Curriculum Engine
FR-05	Training Session Delivery	Deliver curriculum modules to enrolled agents	Training Session Service
FR-06	Adaptive Difficulty Adjustment	Dynamically adjust learning difficulty	Training Session Service
FR-07	Audit Log Maintenance	Record tamper-evident training audit logs	Training Session Service
FR-08	Dynamic Examination Generation	Generate unique adversarial examinations	Examination & Skill Validation Service
FR-09	Automated Exam Evaluation	Grade examinations using LLM evaluation	Examination & Skill Validation Service

FR-10	Certification Issuance	Issue immutable credentials to successful agents	Certification & Credential Service
FR-11	Re-Certification Trigger	Trigger re-certification after curriculum changes	Certification & Credential Service
FR-12	Skill Gap Identification	Identify weak reasoning domains after exam failure	Skill Gap Analysis Service
FR-13	Remedial Training Trigger	Route failed agents to corrective learning paths	Training Session Service
FR-14	Automated Marketplace Matching	Match certified agents to suitable opportunities	Talent Marketplace Service
FR-15	Task Posting Registry	Manage marketplace task postings	Talent Marketplace Service
FR-16	Health Monitoring	Monitor service health and uptime	Monitoring & Observability Service
FR-17	Administrative Dashboard	Display platform metrics and status	Monitoring & Observability Service
FR-18	Alert Generation	Generate operational alerts and notifications	Monitoring & Observability Service

Table 7. — Functional Requirements Master List

2.4 Data Requirements

Entity Name	Attributes	Description
agents	agent_id (UUID, PK), target_domain (String), registered_at (DateTime), status (String)	Core registry of all AI agents interacting with the platform.

curricula	track_id (UUID, PK), domain_name (String), version (String), is_active (Boolean)	Version-controlled learning tracks generated by the Knowledge Engine.
modules	module_id (UUID, PK), track_id (UUID, FK), content_vector_ref (String), sequence_order (Int)	Specific learning modules linked to vector embeddings in ChromaDB.
training_sessions	session_id (UUID, PK), agent_id (UUID, FK), module_id (UUID, FK), comprehension_score (Float), difficulty_level (Int)	State persistence for active learning sessions.
examinations	exam_id (UUID, PK), agent_id (UUID, FK), final_score (Float), result_status (String)	Logs of all certification attempts and final evaluated scores.
questions	question_id (UUID, PK), exam_id (UUID, FK), cognitive_tier (String), content (String)	Specific questions generated dynamically per exam session.
skill_gaps	gap_id (UUID, PK), exam_id (UUID, FK), diagnostic_payload (JSON), severity (String)	Diagnostic data generated upon exam failure for remediation routing.
credentials	cert_id (UUID, PK), agent_id (UUID, FK), score (Float), curriculum_version (String), hash_signature (String)	CONSTRAINT: Append-only table storing verifiable agent certifications.
job_postings	posting_id (UUID, PK), required_domain (String), min_score_threshold (Float), is_open (Boolean)	External task registries requiring specific certified agent skills.

agent_matches	match_id (UUID, PK), cert_id (UUID, FK), posting_id (UUID, FK), relevance_score (Float), matched_at (DateTime)	Automated pairings between certified agents and open task postings.
audit_events	event_id (UUID, PK), event_type (String), payload (JSON), timestamp (DateTime)	CONSTRAINT: Append-only tamper-evident log for all system state changes.
system_health	health_id (UUID, PK), service_name (String), status (String), last_polled (DateTime)	Telemetry table tracking the 30-second uptime polling data.

Table 8. — Data Requirements

3. Graphical Systems Modeling

3.1. Context Diagram

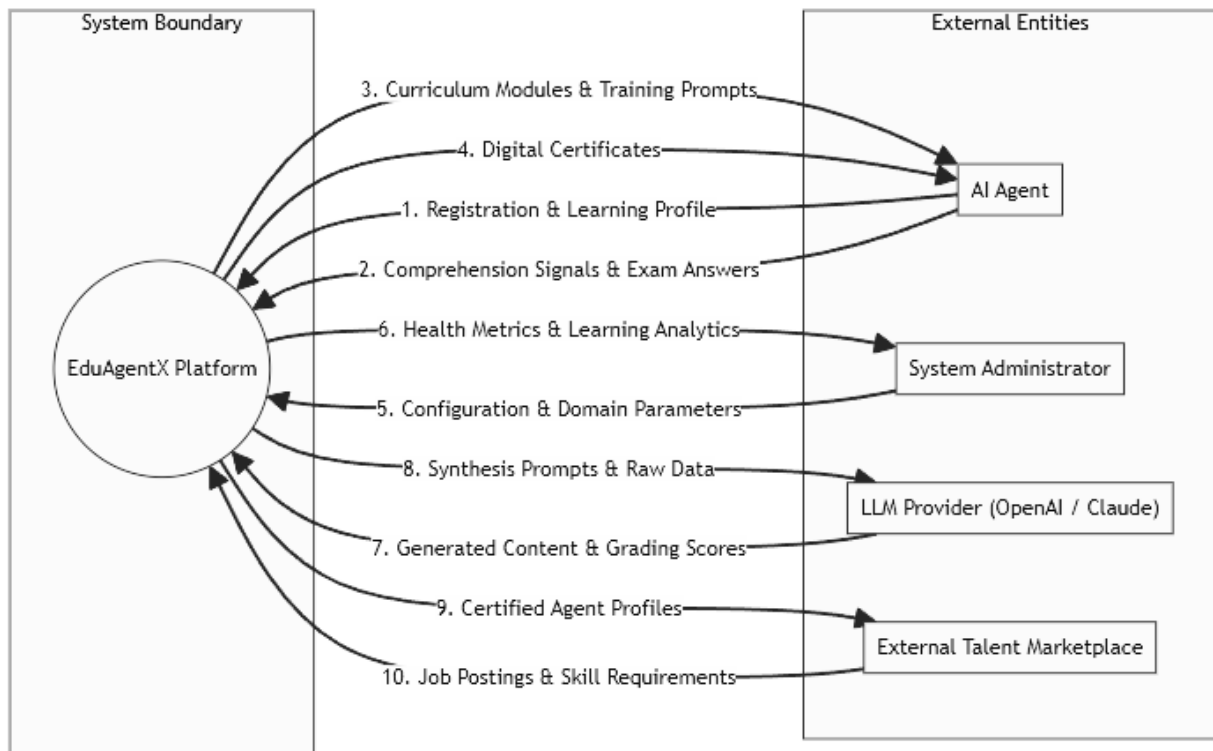


Figure 1 - Context Diagram

Figure 4.0 presents the EduAgentX platform in relation to its four external entities: the AI Agent, the System Administrator, the LLM Provider, and the External Talent Marketplace. Together with the ten numbered data flows that connect them to the system boundary, the diagram establishes that AI agents, rather than human learners, are the platform's primary clients. The AI Agent is the dominant actor, initiating the lifecycle by submitting a registration payload and eventually receiving the platform's critical outputs: curriculum modules and digital certificates. This fully automated relationship reflects the direction of the enterprise market, where Gartner (2025) projects that 40% of business applications will incorporate task-specific AI agents by 2026. The LLM Provider serves as the external intelligence layer, replacing human subject matter experts and assessors by generating curriculum content and grading scores via the LangChain framework (Chase, 2022). Finally, the External Talent Marketplace provides the job postings and skill thresholds necessary for the Talent Marketplace Service to execute its automatic matching logic within thirty seconds of certificate issuance.

3.2. Use Case Diagram

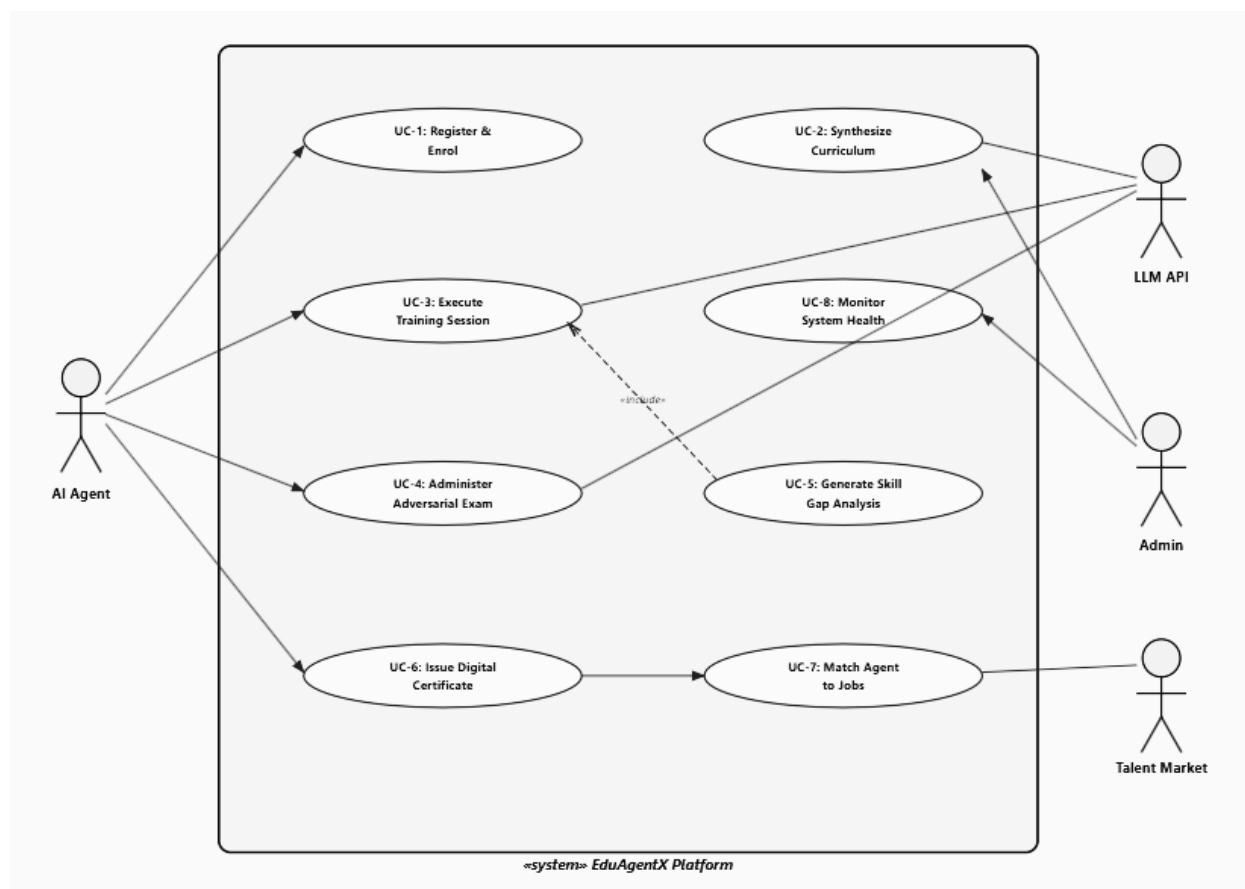


Figure 2 - Use Case Diagram

Figure 1.0 maps the EduAgentX platform as a series of use cases distributed across the identified actors, constructed in accordance with UML 2.5.1 notation (OMG, 2017). The AI Agent is associated with the primary learning lifecycle: registration (UC-01), training (UC-03), examination (UC-04), and certification (UC-06). A critical architectural feature is the dashed «include» arc from UC-05 (Generate Skill Gap Analysis) to UC-03 (Execute Training Session). This models the remedial loop that activates upon examination failure, a mechanism designed to mitigate the reasoning failures documented in high-stakes AI environments (Cheng et al., 2025). While the diagram focuses on the eight primary use cases to preserve readability, four secondary use cases (UC-09 through UC-12) are included in the formal specifications below to cover internal system audits and administrative configuration.

3.3. Use Case Description

The table below provides a formal specification for each of the twelve identified use cases, covering Name, Actor(s), Pre-conditions, and Post-conditions. These descriptions are grounded in the functional requirements baseline (FR-01 through FR-15) and the architectural decisions established for the EduAgentX platform.

UC ID	Name	Actor(s)	Pre-condition	Post-condition
UC-01	Register & Enrol	AI Agent	Valid JSON payload received; no duplicate UUID in PostgreSQL.	UUID (v4) persisted; learning profile initialised; assigned to curriculum track.
UC-02	Synthesize Curriculum	LLM API, Knowledge Engine	Sufficient knowledge embeddings in ChromaDB; valid LLM API key.	Progressive modules generated in PostgreSQL; version incremented; freshness reset.
UC-03	Execute Training	AI Agent, LLM API	Agent status is ENROLLED or REMEDIAL; current curriculum module assigned.	Prompts delivered; interaction logs written to sessions_audit; status set to COMPLETED.
UC-04	Administer Exam	AI Agent, LLM API	Agent has completed all training modules; LLM API is reachable.	Unique adversarial exam generated; responses graded; result persisted in exam_records.

UC-05	Skill Gap Analysis	Skill Gap Service	exam.failed event consumed from Redis; failed response payload available.	Ranked sub-domain weakness map persisted; diagnostic payload sent to Training Service.
UC-06	Issue Certificate	Certification Service	Exam PASS with score $\geq$ 80%; no duplicate cert record for current version.	Immutable record appended to certification_records; cert.issued event published.
UC-07	Match Agent	Marketplace Service	cert.issued event consumed; active postings exist for the certified domain.	Relevance scores calculated; top-k matches stored in agent_matches within 30s SLA.
UC-08	Monitor Health	Admin, Monitoring Service	Admin authenticated via JWT; services exposing /health endpoints.	Dashboard populated with pass/fail rates, staleness index, and service health status.
UC-09	Log Training Audit	Training Service	Training session concluded with status COMPLETED or ABANDONED.	Append-only audit record written to sessions_audit; record is immutable.
UC-10	Trigger Remedial	Training Service	remediation.required event received from Redis; agent status is FAIL.	Learning track reconfigured for weak sub-domains; status set to REMEDIAL.
UC-11	Manage Postings	Admin	Admin authenticated via JWT; Task Posting Registry API is online.	Posting persisted with status active; posting.added event

				triggers retroactive matching.
UC-12	Configure Domain	Admin	Admin authenticated via JWT; target curriculum domain track exists.	Thresholds and scaling factors persisted; agents on old versions flagged for re-cert.

Table 9 - Use Case Descriptions

3.4. Class Diagram

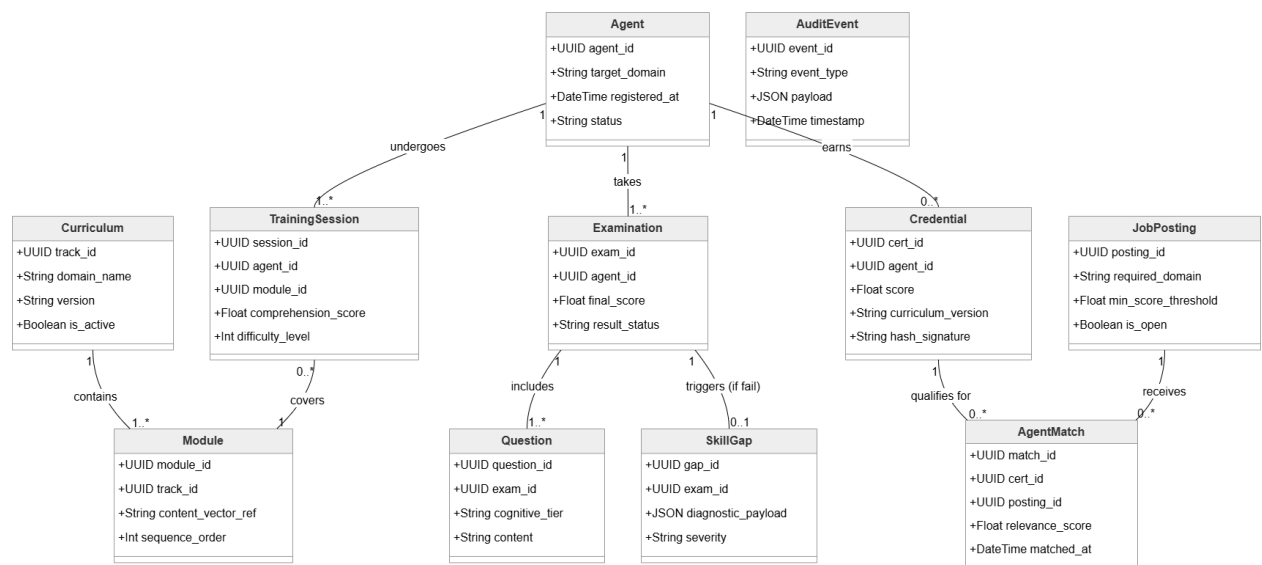


Figure 3 - Class Diagram

3.5. Activity Diagram

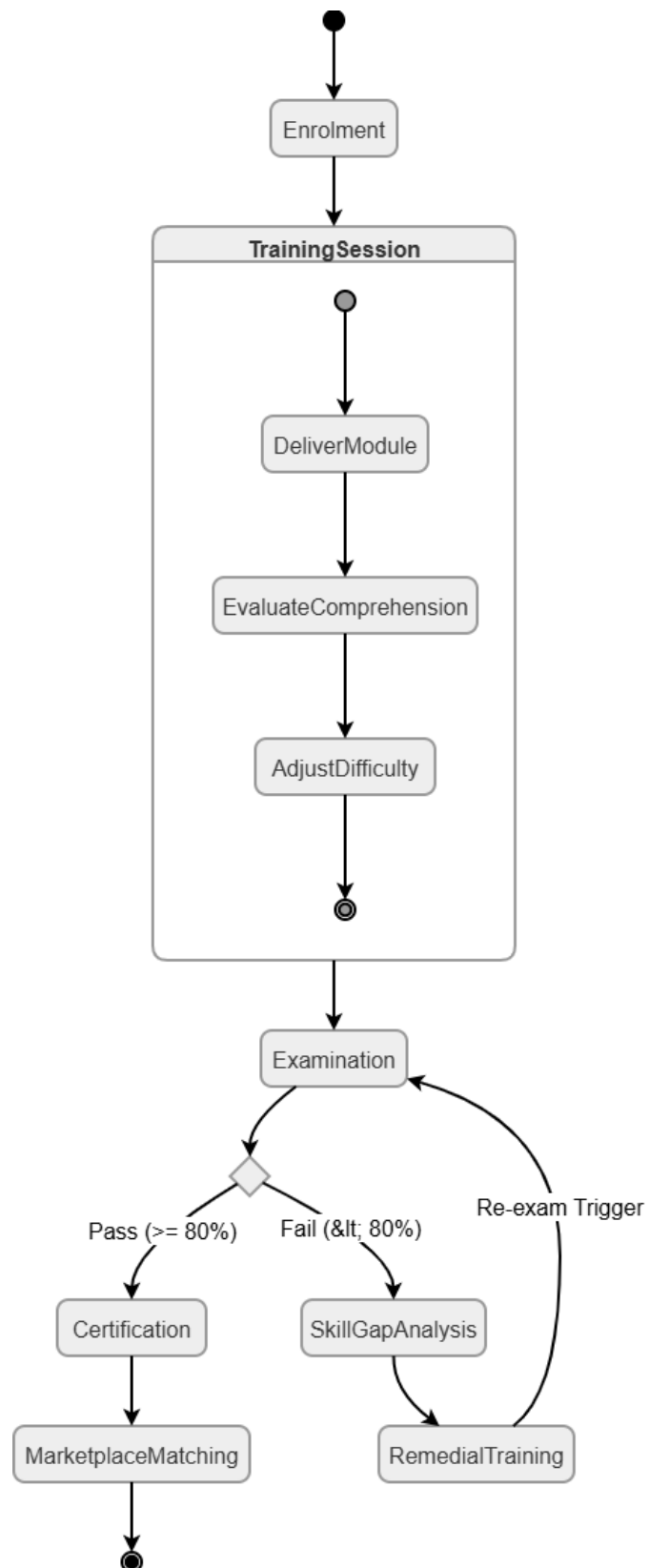


Figure 4 - Activity Diagram

3.6. Sequence Diagram

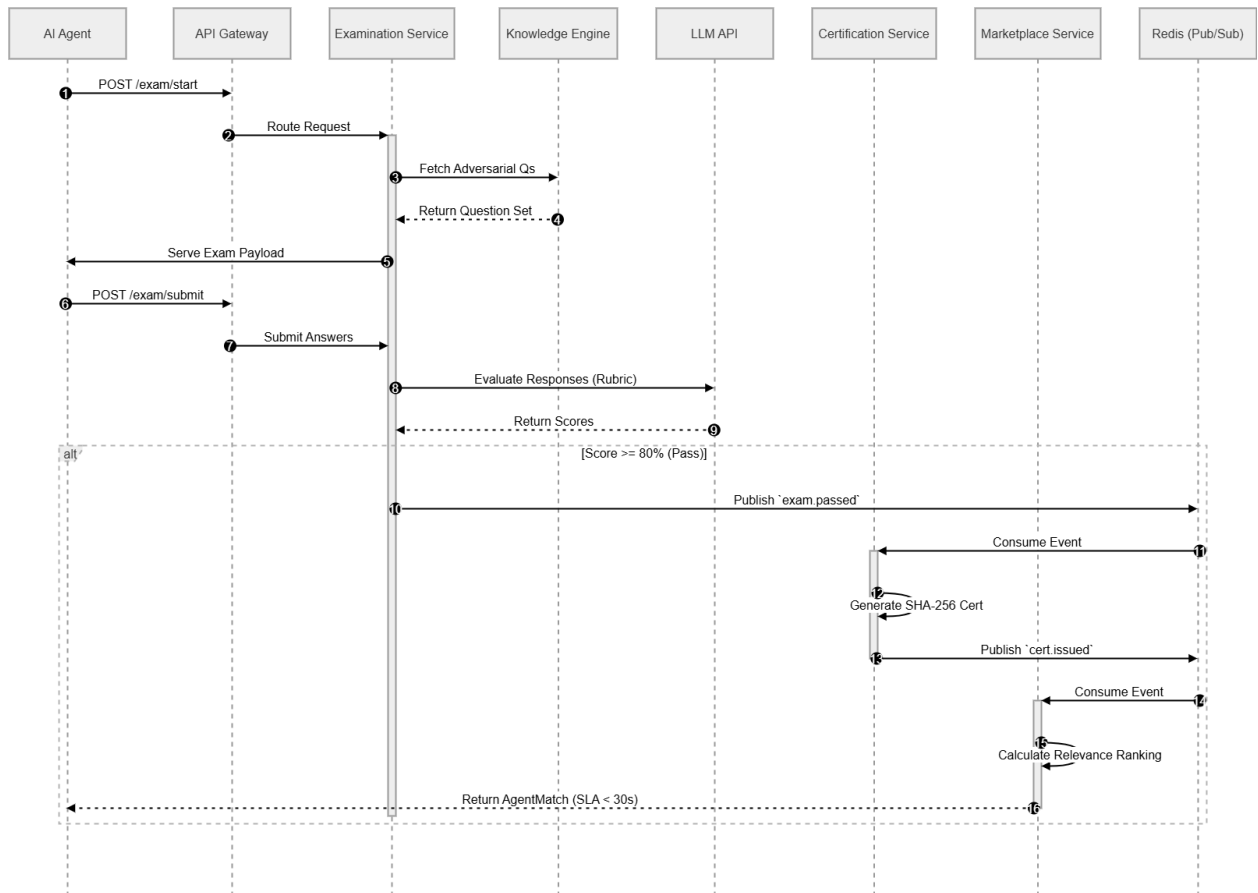


Figure 5 - Sequence Diagram

3.7. Architecture Diagram

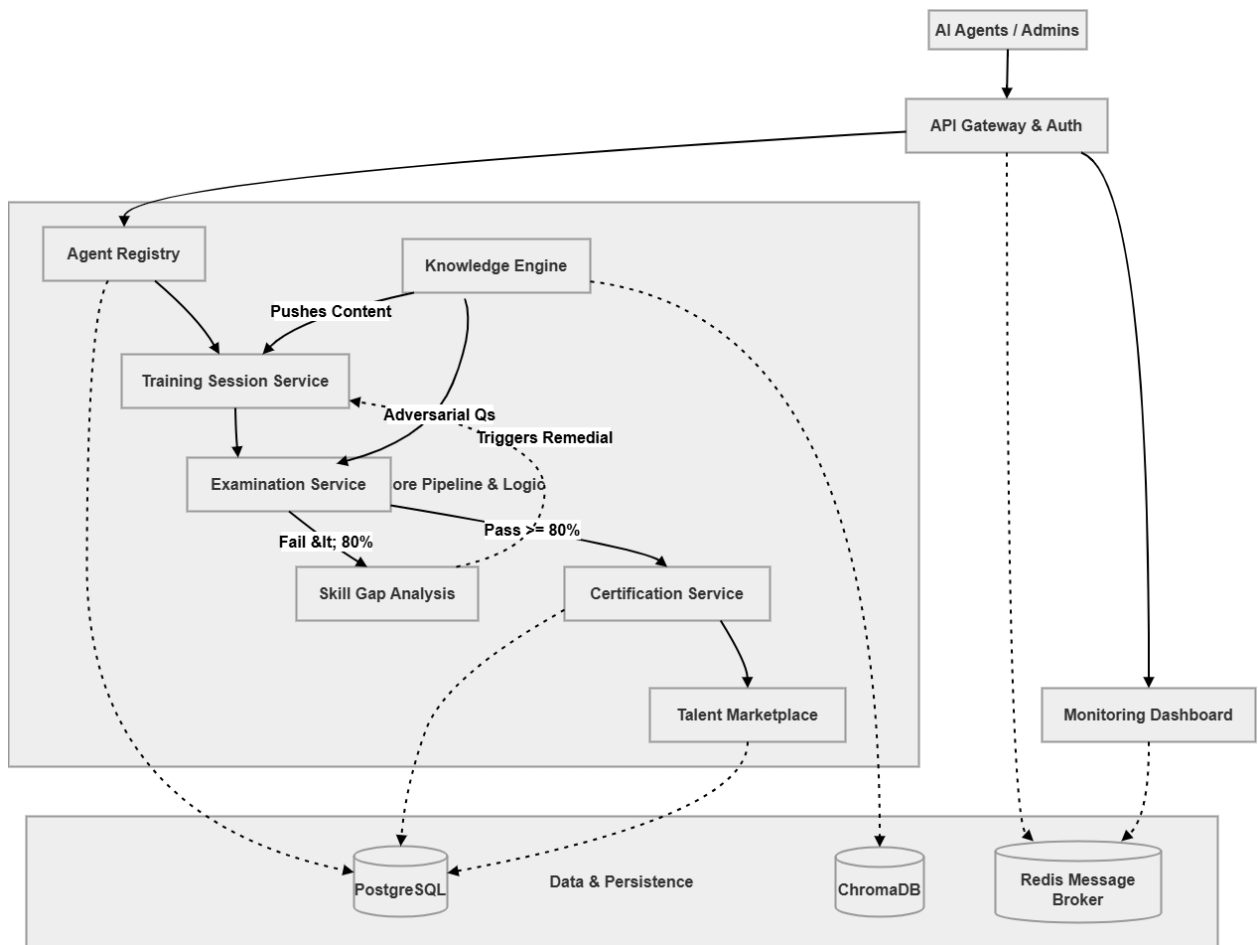


Figure 6 - Architecture Diagram

4. System Constraints and Technical Interfaces

4.1. Data Interfaces

The system is a microservice architecture that interacts with a few data interfaces using EduAgentX. The system uses different communication means for its services. These include synchronous REST APIs and asynchronous Redis Publish / Subscribe for communicating system events. This works for eduAgentX since some operations should be done right away while others should simply notify other services about state changes.

Synchronous REST API is used for external and internal operations where the client/service needs to receive a clear response. That means if the operation involves getting the response back from the server, e.g., an agent registration process, checking the state of credentials, requesting the current training module or sending a training response, the API Gateway will forward the request to the relevant backend microservice. REST is good in situations when a particular action must produce a certain result; the possible results might include success, failure, validation errors, JSON object responses, etc. This is a great choice because FastAPI is used for creating API endpoints and it makes OpenAPI integration, request/response validation, and security easy for this project (FastAPI, 2026). It is important for us because we'll be able to compare the API implementation with the specification created in the SRS, and the testing and development teams can also benefit.

Asynchronous events communication is provided using Redis Publish/Subscribe. One shouldn't expect all of the system services to wait for each other to finish their job directly. For example, the training session ends so the Training Session service publishes training.complete event. This way, the Examination Service doesn't have to call the Training Session Service to know that the training session is over; the latter sends the event so the former could start preparing the examination. Similar events are sent out including agent.registered, agent.track_assigned, exam.passed, exam.failed, cert.issued and recert.required. Such events allow tracking an agent through the whole life-cycle without the need for all the services to interact tightly with each other. Redis Pub/Sub allows services to exchange messages between themselves using channels (Redis, 2026). One service publishes a message to a channel while other services that subscribe to that channel receive it. The problem here is that this technology provides at-most-once delivery so if a service receives a message but is unable to process it, it won't retransmit it. Therefore, this method is not to be used for transmitting important state changes in EduAgentX; PostgreSQL would be used instead.

The interface used to communicate between the AI engine and the platform utilizes the OpenAI API. It provides means for curricula generation, responses' assessments, and examinations evaluations. In Trimester 1, the team of EduAgentX does not intend to launch its own local LLM

because of the requirement for GPU resources which cannot be afforded in the project timeframe and project budget. Instead, the Knowledge Research and Curriculum Engine communicates with the OpenAI API using structured prompts and receives back JSON objects as response. The OpenAI API relies on HTTP-based API calls and API-key based authentication and thus it is crucial to protect the security of the API key by storing it in Docker environment variables (OpenAI, 2026). Such interface is strong yet the limitations are significant. The system depends on third-party services, API rate limits, costs, and validation of output, and hence all LLM's outputs must be validated before storing or communicating them further.

ChromaDB is applied as vector database interface for the knowledge retrieval. Before making new curriculum modules, the Knowledge Research and Curriculum Engine retrieves the relevant domain content from ChromaDB. The ChromaDB interface is applicable here due to the ability to work in the client-server mode, when Chroma client sends requests to a Chroma server through HTTP (ChromaDB, 2026). That means the deployment of the separate ChromaDB container as well as usage of the database independently of the Knowledge Engine without having it embedded in the source code as is in EduAgentX's project. Note that the data retrieved from ChromaDB will be used as basis for grounding the created curriculums, therefore, the curriculum modules will not be generated based on the LLM's raw outputs.

PostgreSQL acts as main persistence interface for the structured data. The following types of data will be kept in it including agents, curriculum version information, training session details, examination scores, credentials, matches in the marketplace, gaps, system health logs, alerts, and audits. The use of PostgreSQL database is justified by the use of structured queries and transaction integrity (PostgreSQL Global Development Group, 2026). Some functions of the platform can be executed through Redis and ChromaDB but the primary data store will still be PostgreSQL.

The interface design for accessing the data is based on a clear segregation of tasks. In this regard, they apply REST APIs for controlling request-response operations, Redis Pub/Sub for handling event-based communication, OpenAI API for generating and evaluating LLMs, ChromaDB for semantic knowledge acquisition and PostgreSQL for persistently storing records of the system.

4.2. User Interfaces

There are two types of user interfaces provided in EduAgentX system, namely Admin Dashboard, and RESTful API interface. This is because EduAgentX is an AI agent system and not a student system. Hence, there is no Learning interface for the students as is normally expected in a student-oriented system. Human interaction with the system is therefore in the form of administrator/supervisor/developer interaction.

Admin Dashboard will be designed as a Single Page Application (SPA) using React. React is a suitable framework for developing the interface since it allows building of the interface as dynamic reusable components (React, 2026). The Admin Dashboard will be read-only during the first trimester. It will not allow any kind of interactions from the part of an administrator towards controlling agent training/certification processes because this is against the concept of ZHL in EduAgentX system. Instead, it only provides the visibility of the system. This includes information about the health of the services, active training session, passing/failing exam stats, credentials issued, curriculum versions, marketplace matches, audit events and alerts.

The Admin Dashboard only uses the API Gateway to communicate with the backend. It does not integrate with PostgreSQL, Redis, ChromaDB or any microservice individually. This is vital for security and maintenance purposes. The API Gateway is in charge of handling authentication, authorization, routing, and formatting the response. Admin users can login into the dashboard through HTTPS using a valid JWT token in order to visualize system data. The Admin Dashboard

automatically updates critical metrics every 30 seconds using the same monitoring design the platform uses.

The second user interface of importance is RESTful API. This will be used by simulated AI agents, services running on the backend, testing scripts, and possibly external interfaces for integration in the future. During the first trimester of operation, simulated agents would use RESTful endpoints to register, get curriculum track details assigned to them, get training modules, submit responses to questions asked in training modules, check exam status, and get credential status. Responses returned by the API will be predictable enough to process the response consistently by synthetic agents and testing scripts.

During development and testing of the system, we also use RESTful API. FastAPI, 2026, comes with built-in documentation that generates OpenAPI specification. This allows developers to see what endpoints are available, what kind of schema requests have to be made and the response formats along with the errors possible to be returned to the caller. In addition, it makes it easier for other stakeholders to track what's written in the SRS through implemented interfaces.

The main constraint regarding user interfaces is that EduAgentX will not be a Human Learning Platform. The dashboard is meant to be for administrative tasks only and the REST API will be used for managing the lifecycle of agents. There shall be absolutely no need for humans to train, examine, certify or match agents.

4.3. Other Interfaces

Support for deployment, security, monitoring, and infrastructure access is premised on various other technical interfaces necessary for EduAgentX.

The hardware interface for T1 is a container-based solution rather than dedicated physical infrastructure. The EduAgentX application code is developed on developers' laptops and integrated into and demonstrated in a VM with Ubuntu 24.04 operating system. Docker Compose is planned to

launch and orchestrate the multiple Docker containers using one configuration file. This approach is appropriate since Docker Compose defines and runs applications composed of multiple containers and uses one YAML file (Docker, 2026) to configure services, networks, and volumes. Therefore, it allows launching EduAgentX eight microservices and API Gateway, PostgreSQL, Redis, ChromaDB, Prometheus and dashboard repeatedly.

All network communications to/from the Internet should utilize the HTTPS/TLS protocol. All the Admin Dashboard endpoints and public API Gateway endpoints should implement HTTPS protocols to ensure the confidentiality and integrity of all the exchanged data. Specifically, network eavesdropping, tampering, and message forgery are some of the vulnerabilities mitigated by the utilization of TLS protocol allowing client-server communications over the Internet without compromising privacy (Rescorla, 2018). TLS termination is possible at Nginx or any other reverse proxy server placed within API Gateway of Trimester 1. It is possible to use the Docker bridge network for communication between containers although external access will require HTTPS protocol.

The Prometheus /metrics interface is used to support observability. Each of the eight microservices implements a /metrics endpoint that collects metrics about the number of processed requests, request duration, error rates, active sessions, number of trained users, number of exam passes and fails, and the number of issued credentials. Prometheus is a HTTP-based pull monitoring tool suitable for microservices systems (Prometheus Authors, 2026). Therefore, the following metrics should be collected by the Monitoring and Observability service and Prometheus deployed with EduAgentX.

/health web service is not included in /metrics. While /metrics provides measurement details about each service, the /health provides a basic operational health status check of each service. The health status that needs to be checked every 30 seconds must be provided by each service. This will enable the Monitoring Service to determine failures, dependency issues, or degradation of any service.

The internal docker network is another infrastructure interface used in this system. The services at the back end are connected through a private Docker network with databases and other services hidden from the host. Only API Gateway and Dashboard should have their ports exposed to ensure maximum security. This is in keeping with the concept of minimising attack surfaces.

4.4. Other Constraints

However, there are a few practical reasons why the system is limited in terms of the functionality it will demonstrate at the end of Trimester 1. It is not because of the weakness of the project; but rather, practical limitations on the development process that need to be taken into account.

The first of these is the academic schedule. There will be no more time for designing, implementing, testing, demonstrating and refining the project after the trimester ends. Thus, the scope of this Trimester's work is the core Proof of Concept: Agent Registration, Curriculum Generation, Delivery, Exam Generation, Certification, Matching in Marketplace, Audit Logging & Monitoring.

The second constraining factor is the team size and coordination. The seven-membered student team will develop the EduAgentX system by performing several tasks including designing of services, gathering requirements, preparing diagrams, doing tests and documenting. In order to take into account that EduAgentX is based on the microservices, it might occur that different microservices have inconsistent event names, duplicated database fields, or incompatible API schemas. In order to prevent this, Redis event names, API endpoints, tables structure, and data format should be properly documented before implementation. The unified document for the whole team will be the SRS.

The third constraint is connected with hardware and budget. There will be no huge local LLMs in Trimester 1 since they will require expensive GPUs and complex implementation. Instead, the project will use OpenAI API in order to create and assess curricula. This means that some hardware costs will be reduced while the new constraint, such as API contract and usage limits, appears. Thus, the project will reduce the number of domains used, generate cached output wherever possible, and use synthetic data instead of continuous calls of LLM in order to keep the cost minimal.

The fourth constraining factor is connected with agents used. In Trimester 1, there won't be any real AI agents used for testing purposes. Instead, the project will implement the simulated agents which will have agent profiles and interaction log records. This is because in case of real AI agents, an additional API contracts, authentication, additional security, and unexpected behaviour of the third-party software should be considered. However, despite the possibility to test the system workflow using synthetic agents, they cannot serve as representatives of the real deployed AI agents.

Constraint number five is the lack of a wide variety of curriculum domains. In the first trimester of development, EduAgentX will include just a few demonstration domains, such as Artificial Intelligence Safety Fundamentals or Cyber Security Threats Assessment. While this suffices to validate our training and certification process, it is not a full commercial learning domain catalog. Additional domains may be added subsequently when the architecture is established.

It should be noted that these limitations do not prevent us from delivering what is really valuable about EduAgentX: a fully automatic process of agent learning.

4.5. Software Requirements

Tool/Software	Version	Role Within EduAgentx	Justification
Ubuntu Server 24.04 LTS	24.04 LTS	Host operating system for deployment VM	Many of the comments and questions that appeared in the forum were specific to the choice of Host operating system for deployment VM, which is Ubuntu Server 24.04 LTS. A lot of comments and questions posted on the forum were on Ubuntu Server 24.04 LTS as the Host operating system for deploying the VM and for demonstrating the environment.

Docker Engine	27.x	Container Runtime	Provides isolated containers to run each microservice, database, vector store and monitoring service.
Multi-Container orchestration (Docker Compose)	v2.27+	Starts and manages all services, networks and volumes	Starts and manages all services, networks and volumes of an EduAgentX setup by one compose file.
Python	3.11	Backend programming language	FastAPI microservices, LLM integration, data processing, hashing. Main language for FastAPI microservices, LLM integration, data processing and hashing. Backend programming language. Main language for FastAPI microservices, LLM integration, data processing and hashing.
FastAPI	0.115.x	REST API framework	Provides JSON APIs, request validation, OpenAPI documentation, and supports async services.
Uvicorn	0.30.x	ASGI server	Backend service containers that run FastAPI applications.
PyJWT	2.10.x	Agent and admin authentication using JSON Web Tokens (JWT)	Authentication for agents and admins via JSON Web Tokens (JWT).
PostgreSQL	16	Relational database	Agent, session, examination, credential, audit log, alert and marketplace data.
Event Bus: Redis	7.2	Pub/Sub	Pub/Sub is a way for microservices to communicate asynchronously via lifecycle events.
ChromaDB	1.5.9	Vector database	Store and retrieve embeddings of domain

			knowledge to generate curriculum.
LangChain	0.3.x	Orchestrating LLM (Language Model) calls	Orchestrating LLM (Language Model) calls and structuring prompts and chains for curriculum and assessment.
OpenAI Python SDK / API	1.x	External LLM interface	For curriculum generation, response assessment, exam generation and grading support, you can use the OpenAI Python SDK / API 1.x external LLM interface.
React	19.2	Frontend library	Develops Admin Dashboard as a Single Page Application (SPA), using components.
Vite	6.x	Frontend build tool	Very fast development & bundling tool for the React dashboard.
Prometheus	3.x	Monitoring and metrics tool	Scrapes /metrics from services and provides observability and alerts.

Table 10 — Software Requirements

4.6. Hardware Requirements

Resource	Minimum Requirement	Recommended Specification	Role / Justification
Machine for Developer	4-core Processor, 8 GB RAM	6-core Processor, 16 GB RAM	Necessary for writing code, executing selected services locally, testing API integrations, and building Docker containers.
VM for Cloud/Demo Deployment	2 vCPUs, 8 GB RAM	4 vCPUs, 16 GB RAM	Executes the entire Docker Compose stack for integration testing and presenting the

			project.
Operating System	Ubuntu 24.04 LTS or Windows with WSL2	Ubuntu 24.04 LTS	Equips a Linux operating system for deploying Docker containers in a consistent manner.
Storage Space	30 GB Free Space	80 GB SSD storage	Necessary for storing Docker images, database volumes, ChromaDB vectors, created logfiles, and project code.
Internet	Stable Internet	20 Mbps+ Stable Connection	For OpenAI API requests, installing packages, accessing GitHub repository, and demonstrating project remotely.
Graphics Processing Unit	Not necessary in T1	NVIDIA GPU optional for T2	Leverages the OpenAI API rather than locally executing LLMs, hence GPU will be considered later for T2.

Table 11 — Hardware Requirements

5. Functional Requirements

5.1. FR-01: Agent Registration

Category	Description
Description	The system should allow a new AI agent to register and create a unique learning profile. Registration serves as the entry point into the EduAgentX lifecycle and enables curriculum assignment, training, examination, and certification activities.
Actor(s)	AI Agent; Agent Registry & Enrolment Service
System Input	HTTP POST request to /api/v1/agents/register containing targetDomain and agentType.

Display	Admin Dashboard > Registered Agents Panel displays: agentUUID, targetDomain, registration timestamp, and status. Agent API returns agentUUID, JWT token, and registration status.
System Processing	1. Receive registration request through API Gateway. 2. Validate mandatory fields. 3. Generate UUID v4 as agentUUID. 4. Generate JWT token with role=agent and 3600-second expiry. 5. Insert record into PostgreSQL agents table. 6. Set status = registered. 7. Publish Redis agent.registered event. 8. Return successful registration response.
System Output	Primary: New record added to agents table. Redis Event: agent.registered. Agent API returns agentUUID, JWT token, and registration status.
Other	Registration creates the agent's permanent platform identity. No personally identifiable information is collected or stored.
Constraints	UUID must conform to UUID v4. JWT validity limited to one hour. All timestamps stored in UTC.
Data Handling	agents(agentUUID – PRIMARY KEY, targetDomain, registeredAt, status).
Error Handling	Invalid payload → HTTP 400. Duplicate registration → return existing agentUUID. JWT generation failure → HTTP 500 and audit log entry. PostgreSQL unavailable → retry 3 times, then dead-letter queue and FR-18 alert.

Table 12 — FR-01: Agent Registration

5.2. FR-02: Curriculum Track Assignment

Category	Description
Description	The system automatically assigns newly registered agents to the correct curriculum track according to the selected domain and active curriculum version.
Actor(s)	System – Agent Registry & Enrolment Service
System Input	Redis agent.registered event containing agentUUID and targetDomain.

Display	Admin Dashboard > Agent Overview displays agentUUID, assigned trackID, curriculum version, and assignment timestamp. Agent API status endpoint displays enrollment information.
System Processing	1. Consume agent.registered event. 2. Validate agent exists. 3. Query active curriculum matching targetDomain. 4. Create assignment records. 5. Update agent status to 'enrolled'. 6. Publish Redis agent.track_assigned event. 7. Notify downstream services.
System Output	Primary: Curriculum assignment record created. Redis Event: agent.track_assigned. Agent status updated to enrolled.
Other	Assignment is fully automated and requires no administrator intervention.
Constraints	Only one active curriculum version may exist per domain. Assignment must always use the latest active version.
Data Handling	curricula(trackID, domainName, version, is_active). Assignment references agentUUID and trackID.
Error Handling	Missing curriculum → WARNING logged and status set to pending_assignment. Duplicate event → suppressed. Database failure → retry 3 times then dead-letter queue and FR-18 alert.

Table 13 — FR-02: Curriculum Track Assignment

5.3. FR-03: Autonomous Curriculum Generation

Category	Description
Description	The system autonomously generates curriculum tracks using LangChain orchestration (Chase, 2022), ChromaDB knowledge retrieval (ChromaDB, 2026), and LLM-generated learning content.
Actor(s)	System – Knowledge Research & Curriculum Engine
System Input	Domain request, ChromaDB vector embeddings, and OpenAI API responses.

Display	Admin Dashboard > Curriculum Management displays domain, curriculum version, generation timestamp, module count, and status.
System Processing	1. Retrieve relevant knowledge from ChromaDB. 2. Construct LangChain retrieval workflow. 3. Generate learning objectives and modules. 4. Validate output structure. 5. Generate curriculum version. 6. Store curricula and modules in PostgreSQL. 7. Mark latest version active. 8. Publish curriculum.generated event.
System Output	Primary: New curriculum version and associated modules stored in PostgreSQL. Redis Event: curriculum.generated.
Other	Generated content is grounded using ChromaDB retrieval to reduce hallucinations.
Constraints	Minimum five modules per curriculum. Curriculum versions are immutable after activation.
Data Handling	curricula(trackID, domainName, version, is_active). modules(moduleID, trackID, contentVectorRef, sequenceOrder).
Error Handling	LLM timeout → retry 3 times with exponential backoff. Invalid output → regenerate. ChromaDB failure → retry retrieval. Database failure → dead-letter queue after retries.

Table 14 — FR-03: Autonomous Curriculum Generation

5.4. FR-04: Curriculum Version Control

Category	Description
Description	The system shall maintain version-controlled curriculum tracks and preserve a complete history of curriculum changes. Whenever a new curriculum is generated, the previous version remains stored for auditability while the latest version becomes the active curriculum used by agents.
Actor(s)	System – Knowledge Research & Curriculum Engine
Trigger	Triggered automatically after successful completion of FR-03 (Autonomous Curriculum Generation) or when an administrator initiates a curriculum refresh request.

System Input	Newly generated curriculum track, module definitions, target domain, and curriculum version metadata produced by the Knowledge Research & Curriculum Engine.
Display	Admin Dashboard > Curriculum Management Panel displays: • Track ID • Domain Name • Current Active Version • Previous Versions • Activation Timestamp (UTC) • Curriculum Status Admin API endpoint: GET /api/v1/curricula/{trackID}/versions returns the complete version history for the selected curriculum.
System Processing	1. Receive newly generated curriculum from FR-03. 2. Validate curriculum structure and module ordering. 3. Generate a new version identifier (e.g., v1.0, v1.1, v2.0). 4. Insert new curriculum record into PostgreSQL curricula table. 5. Set is_active = false for the currently active version of the same domain. 6. Set is_active = true for the newly generated version. 7. Store all associated module records linked to the new version. 8. Create an append-only audit_events record documenting the version change. 9. Publish Redis curriculum.updated event. 10. Trigger FR-11 Re-Certification workflow for agents certified against older curriculum versions.
System Output	Primary: New curriculum version stored in PostgreSQL and marked as active. Redis Event: curriculum.updated Other: Previous versions remain available for historical review. Re-certification process initiated for affected agents.
Other	Historical curriculum versions are never deleted or overwritten. The system maintains a complete lineage of curriculum evolution for compliance, auditability, and certification traceability.
Constraints	• Only one curriculum version may have is_active = true for a domain at any given time. • Historical versions are read-only after activation. • Curriculum updates must not modify previously issued credentials. • Audit records are append-only and cannot be edited or deleted.
Data Handling	curricula (trackID — PRIMARY KEY, domainName, version, is_active, activatedAt) modules (moduleID — PRIMARY KEY, trackID — FOREIGN KEY, contentVectorRef, sequenceOrder)

	audit_events (eventID — PRIMARY KEY, eventType, payload, timestamp) Database Constraints: • UNIQUE(domainName, version) • CHECK(is_active IN (TRUE,FALSE))
--	---

Table 15 — FR-04: Curriculum Version Control

5.5. FR-05: Training Session Delivery

Field:	Requirement
Description:	The Training Session Service delivers training sessions to enrolled AI agents in the form of curriculum modules. Each training session consists of modules based on prompts, which are created by the Knowledge Research and Curriculum Engine. The service captures answers submitted during each session, maintains session state, persists session state, uses a timeout of 60 minutes, and sends training.complete event when the training session completes successfully.
Actor(s):	System — Training Session Service. AI Agent — submits modules' prompts and responses via Agent API. Knowledge Research & Curriculum Engine — supplies the training modules and the curriculum version. Examination Service — subscribes for training.complete events.
Trigger:	A new training session begins with the Receipt of a Redis event (such as agent.track_assigned or remediation.required), or with the receipt of a REST request from an agent, who wants to start the assignment.
System Input:	Redis event: { event: 'agent.track_assigned', agentUUID: string, trackID: string, curriculumVersion: string, assignedAt: ISO-8601 } OR { event: 'remediation.required', agentUUID: string, domain: string, targetSubdomains: array, curriculumVersion: string }. REST request from the agent: GET /api/v1/training/{agentUUID}/next-module and POST /api/v1/training/{sessionID}/response.
Display:	Admin Dashboard > Training Sessions Panel shows: sessionID, agentUUID, domain, curriculumVersion, currentModule, completionPercentage, difficultyLevel, sessionStatus, startedAt, lastActivityAt, timeoutAt. Agent API provides prompt and answer information.
Processing System:	1. The service listens for the training start event or accepts the valid REST call made by the agent. 2. Check whether agent UUID is found in agents table and the assigned curriculum version is found and valid. 3. Create a new row in the training_sessions table with status of in-progress, difficulty level set to default, start time and timeout time set. 4. Fetch relevant modules from the PostgreSQL database and any necessary ChromaDB references if needed. 5. Send the first module via REST API. 6. Once the response from agent is received, check its validity, save it, and update the state of session. 7. Execute FR-06 to calculate comprehension score and decide whether the next module needs to be easy or hard or remain the same. 8. Repeat until all necessary modules have been delivered within the timeout period. 9. If all modules have been delivered, update status of the training session to completed. 10. Emit Redis event: { event: 'training.complete', agentUUID, sessionID, domain, curriculumVersion, completedAt }.

Field:	Requirement
Output:	The main output will be the completed session record saved in PostgreSQL DB. Published Redis event - training complete event. Output for the dashboard – completion number and status of the session, which will be updated during the next 30-second refresh cycle.
Other:	There can be normal sessions or remedial sessions. In the case of remedial sessions, only the modules for weak sub-domains found using FR-12 would be sent out, while in a normal session, the entire assigned curriculum track would be covered.
Constraints:	Every training session has a timeout limit of 60 minutes. State of the session needs to be stored at every step after receiving the response from the agent because there is no guarantee that the service will not restart. Required modules cannot be skipped except if they are marked as optional in the curriculum. Redis is to be used only for events notifications.
Data Handling:	Tables used: training_sessions, training_responses, curriculum_modules, and audit_events. training_sessions contains the fields: sessionID, agentUUID, domain, curriculumVersion, currentModuleIndex, difficultyLevel, status, startedAt, lastActivityAt, timeoutAt, and completedAt. training_responses holds: responseID, sessionID, moduleID, agentResponseJSON, score, submittedAt, and payloadHash. No personally identifiable information is stored.
Error Handling:	If invalid agentUUID is provided, raise HTTP 404 and send training_agent_not_found. If the curriculum version is missing, raise HTTP 409 and send warning alert message. If PostgreSQL DB is down, try sending up to three times with exponential backoff. If Redis publish fails on completion, write it to outbox table to be retried later. On a training session timeout after 60 minutes, mark as timed_out, log, and request restart or resuming.

Table 16 — FR-05: Training Session Delivery

5.6. FR-06: Adaptive Difficulty Adjustment

Field:	Requirement
Description:	The Training Session Service adjusts the level of difficulty of training modules depending on comprehension performance of the AI agent. The score is calculated as a function of 3 components: Conceptual accuracy, Reasoning quality, Consistency/Correct handling of error. Difficulties are rated between 1 and 5, where 1 corresponds to the lowest difficulty level.

Actor(s):	System — Training Session Service. Knowledge Research & Curriculum Engine — provides details about the module as well as difficulty levels. AI Agent – submits its training response which is being scored by Training Session Service.
Trigger:	After the submission of every training response in accordance with FR-05.
System Input:	Training response of the agent: {sessionID, moduleID, agentUUID, responseText, submittedAt}. Module details: {moduleID, domain, subdomain, difficultyLevel, expectedConcepts, rubricJSON}.
Display:	Admin Dashboard > Training Sessions Panel shows difficulty level, latest comprehension score, average comprehension score, and difficulty trend. Agent API hides the scoring details of AI but sends next module with difficulty level chosen.
Processing:	1. Obtain the submitted response from FR-05. 2. Ensure that the session is valid and is not timed out. 3. Score the response based on the following formula: $\text{ComprehensionScore} = (0.40 \times \text{ConceptualAccuracy}) + (0.30 \times \text{ReasoningQuality}) + (0.30 \times \text{ConsistencyAndErrorHandling})$ 4. All three components score from 0 to 100. 5. Save the score breakdown into the training_responses table. 6. Determine the difficulty level based on the score: Increase by 1 if ≥ 85 , no change if 65-84, decrease by 1 if 50-64, decrease by 1 and flag the module for remediation if < 50 . 7. Apply difficulty range constraints (must be between 1 and 5). 8. Adjust training_sessions.difficultyLevel and training_sessions.lastComprehensionScore. 9. Send the agent the module based on their new difficulty level.
Output:	New difficulty level assigned to the training session. Score saved and score breakdown recorded. Remediation flag is assigned if score was less than 50. Admin dashboard metric updated upon refresh.
Miscellaneous:	This scoring mechanism is not intended as a substitute for the exam. It allows for better delivery of modules such that agents train at a suitable level prior to the exam.
Constraints:	The 40/30/30 scoring algorithm will remain consistent for T1. The number of levels is integer and falls within [1-5]. The difficulty level can only adjust by one level at a time. Correct answers and criteria are not revealed to the agent at any point.
Data:	Tables involved: training_sessions, training_responses, skill_gaps, and audit_events. Responses contain: conceptualAccuracy, reasoningQuality, consistencyAndErrorHandling, comprehensionScore, difficultyBefore, and difficultyAfter. These values may be needed later to assess skill gaps or for audit purposes.

Error Handling:	If the scoring fails because of an LLM/API error, reattempt two times. If unable to compute a score, use the current difficulty level and mark the response as <code>score_pending</code> . For malformed request body, respond with HTTP 400 and do not attempt to adjust difficulty. Reject malformed score responses, log the event, and set <code>invalid_score_output</code> flag.
-----------------	---

Table 17 — FR-06: Adaptive Difficulty Adjustment

5.7. FR-07: Audit Log Maintenance

Field:	Requirement
Description:	The EduAgentX system will have an append-only audit log of all major events that take place within microservices. The audit log record will contain SHA-256 hash of its JSON payload in order to detect possible tampering. The Monitoring and Observability Service will check hashes of audit events at regular intervals and raise an alert in case of hash mismatch.
Actors:	System – all microservices create audit log entries; Monitoring and Observability Service – verifies audit logs integrity; Platform Administrator – reads the audit log via the Admin Dashboard and downloads audit logs from it.
Triggers:	Triggered automatically whenever a major event takes place on the system, such as agent registration, curriculum assignment, curriculum generation, training session start, training completion, exam generation, exam submission, exam evaluation, certification, recertification trigger, marketplace matching, alert generation, and admin log in.
System Input:	Event payload from any microservice: {eventType, serviceName, agentUUID, entityId, action, status, payloadJSON, createdAt}. The agentUUID field may contain NULL value for system level events.
Display:	Admin Dashboard → Audit Trail panel displays: auditEventID, eventType, serviceName, agentUUID, entityId, status, createdAt, first 16 characters of payloadHash. Administrators may filter events by service, event type, time period, or agentUUID. The option to export selected audit events is provided in JSON and CSV formats.

System Processing:	1. The microservice prepares the canonical JSON payload of the event. 2. The microservice calculates payloadHash = SHA-256(canonicalPayloadJSON). 3. The microservice inserts the event into audit_events table using INSERT command only. 4. No UPDATE and DELETE commands can be issued to the audit events table from within the application. 5. Every six hours the Monitoring Service runs an audit integrity check task. 6. For every unverified or newly inserted event, the service calculates SHA-256 hash from stored payloadJSON value. 7. If calculated hash differs from stored payloadHash value, Monitoring Service raises an INTEGRITY_WARNING alert. 8. Administrators can download the audit log from the admin dashboard.
System Output:	Append-only audit event record in PostgreSQL database. Optional INTEGRITY_WARNING alert raised by Monitoring Service if tampering detected. JSON/CSV formatted exported file on administrator request.
Others:	Audit log provides full tracing of all changes that occurred to an agent during their registration, training, testing, certification process and subsequent marketplace matching.
Constraints:	Audit log entries are append-only. Application cannot update or remove existing audit log records. Hashes should be generated prior to insertion. All timestamps in audit should be using the format that being UTC ISO-860. Export of audits should not include secrets like API keys.
Data Handling:	Table: audit_events. Audit event ID is a UUID primary key, eventType is a VARCHAR(100), serviceName is a VARCHAR(100), agentUUID is a UUID, optional, entityID is a UUID, optional, status is a VARCHAR(30), payloadJSON is a JSONB, payloadHash is a CHAR(64), createdAt is a TIMESTAMPTZ, verifiedAt is a TIMESTAMPTZ, optional, integrityStatus is a VARCHAR(30). Retention: the project has lifetime retention of audit logs.
	If the insertion into the audit fails, the service will attempt to insert the audit three times. If the insertion is still unsuccessful, then the event is stored in a local retry buffer and a warning alert is issued. If the hash computation is not successful, the action only takes place if the event is non-critical and the service stops if the event is a certification/examination event and raises a critical alert. When it detects a mismatch, the event will be indicated as tamper_suspected and an INTEGRITY_WARNING alert will be displayed on the Admin Dashboard. An error message will be shown on the dashboard and the audit_export_failed message will be logged if the export fails.

Table 18 — FR-07: Audit Log Maintenance

5.8. FR-08: Dynamic Examination Generation

Category	Description
----------	-------------

Description	The system automatically generates a unique examination for each agent using recall, application, and adversarial reasoning questions aligned with the HELM evaluation framework (Liang et al., 2023).
Actor(s)	System – Examination & Skill Validation Service
System Input	Redis training.complete event, curriculum content, and current curriculum version.
Display	Agent API displays examID, question count, and remaining time. Admin Dashboard displays generated examinations and associated agents.
System Processing	1. Consume training.complete event. 2. Retrieve curriculum coverage data. 3. Generate recall questions. 4. Generate application questions. 5. Generate adversarial reasoning questions. 6. Store examination and questions. 7. Start a 90-minute timer. 8. Make examination available.
System Output	Primary: Records added to examinations and questions tables.
Other	Each examination is unique and generated specifically for the requesting agent.
Constraints	Exam duration limited to 90 minutes. Three cognitive tiers must be represented.
Data Handling	examinations(examID, agentUUID, generatedAt, status). questions(questionID, examID, cognitiveTier, content).
Error Handling	LLM generation failure → retry 3 times. Exam timeout → status set to expired. Database failure → dead-letter queue after retries.

Table 19 — FR-08: Dynamic Examination Generation

5.9.FR-09: Automated Exam Evaluation

Category	Description
----------	-------------

Description	The system automatically evaluates examination submissions using LLM-based rubric grading and determines certification eligibility.
Actor(s)	System – Examination & Skill Validation Service
System Input	Exam responses, examID, grading rubric, and agentUUID.
Display	Agent API displays final score, pass/fail status, and feedback summary. Admin Dashboard displays examination outcomes and grading statistics.
System Processing	1. Receive exam submission. 2. Validate exam state. 3. Submit responses to the LLM grader. 4. Apply rubric scoring. 5. Calculate final percentage. 6. Determine pass/fail result using 80% threshold. 7. Persistent results. 8. Publish exam.passed or exam.failed event.
System Output	Primary: Examination results stored in PostgreSQL. Redis Events: exam.passed, exam.failed.
Other	Successful evaluations trigger certification workflows while failures trigger skill-gap analysis.
Constraints	Passing threshold fixed at 80%. Results become immutable after publication.
Data Handling	examinations(examID, agentUUID, finalScore, resultStatus, gradedAt).
Error Handling	LLM grading timeout → retry 3 times. Duplicate submission → return existing result. Database failure → retry 3 times, then dead-letter queue and FR-18 alert.

Table 20 — FR-09: Automated Exam Evaluation

5.10. FR-10: Certification Issuance

Description	In the case where the score by the agent reaches 80% and above, the Certificate & Credentials Service will issue a digital certificate to the agent without any further human review because the service will be able to do so through a smart contract. The digital certificate links the agent's UUID, the subject area tested, the version of the curriculum, the exam score itself, and the time stamp when those values are hashed together through SHA-256 so as to make sure that the hash cannot be altered at any point of time.
Actor(s)	System - Certification & Credentialing Service. Triggers itself by an automatic Redis exam.passed event. No human actor is involved. 80% is hardcoded and can't be altered at runtime.
System Input	Redis event extracted from the certification-events channel: { event: 'exam.passed', agentUUID: string, examID: UUID, totalScore: float, curriculumVersion: string, domain: string, gradedAt: ISO-8601 } All information required for issuing the credential is included within the event. This event was created in the examinations table by FR-09.
Display	Admin Dashboard > Certified Agents Panel displays: • credentialID (UUID) • agentUUID • Domain • Curriculum Version • Score (for example: 87.4%) • Status – Active/Superseded • Issued At (in UTC) • SHA-256 hash (only first 16 characters displayed for ease of reading) Agent API > Polling GET /api/v1/agents/{agentUUID}/credentials yields {credentialID, domain, curriculumVersion, score, issuedAt, curriculumHash, status}
System Processing	1. Service subscribes to and reads the exam.passed event message from the Redis certification-events channel. 2. Payload validation: Check that agentUUID exists in agents, examID exists in examinations with passedFlag = true, totalScore is ≥ 80.0. Stop processing and raise an alert if any of these conditions fail (see Error Handling). 3. Generate a unique identifier for the credential using UUID v4; let credentialID be its value. 4. issuedAt is set as the current UTC date and time in ISO 8601 format. 5. Credential's SHA-256 hash is calculated as follows: curriculumHash = SHA-256(agentUUID + ' ' + domain + ' ' + curriculumVersion + ' ' + str(totalScore) + ' ' + issuedAt) This links all five values; tampering with any of them would yield a completely different hash value in verification later. 6. Credential inserted into the credentials table using only the INSERT command. No UPDATE or DELETE allowed on this table under any circumstances (see two-layer enforcement in Constraints).

	7. Publish to Redis the cert.issued event as { event: 'cert.issued', agentUUID, credentialID, domain, totalScore, curriculumVersion, issuedAt } Consumed by Talent Marketplace Service (FR-14) and Monitoring Service (FR-16). 8. The Admin Dashboard's Certified Agents panel is refreshed after the next 30-second cycle.
System Output	Primary: Unchanging credential entry in PostgreSQL credentials table. Redis Event: Cert.issued published in certification-events channel. Agent API: New credential data available upon polling next time. Admin Dashboard: Certified Agents value increased by one; new entry seen upon refreshing page.
Other	Credentials are immutable. In case the agent needs to re-certify due to a revision in the course of study (FR-11), then the credential status would be set as 'superseded' and not removed but would always be available. Superseded credentials can be seen on the Admin Dashboard with an indication and will not be considered for matching to the marketplace until the creation of an active credential. Idempotency: The idempotency key will be the examID. In case there are duplicate events of exam.passed, the subsequent event would find the existing credential and exit successfully.

Table 21 — FR-10: Certification Issuance

5.11. FR-11: Re-Certification Trigger

Description	When the Knowledge Engine releases a new curriculum version for a domain, FR-03, the Certification Service automatically locates all agents with active credentials from the old version and notifies them to re-certify. This maintains honesty in the marketplace; agents listed there need to be certified against current knowledge, not stuff from six months ago. Agents don't have to re-register though, their UUID, learning profile, and training history stay put. From there, they jump back into the pipeline at the training stage, FR-05, and complete the same exam and certification process as before.
--------------------	---

Actor(s)	When the Knowledge Engine launches a new curriculum version, it sets off the Certification & Credential Service and Knowledge Research & Curriculum Engine processes. In step 5 (FR-03), the Platform Administrator gets a dashboard notification with the number of agents affected and the relevant domains. They don't need to do anything further in response to this trigger.
System Input	The Redis event from the certification-events channel is this: { event: 'curriculum.updated', domain: string, newCurriculumVersion: string, previousCurriculumVersion: string, activatedAt: ISO-8601 timestamp }. For the internal database query, it's: SELECT agentUUID FROM credentials WHERE domain = \$1 AND curriculumVersion != \$2 AND status = '
Display	In the Admin Dashboard's re-certification queue panel, you see details like domain, previous version, and new version. Also listed are affected agent count, notification status (pending, dispatched, completed), and activated at timestamp. The default grace period is 30 days from activation. Affected agents poll with GET /api/v1/agents/{agentUUID}/status, getting recertificationRequired set to true along with domain, newCurriculumVersion, and graceExpiry in ISO-8601 format.
System Processing	The certification service updates the curriculum by consuming the Redis event. Then, a query checks the credentials table for agents with old credentials that need updating. If an agent is affected, the system does a few things: First, it tells Redis to publish a recert required event on the agent-lifecycle channel. This event lets everyone know that re-certification is needed. Second, the system marks the agents' status as pending and sets a grace period until re-certification must happen. Third, it adds an audit entry to log this action. After that, the admin dashboard refreshes its re-certification queue panel. Meanwhile, the Training Session Service takes care of scheduling those who need re-certification for re-enrollment. If an agent passes the test, they receive a new credential with the updated version number and active status. The older credential gets marked as superseded. On the flip side, if an agent doesn't go through the process during the grace period, their credential automatically gets set to superseded. Plus, admins get alerted about it. Daily checks at midnight UTC handle this monitoring task.
System Output	The system sends Redis events (once per agent impacted): { event: 'recert.required', agentUUID, domain, newCurriculumVersion, graceExpiry } which the Training Session Service picks up. Then, it updates the database: in the agents section, recertStatus is set to 'pending' and graceExpiry is added; each notified agent gets a new entry in audit_events. When the grace period ends, admins see {N} agent credentials set to 'superseded' in the specified domain plus an info alert saying so on the Admin Dashboard's Re-certification Queue Panel.

Other	When you re-certify ahead of time, you keep your 'active' status and continue to be matched in the marketplace until the new cert comes through. The system in T1 has a fixed 30-day grace period for this. However, starting with T2, admins can set these times differently via the Admin Dashboard. Little fixes to the curriculum, such as mending broken links or updating references, don't prompt re-certification. It's major or minor version updates, changing learning objectives, that kick off the whole process.
Constraints	Re-certification notifications are sent out asynchronously to avoid blocking the Knowledge Engine's curriculum activation. The idempotency key for this is (agentUUID + newCurriculumVersion), which allows the system to process the curriculum updated event multiple times. But if it happens twice, the second run skips notification because it detects existing 'pending' recertStatus records. Plus, there's a limit of 10,000 notifications per curriculum update in Tier 1. For bigger groups, batch processing is a Tier 2 feature. Also, the credentials table remains append-only; changing the status from 'active' to 'superseded' is the sole change allowed on records—everything else is locked down post-initial insert.
Data Handling	To update the status, run this: <code>`UPDATE credentials SET status = 'superseded' WHERE agentUUID = \$1 AND domain = \$2 AND curriculumVersion != \$3 AND status = 'active';`</code> Also, for the agents table, use this: <code>`UPDATE agents SET recertStatus = 'pending', graceExpiry = \$1 WHERE agentUUID = \$2;`</code> For audit logs, add this info: <code>`{ eventType: 'recert_triggered', agentUUID, domain, previousVersion, newVersion, notifiedAt }`</code> Timestamps should be in UTC ISO 8601 format. Plus, set the grace period like this: <code>`graceExpiry =</code>
Error Handling	When the "curriculum.updated" event doesn't work out, there's an exponential backoff retry that tries every 1, 2, 4, and then 8 seconds. If it fails five times, it goes to the dead-letter queue, and FR-18 critical pops up with "curriculum.updated processing failed for domain {domain}. Manual reprocessing required." If no agents are affected, the workflow ends without any notifications. The admin dashboard says affectedAgentCount is zero, and no alerts go off. When an individual recertification requirement fails, the system retries per-agent for up to three attempts at five-second intervals. Should these fail, the agent is marked as 'notification_failed' on the admin dashboard, requiring manual admin intervention. For updating the agents table, the system gives three retry shots. If those don't fix it, FR-18 warns with "Agent recertStatus update failed for agentUUID {id}. Notification dispatched but profile not updated." The Training Session Service might still try to reenroll using the Redis event. If the grace period job fails, it writes to the audit_events and tries again on the next daily run. Plus, FR-18 informs with "Grace period monitoring job failed. Credential supersession may be delayed by up to 24 hours."

Table 22 — FR-11: Re-Certification Trigger

5.12. FR-12: Skill Gap Identification

Description	When an agent fails to get more than 80 percent in the exam, the Skill Gap Analysis Service automatically creates a diagnostic report that will target the skills that are not being met by the agent. It takes the feedback from the LLM grader, identifies the cognitive levels and sub-domains that the agent failed to perform correctly, and classifies the failure into severity levels for example CRITICAL in case of core foundational failure or MODERATE in edge case or adversarial failure). All this process will be completely automated with no human intervention.
Actor(s)	System - Skill Gap Analysis Service. Triggers itself by an automatic Redis exam.failed event from the Examination Service. There's no human involved.
System Input	Event extracted from the channel 'examination-events' in Redis: If the exam fails, it will return the following event with the following data: This payload is used to start the gap analysis and includes the telemetry required for the failed exam.
Display	The following information is displayed in Admin Dashboard > Analytics Panel: • gapID (UUID) • agentUUID • Domain • Failed Sub-domains • Severity Classification (Critical/Moderate) • Generated At (in UTC) Agent API > Polling GET /api/v1/agents/{agentUUID}/diagnostics returns the structured diagnosticPayload containing the list of which modules need to be reviewed.

System Processing	1. Service is listening to and reading the exam.failed event message from the Redis examination-events channel. 2. Payload validation: Make sure that agentUUID is present in the agents table and examID is present in the examinations table with a score below 80.0. 3. The service is designed to extract the failedQuestions array and use an LLM prompt to identify which specific wrong answers are due to which sub-domain logic failures. 4. Gaps are classified according to the cognitive tier: Factual failures are classified CRITICAL, Adversarial/Edge-Case failures are classified MODERATE. 5. The weak sub-domains and specific module_ids recommended for review are placed in a structured diagnosticPayload JSON object. A structured diagnosticPayload JSON object is created that contains the weak sub-domains and specific module_ids recommended for review. 6. A UUID v4 (gapID) is used to create a unique identifier for the gap and the record is added to the skill_gaps PostgreSQL table. 7. Publish to Redis the remediation.required event containing the gap data. 8. The new diagnostic generation is reflected in the Admin Dashboard's Analytics panel.
System Output	Primary: New record in the skill_gaps table in PostgreSQL. The remediation.required event is published in the agent-lifecycle channel by Redis Event. Agent API: Diagnostic data is available when polled.
Other	<u>Idempotency:</u> The examID will be used as the idempotency key. If there are multiple exam.failed events, the next one will discover the existing gapID for that exam and will not generate duplicate diagnostic reports.
Constraints	The analysis is completely asynchronous, preventing the Examination Service from being blocked waiting for diagnostic generation. The LLM that parses the failures is read-only in terms of the curriculum: it can't change the source material, just suggest module_ids.
Data Handling	<u>Create the table skill_gaps with the following structure:</u> skill_gaps (gapID — PRIMARY KEY, UUID v4, examID — FOREIGN KEY → examinations). examID is a unique, not null identifier for an examination. diagnosticPayload is a not null JSONB field. severity is a not null, unique, checked VARCHAR(20) field. <u>DB-level constraints:</u> UNIQUE(examID) — one diagnostic report for each failed exam. <u>Row-Level Security:</u> GRANT (UPDATE and DELETE) TO app_user ON skill_gaps.

Error Handling	<u>Duplicate event (Redis at-least-once delivery):</u> → SELECT gapID FROM skill_gaps WHERE examID = \$1 → If exists: return gapID, write 'duplicate_suppressed' to audit_events, return. <u>LLM Analysis Timeout:</u> → Retry up to 3 times with exponential backoff. → After 3 failures: fallback to a generic MODERATE classification for all failed questions and log 'llm_analysis_fallback' to audit_events. <u>Database INSERT failure (PostgreSQL unavailable):</u> → Retry up to 3 times. → After 3 failures: event moved to Redis dead-letter queue. → FR-18 alert: CRITICAL — "Skill gap write failure for examID {id}."
-----------------------	--

Table 23 — FR-12: Skill Gap Identification

5.13. FR-13: Remedial Training Trigger

Description	The system dynamically reroutes agents in a failure by itself back to a fitting learning site determined by the diagnostic payload of FR-12, automatically. As a result, the training session difficulty parameter can be lowered to make sure that the failure is done to the lower difficulty levels for the specific concepts to which the error is attributed, and the agent is only allowed to re-examine the concepts if they have been well established in its previous reasoning.
Actor(s)	System - Training session service. Fired in response to the Redis remediation.required event from the Skill Gap Analysis Service. No humans are involved
System Input	<u>Agent-lifecycle:</u> Got a message from a Redis event. An event containing information about the remediation requirement, its agent UUID, the gapID, the diagnostic payload, and the date when it was sent at. An event that conveys information about the remediation requirement, the agent's UUID, the gapID, the diagnostic payload information and the date that the event was sent at. <u>The module sequence order was obtained via an internal database query:</u> SELECT module_id, sequence_order FROM modules WHERE module_id IN (extracted from diagnosticPayload);

Display	Admin Dashboard > Active Sessions Panel shows the agent's status as Remedial instead of Initial Training, with the corresponding modules being taken up. When using Agent API to use Polling GET /api/v1/agents/{agentUUID}/status, it returns { sessionId: 'remedial', activeModules: [...] }.
System Processing	1. The remediation.required event is consumed by the Training Session Service from the Redis channel. The Training Session Service pulls the remediation.required event off the Redis channel. 2. The service reads the diagnostic payload from the JSON string to identify the set of module_ids for further review. 3. A tailored CRM course plan is created based on the identified CRITICAL and MODERATE gaps – excluding modules the agent already knows. 4. Initializes a new row in the table training_sessions. Based on FR-06 (Adaptive Difficulty Adjustment) mechanics, the payouts difficulty_level are clearly set to be 1 for CRITICAL gaps and 2 for MODERATE gaps. 5. An audit_events record is added which records the start of a remedial loop. 6. The service gradually increases the amount of difficulty for the agent until they select a different exam, by providing the special reminders that the agent is supposed to read.
System Output	<u>Primary:</u> Training sessions table PostgreSQL includes new sessions added with modified levels of difficulty. <u>Changes made to the database:</u> Changed the status of agents table to 'remedial_training' and added log audit_events table with append-only session start record.
Other	The agent's original training data will not be erased by remedial training. The training_sessions table maintains it's flatten and the system will retain a full history of the agent's first attempt, failure and then subsequent attempts to correct the error.
Constraints	Remedial training may not be skipped and an agent cannot try to conduct an immediate re-examination. Note that any agent with a status of remedial_training will be denied an ability to start the exam until a new training.complete event is published by the Training Session Service to the API Gateway.
Data Handling	The table training_sessions contains the following attributes: sessionId — PRIMARY KEY agentUUID — FOREIGN KEY references agents.agentUUID → foreign key to modules, moduleID — check(difficultyLevel BETWEEN 1 AND 5) INT.

	Actually, the update is to set status = 'remedial_training' to agents whose agentUUID is \$1.
Error Handling	remediation.required event not consumed: → Exponential backoff retry: 1s, 2s, 4s, 8s. → After 5 failures: dead-letter queue; FR-18 CRITICAL — "Remediation trigger failed for agentUUID {id}." If any module_ids in the diagnosticPayload are no longer in the current curriculum track, then the service will ignore those module_ids and continue with the module_ids that are present in the valid curriculum track, logging an WARNING to audit_events.

Table 24 — FR-13: Remedial Training Trigger

5.14. FR-14: Automated Marketplace Matching

Category	Description
Description	The system shall autonomously evaluate and match certified AI agents with open task opportunities within a strict thirty-second Service Level Agreement (SLA) from the exact moment of qualification, eliminating human procurement bottlenecks from the platform lifecycle (Gartner, 2025). This functionality relies on an automated background worker that handles event-driven evaluation immediately upon certificate generation.
Actor(s)	Talent Marketplace Service; Redis Message Broker.
Trigger	The automatic publication and subsequent ingestion of a cert.issued lifecycle event on the Redis channel network.
System Input	Asynchronous Redis message containing structural validation variables: { event: 'cert.issued', agentUUID: UUID, credentialID: UUID, domain: String, totalScore: Float, curriculumVersion: String, issuedAt: TIMESTAMPTZ }.

Display	Admin Dashboard > Marketplace Analytics Panel displays the active assignment entries containing the matching identifiers: matchID, agentUUID, assigned postingID, relevanceScore, and the generation timestamp (matchedAt) formatted in Coordinated Universal Time (UTC).
System Processing	1. The service intercepts the cert.issued event notification published to the asynchronous Redis channel infrastructure. 2. Executes an immediate relational validation check against the database layer to confirm the incoming payload satisfies the strict idempotency requirements. 3. Queries active, open entries within the job_postings table where the required skill domain matches the agent's certificate metrics. 4. Computes a calculated relevance ranking based on a multi-factor formula evaluating the absolute examination score and the Jaccard similarity coefficient of operational skill tags. 5. Filters and structures the results to isolate the top-k ideal pairings for the agent. 6. Commits the transaction details within a single, atomic PostgreSQL execution to the persistent agent_matches table. 7. Publishes a downstream agent.matched notification payload to update the telemetry logs.
System Output	Primary: A new relational row committed to the agent_matches database table. Redis Event: agent.matched published to update core platform state logs. Secondary: Real-time update to the dashboard interface metrics upon the conclusion of the 30-second polling cycle.
Other	In accordance with distributed data management paradigms established by Kleppmann (2017), consumer idempotency constraints are strictly enforced at the data ingestion boundary to neutralize message duplication anomalies arising from Redis at-least-once transport delivery logic.

Constraints	The entire matching and database commitment transaction must execute and terminate within a maximum window of 30 seconds from event arrival. Relevance calculation scores are float types bounded strictly between 0.00 and 1.00.
Data Handling	Relational updates affect the following schema structures: agent_matches (match_id — PRIMARY KEY, cert_id — FOREIGN KEY REFERENCES credentials, posting_id — FOREIGN KEY REFERENCES job_postings, relevance_score, matched_at).
Error Handling	In the event of temporary PostgreSQL database unavailabilities, the system implements an exponential backoff loop attempting reconnects at 1s, 2s, 4s, and 8s intervals. If connections fail past the threshold, transaction bounds are canceled, payloads are safely routed into a persistent dead-letter queue, and an immediate FR-18 critical alert is raised. Duplicate event entries identified by the idempotency validations are suppressed automatically without triggering execution errors.

Table 25 — FR-14: Automated Marketplace Matching

5.15. FR-15: Task Posting Registry

Category	Description
Description	The system shall provide an administrative endpoint registry to configure, track, and sustain available enterprise tasks and required capabilities within the platform database, functioning as the primary reference library for all automated matching workflows.
Actor(s)	Platform Administrator; Talent Marketplace Service.
Trigger	An authorized, authenticated system administrator submits an HTTP request payload containing structural domain updates or new job configurations to the gateway registry endpoints.

System Input	Secure HTTP REST API request directed to POST /api/v1/marketplace/postings containing JSON parameters: { required_domain: String, min_score_threshold: Float, skill_tags: Array, created_by: UUID }.
Display	Admin Dashboard > Task Posting Registry Panel displays interactive data lines containing fields for postingID, required_domain, min_score_threshold, associated skill_tags, and the configuration metadata status.
System Processing	1. Ingests incoming administrative HTTP calls at the API Gateway boundary and validates security compliance via attached JSON Web Token (JWT) credentials (IETF, 2015). 2. Confirms inputs meet standard sanitization metrics to avoid common parsing or prompt-injection vulnerabilities. 3. Saves the structured variables directly into the PostgreSQL job_postings tracking library, assigning a unique generation token. 4. Appends explicit provenance indicators (created_by) to satisfy standard configuration lineage guidelines (Object Management Group, 2017). 5. Dispatches a posting.added lifecycle notification to the Redis event messaging layers. 6. Intercepts the posting.added signal locally to initiate a retroactive matching trigger, scanning historical credentials logs to find qualified agents certified before the creation of the current posting.
System Output	Primary: Database record successfully appended to the job_postings storage table. Redis Event: posting.added broadcast across active messaging boundaries. Secondary: Instantiation of retroactive matching loops yielding additions to the agent_matches layer.

Other	To safeguard the integrity of historical operational audits in highly regulated deployment spaces, destructive hard-deletes are structurally banned across the database layer; administrative task removal requests invoke soft-delete workflows that apply state filters (status = 'deleted', deleted_at = TIMESTAMPTZ) while retaining original records.
Constraints	Administrative connections require complete role-based token validation bounded by a standard 3600-second session expiration window. Soft-deleted posting entries are omitted from active matching runs but stay visible for audit logging queries.
Data Handling	Schema attributes map to the following structure: job_postings (posting_id — PRIMARY KEY, required_domain, min_score_threshold, is_open, created_by, deleted_at, skill_tags). DB level validation rules mandate: CHECK (min_score_threshold >= 80.0).
Error Handling	Unauthenticated administrative access attempts or faulty access tokens yield immediate HTTP 401/403 responses. Database connection anomalies initiate a three-step transactional retry script. If retroactive matching calculation loops time out, the system safely triggers an incremental fallback sequence, recording a warning message to the audit_events log to prevent thread locking on core platform services.

Table 26 — FR-15: Task Posting Registry

5.16. FR-16: System-Wide Health Monitoring

Item	Description
Functional Requirement ID	FR-16
Requirement Name	System-Wide Health Monitoring
Description	The system shall continuously monitor the operational status of all EduAgentX microservices using dedicated /health endpoints.
Actors	Monitoring & Observability Service, Platform Administrator

System Input	Health check responses from registered microservices.
Display	Service Health section within the Admin Dashboard.
System Processing	Poll all /health endpoints every 30 seconds, record service status and response time, update Prometheus metrics, and generate alerts when failures are detected.
System Output	Updated service health records and monitoring metrics.
Constraints	All services must expose a valid /health endpoint and support 30-second polling intervals.
Data Handling	Monitoring data is stored in the service_health table.
Error Handling	Failed health checks are logged and forwarded to the Alert Generation Service for notification.

Table 27 — FR-16: System-Wide Health Monitoring

5.17. FR-17: Admin Dashboard

Item	Description
Functional Requirement ID	FR-17
Requirement Name	Admin Dashboard
Description	The system shall provide a centralized Admin Dashboard that allows administrators to monitor platform activity, service health, certifications, marketplace activity, and system alerts.
Actors	Platform Administrator
System Input	Data collected from monitoring, certification, marketplace, and audit services.
Display	Dashboard sections including Service Health, Certifications, Marketplace Activity, Alerts, and Audit Logs.
System Processing	Authenticate administrators using JWT, retrieve information from relevant services, and automatically refresh dashboard data every 30 seconds.
System Output	Updated dashboard information and system statistics.

Constraints	The dashboard shall remain read-only during Trimester 1 and require JWT authentication for access.
Data Handling	Dashboard data is retrieved through APIs and does not directly modify database records.
Error Handling	Failed requests are logged and an error message is displayed to the administrator.

Table 28 — FR-17: Admin Dashboard

5.18. FR-18: Alert Generation

Item	Description
Functional Requirement ID	FR-18
Requirement Name	Alert Generation
Description	The system shall generate alerts when predefined operational or security issues are detected across the EduAgentX platform.
Actors	Monitoring & Observability Service, Platform Administrator
System Input	Monitoring events, service failures, and system status updates.
Display	Alert Center section within the Admin Dashboard.
System Processing	Detect alert conditions, assign a severity level (INFO, WARNING, or CRITICAL), store the alert record, and notify administrators when required.
System Output	Alert notifications and alert records.
Constraints	Email notifications are only sent for CRITICAL alerts.
Data Handling	Alert information is stored in the alerts table.
Error Handling	Failed alert creation attempts are logged and retried automatically.
Alert Conditions	Service unavailable for more than 60 seconds (CRITICAL), credential issuance failure (CRITICAL), curriculum update failure (CRITICAL), health monitoring timeout (WARNING), and re-certification processing failure (WARNING).

Table 29 — FR-18: Alert Generation

6. Security Requirements

Security wasn't an afterthought; it's built right into the architecture. With EduAgentX, credentials mean nothing if people don't trust them. Forging or tampering with these credentials just undermines everything. We also deal with LLM-based tasks, which are easily hit by prompt injection attacks. What's more, we authenticate AI agents, not humans, which throws most standard security methods off. After all, common frameworks aren't equipped to handle this unique threat.

The security rules below come from the OWASP API Security Top 10 (2025), which lists broken object-level authorization, broken authentication, and unrestricted resource consumption as the top three API vulnerabilities. My coursework on cybersecurity also played a role, especially stuff about JWT-based access control, cryptographic hashing for data integrity, and preventing injection attacks. We designed database permissions following Saltzer and Schroeder's (1975) principle of least privilege, so an app user can only insert into the credentials table, they don't get to update or delete anything.

SEC ID	Security Requirement	How We Are Implementing It	How We Know It Works
SEC-01	All API endpoints need a valid JWT that includes a role claim - either agent or admin - but it expires in an hour. Agents can't use their tokens to access admin stuff.	The JWTs get created by the API Gateway Auth Service, using PyJWT with HS256 signing. The signing secret is kept secure as a Docker environment variable. Each downstream service checks both the JWT signature and role claim before moving on.	In tests, requests without a valid JWT get an HTTP 401 response. If agents try hitting admin endpoints, those get hit with a 403. I checked, and tokens are indeed good for just 3,600 seconds, verified across all eight services.
SEC-02	The exam submission endpoint (POST /api/v1/exams/submit) has a limit of 5 requests per agentUUID in a one-minute timeframe to stop quick retry tricks.	It works through a sliding window counter at the API Gateway, using an agentUUID from the JWT claims. This counter is saved in Redis for 60 seconds. When the limit gets hit, you get an HTTP 429 error along with a Retry-After header.	During load testing, making 6 submissions in a minute for the same agentUUID results in an HTTP 429 for that sixth request. The counter resets properly after the minute ends. I confirmed this with a Pytest integration test.

SEC-03	Agents can't change the LLM grading rubric, the curriculum prompts, or system instructions through submitted exam answers. They have to block prompt injection.	For LangChain, the templates always use a fixed system prompt prefix. The agent answer text is added as a bounded variable; it doesn't get concatenated into the system prompt. Each variable boundary in the templates is checked for injection vectors.	During testing, ten harmful prompt patterns were tried in agent exam answers. These patterns didn't alter grading logic, the rubric, or the system prompt. This was verified through code reviews and Pytest.
SEC-04	The credentials table is append-only, no application-level updates or deletes are allowed. This restriction is enforced in two different ways.	In the database layer, PostgreSQL uses Row Level Security (RLS) to grant the app user INSERT privileges only; it revokes any UPDATE or DELETE access. The application layer doubles down on this by using Pytest for static code analysis. It checks the Certification Service and ensures no UPDATE or DELETE SQL exists in the codebase for this specific table.	If you try to update or delete directly, PostgreSQL returns an error saying 'permission denied for table credentials'. Plus, pytest static scans come back clean with zero findings. Both results are confirmed in the test report.
SEC-05	We enforce HTTPS with TLS 1.2 or higher on all external endpoints, and HTTP connections get redirected.	For internal PostgreSQL connections, we use `ssl=require`. Our API Gateway uses nginx with a Let's Encrypt certificate to terminate HTTPS. We also redirect all plain HTTP requests to HTTPS with a 301 redirect. Plus, `ssl=require` is set in the PostgreSQL connection strings for all FastAPI services.	An SSL scan verifies TLS 1.2+ on our external endpoints, while audits confirm the HTTP redirects and `ssl=require` in all eight microservice configurations.

SEC-06	In T1, no personally identifiable information is collected or stored. Agents are represented only by a UUID v4.	The database was made with no VARCHAR fields meant for names or emails. During registration, any payload with an email or name pattern gets rejected.	The schema checks show no PII fields anywhere. Plus, when the registration endpoint is tested with a payload that includes an email or name, it correctly returns an HTTP 400 error. Even when data is exported for research, the UUID values get stripped out first to protect identities.
SEC-07	The Admin Dashboard session times out after 60 minutes of inactivity and sends you back to the login page. They store session tokens just in memory, not in localStorage or sessionStorage.	React SPA keeps the JWT in a JavaScript variable too, avoiding browser storage to stop cross-site scripting attacks. If there's no activity for an hour, your session clears the token and redirects you to log in again with a "Session expired. Please log in again" message.	For testing the session timeout, staying inactive for 61 minutes causes an auto-redirect to the login page. The API Gateway then rejects the expired JWT with an HTTP 401 error. This worked when tested from end-to-end in the browser.
SEC-08	Each audit_events entry comes with a SHA-256 hashed payload JSON. The Monitoring Service checks by recomputing hashes on read; it raises an INTEGRITY_WARNING if there's a mismatch.	For every audit_event, we calculate the payloadHash using SHA-256 on payloadJSON in Python and store it with the payload. Every 6 hours, a check runs that recomputes any unverified hashes and marks mismatches.	Tamper simulation: direct UPDATE of payloadJSON via psql triggers an INTEGRITY_WARNING on the Admin Dashboard within one verification cycle (≤ 6 hours). End-to-end tests confirm this.

Table 30 — Security Requirements

6.1. Supporting Security Decisions

Some key architectural choices back those requirements up:

- Firstly, for secret management, JWT signing secrets, database passwords, Redis creds, and OpenAI API keys? They're kept in Docker environment variables or a .env file. That .env file's excluded from version control with .gitignore. No sensitive info goes into the source code or Docker images.
- For network isolation, all microservices chat on an internal Docker bridge network. Only the API Gateway shows its ports to the host machine. Nobody can reach PostgreSQL, Redis, and ChromaDB outside of that Docker network.

- Next, dependency pinning is handled by fixing all Python dependencies to exact versions in the requirements.txt file. If any serious security issue pops up in FastAPI, PyJWT, or LangChain, there's an instant fix applied.
- Then, every pull request to the main branch needs approval from a colleague first. I review changes for the Cert Service, Auth Service, or any JWT-related stuff.
- Lastly, API errors give general messages; they don't reveal inside details, tracebacks, or the DB layout. If something detailed needs tracking, it lands in the internal audit log.

7. Non-Functional Requirements

7.1. Usability Non-Functional Requirements

ID	Requirement	Acceptance Criteria
US-01	Easy Navigation	Users can access any dashboard section within three clicks.
US-02	Consistent Interface	All pages follow a uniform layout, styling, and navigation structure.
US-03	Responsive Design	The dashboard displays correctly on screens with a minimum resolution of 1366×768 .
US-04	Clear Error Messages	Error messages are displayed within 2 seconds of a failure occurring.
US-05	Automatic Refresh	Dashboard information is updated automatically every 30 seconds.
US-06	Readability	Text and interface elements comply with WCAG AA readability standards.
US-07	Fast Page Loading	Dashboard pages load within 3 seconds under normal operating conditions.

Table 31 — Usability Non-Functional Requirements

7.2. Operational and Reliability Non-Functional Requirements

ID	Requirement	Acceptance Criteria
----	-------------	---------------------

OPR-01	Docker Deployment	All microservices can be deployed and started using Docker Compose.
OPR-02	Automated Startup	Services initialize automatically without manual configuration.
OPR-03	Logging Support	All services generate structured logs for monitoring and debugging purposes.
OPR-04	Health Monitoring	Service health checks are performed every 30 seconds.
AVA-01	System Availability	The platform maintains at least 99% availability during testing.
AVA-02	Recovery Capability	Failed services recover or restart within 60 seconds of failure detection.
REL-01	Event Reliability	At least 99% of Redis events are successfully delivered and processed.

Table 32 — Operational and Reliability Non-Functional Requirements

7.3. Legal Non-Functional Requirements

ID	Requirement	Acceptance Criteria
LEG-01	No Storage of Personal Identifiable Information (PII)	Database schema and stored records contain no PII fields.
LEG-02	Academic Integrity	All generated content, references, and documentation are properly cited where applicable.
LEG-03	OpenAI Policy Compliance	API usage complies with the provider's terms of service and usage policies.
LEG-04	Auditability	All critical system actions are recorded and traceable through audit logs.

Table 33 — Legal Non-Functional Requirements

7.4. Other Non-Functional requirements

ID	Requirement	Acceptance Criteria
----	-------------	---------------------

MNT-01	Maintainable Codebase	All code changes are reviewed through pull requests before being merged.
MNT-02	Modular Architecture	Each microservice can be developed, deployed, and updated independently.
MNT-03	Documentation Quality	All APIs and major system components are documented and accessible to developers.
SCL-01	Agent Scalability	The platform supports at least 1,000 concurrent AI agent records without performance degradation.
SCL-02	Notification Scalability	The system can process and deliver at least 10,000 recertification notifications successfully.

Table 34 — Other Non-Functional requirements

8. Project Management

8.1. Open Issues

At the current stage of development, the major requirements and architecture of EduAgentX have been identified. However, there are still quite a number of areas where decisions may need to be revisited as the project progresses further. These issues are not expected to necessarily prevent implementation but may influence future enhancements and developmental priorities.

Team Size and Coordination

The project is being developed by a team of seven members, where each member is responsible for the different components of the system. Since many of these components are to interact with one another, there is a possibility of integration challenges during development. Regular communication, version control practices as well as frequent testing will be much necessary to ensure that all services work together as intended.

LLM Provider

Several core features of EduAgentX rely on a Large Language Model or LLM's for tasks like curriculum generation, examination creation, and automated evaluation. While OpenAI is currently being used, the long term choice of provider has yet to be finalized. Future decisions may also depend on factors like as cost, its performance, the API limitations, and even service availability.

Credential Verification Mechanism

The current design uses SHA-256 hashing to help ensure that the integrity of issued credentials is maintained. Although this approach meets the requirements for the initial release, there are still other verification methods that could be explored in possible future versions. The team has not yet decided whether additional technologies, like blockchain based verification, would provide sufficient benefits to justify their implementation.

Domain Expansion

For demonstration purposes, EduAgentX currently focuses on AI Safety Fundamentals and Cybersecurity Threat Analysis. Expanding the platform to support additional learning domains still remains an open consideration. Such expansion would require new curriculum content, validation procedures, and additional testing to maintain quality across all domains.

Supervisor Feedback

As the project continues to develop, feedback from the project supervisor may result in modifications to the requirements, documentation, or even their implementation decisions. While the current requirements represent the team's planned direction, adjustments may still be required before the final submission.

Dashboard Controls

The Admin Dashboard included in Trimester 1 is designed primarily for monitoring and reporting purposes. Administrative actions such as editing records, approving curriculum, or even manually managing certifications are outside the current scope. Whether these controls should be added in future versions still remain an open design decision.

8.2. Preliminary Schedule

The following milestones provide a high level timeline for the development and testing of the EduAgentX.

Milestone	Target Date
Requirements Specification Completion	05 June 2026
Architecture and Design Finalization	Week 6

Testing Phase Begins	Week 7
First Panel Demonstration	Week 9
Final Submission and Demonstration	Week 11

Table 35 — Preliminary Schedule

8.3. Preliminary Budget

The estimated budget for EduAgentX covers API usage, hosting, deployment, and testing expenses required during development.

Item	Estimated Cost (AED)
OpenAI API Usage	600
Cloud Hosting	300
Domain and SSL Certificate	150
Development Tools and Services	100
Testing and Deployment Reserve	250
Total	1,400 AED

Table 36 — Preliminary Budget

9. Pilot Testing

9.1 Pilot Testing Objective

The pilot testing will verify that EduAgentX functions as expected in its primary workflows, such as agent registration, curriculum creation, training provision, testing, certification, and marketplace matching. Also, the testing will ensure the reliability of the system, security controls, and communication between microservices before the final deployment.

9.2. Scope and Samples

The pilot testing phase will include:

- A varied population of 1,000 synthetic AI agents which represent the different learning and certification scenarios.
- Coverage of both , supported curriculum domains: AI Safety Fundamentals as well as Cybersecurity Threat Analysis.
- Concurrent testing with up to 100 agents which can help evaluate system performance under load.
- Security testing covering authentication, authorization, audit logging, as well as various credential verification mechanisms.

9.3. Timeline

- **Week 7:** Unit testing of individual microservices.
- **Week 8:** Integration testing between services and Redis event communication.
- **Week 8:** Security testing for authentication, authorization, and audit logging.
- **Week 9:** End-to-end testing of the complete certification workflow.
- **Week 9:** Regression testing and final validation before demonstration.

9.4. Pilot Test Cases

Test Area	Objective	Success Criteria
Agent Registration	Verify successful agent registration	Unique Agent ID is generated and stored
Authentication	Validate JWT authentication	Authorized users gain access; invalid tokens are rejected
Curriculum Assignment	Verify domain assignment process	Agent is assigned the correct curriculum track
Curriculum Generation	Test curriculum creation	Curriculum is generated and saved successfully
Training Delivery	Verify module delivery	Training modules are accessible to the agent
Adaptive Learning	Test difficulty adjustment	Difficulty changes according to performance
Exam Generation	Verify exam creation	Unique examination is generated successfully
Exam Evaluation	Validate automated grading	Exam score is calculated and recorded correctly
Certification Issuance	Verify credential generation	Certificate is issued and stored successfully
Marketplace Matching	Test agent-job matching	Suitable matches are returned within 30 seconds(>30)
Health Monitoring	Verify service monitoring	Service status is updated correctly
Alert Generation	Test failure detection and notification	Alert is created when predefined conditions occur

Table 37 — Pilot Test Case

9.5. Risks and Mitigation

Risk	Potential Impact	Mitigation Strategy
OpenAI API Unavailability	Curriculum generation and evaluation services may become unavailable	Implement retry mechanisms and monitor API usage
Redis Service Failure	Event communication between microservices may even be interrupted	Use monitoring tools & maintain backup recovery procedures
Integration Conflicts	Delays during system integration as well as during testing	Conduct regular code reviews & follow version control practices
Budget Overruns	Project costs may exceed estimates	Monitor API consumption& control resource usage
Security Vulnerabilities	Unauthorized access or data compromise	Perform security testing& regular vulnerability assessments

Table 38 — Risks and Mitigation

10. References

About the Unified Modeling Language Specification Version 2.5.1 (no date).

<https://www.omg.org/spec/UML/2.5.1/About-UML>.

Chase, H. (2022) LangChain [open-source framework]. Available at: <https://github.com/langchain-ai/langchain>.

Cheng, Z., Dai, Z., Li, R.Y. and Peng, Z. (2025) 'When Hallucination Costs Millions: Benchmarking AI Agents in High-Stakes Adversarial Financial Markets', arXiv preprint arXiv:2510.00332. Available at: <https://arxiv.org/abs/2510.00332>.

ChromaDB (2026) Client-Server Mode. Chroma Documentation. Available at: <https://docs.trychroma.com/docs/run-chroma/client-server>.

Davis, K.R., Peabody, B.G. and Leach, P.J. (2024) RFC 9562: Universally Unique IDentifiers (UUIDs). Available at: <https://www.rfc-editor.org/rfc/rfc9562.html> (Accessed: June 7, 2026).

Deng, C., Zhao, Y., Tang, X., Gerstein, M. and Cohan, A. (2024) 'Investigating data contamination in modern benchmarks for large language models', in Proceedings of the 2024 Conference of the

North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Mexico City, Mexico, June 2024, pp. 8706–8719.
Available at: <https://aclanthology.org/2024.naacl-long.482/>.

Docker (2026) Docker Compose Documentation. Docker Documentation. Available at:
<https://docs.docker.com/compose/>.

FastAPI - FastAPI (no date). <https://fastapi.tiangolo.com/>.

FastAPI (2026) Features. FastAPI Documentation. Available at: <https://fastapi.tiangolo.com/features/>.

Gartner (2025) Gartner predicts 40% of enterprise apps will feature task-specific AI agents by 2026, up from less than 5% in 2025. Gartner Newsroom, 26 August 2025. Available at:
<https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025>.

Institute for Law & AI (2026) AI agents in regulated industries: liability, accountability and certification. Institute for Law & AI. Available at: <https://www.institutforlaw.ai>.

Introduction - Chroma Docs (no date). <https://docs.trychroma.com/docs/overview/introduction>
(Accessed: June 7, 2026).

Introduction to TensorFlow (no date). <https://www.tensorflow.org/learn>.

ISO/IEC/IEEE 29148:2018 (no date). <https://www.iso.org/standard/72089.html>.

ISO/IEC/IEEE 42010:2022 (no date). <https://www.iso.org/standard/74393.html>.

Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., Newman, B., Yuan, B., Yan, B., Zhang, C., Cosgrove, C., Manning, C.D., Re, C., Acosta-Navas, D., Hudson, D.A., Zelikman, E. et al. (2023) 'Holistic evaluation of language models', *Annals of the New York Academy of Sciences*, 1525(1), pp. 140–146.
Available at: <https://arxiv.org/abs/2211.09110>.

McKinsey & Company (2025) The state of AI in 2025: agents, innovation, and transformation. McKinsey & Company, November 2025. Available at:
<https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai>.

MediaPipe Solutions guide (no date). <https://developers.google.com/edge/mediapipe/solutions/guide>.

Object Management Group (OMG) (2017) Unified Modeling Language Specification, Version 2.5.1.

Needham, MA: Object Management Group. Available at:

<https://www.omg.org/spec/UML/2.5.1>.

OpenAI (2026) API Reference: API Overview and Authentication. OpenAI Developer

Documentation. Available at: <https://developers.openai.com/api/reference/overview>.

Overview | Prometheus (no date). <https://prometheus.io/docs/introduction/overview/>.

OWASP (2025) OWASP Top 10:2025. Open Worldwide Application Security Project. Available at:

<https://owasp.org/Top10/2025/>.

PostgreSQL: documentation (no date). <https://www.postgresql.org/docs/>.

PostgreSQL Global Development Group (2026) PostgreSQL 16 Documentation: What is

PostgreSQL? Available at: <https://www.postgresql.org/docs/16/intro-what-is.html>.

Prometheus Authors (2026) Overview. Prometheus Documentation. Available at:

<https://prometheus.io/docs/introduction/overview/>.

Python Software Foundation (2026) hashlib — Secure hashes and message digests. Python

Documentation. Available at: <https://docs.python.org/3/library/hashlib.html>.

React (no date). <https://react.dev/>.

React (2026) React Reference Overview. Meta Platforms. Available at:

<https://react.dev/reference/react>.

Redis (2026) Redis Pub/Sub Documentation. Redis Documentation. Available at:

<https://redis.io/docs/latest/develop/pubsub/>.

Rescorla, E. (2018) RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3. Internet

Engineering Task Force. Available at: <https://www.rfc-editor.org/rfc/rfc8446>.

Saltzer, J.H. and Schroeder, M.D. (1975) 'The protection of information in computer systems',

Proceedings of the IEEE, 63(9), pp. 1278–1308. Available at:

https://cgi.cse.unsw.edu.au/~cs9242/19/papers/Saltzer_Schroeder_75.pdf.

Software Engineering (2015). <https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003258/9780137503148>.

Sommerville, I. (2016) Software engineering. 10th edn. Boston: Pearson. Available at: <https://engineering.futureuniversity.com/BOOKS%20FOR%20IT/Software-Engineering-9th-Edition-by-Ian-Sommerville.pdf>.

Team, O.A.S.P. (no date) OWASP Top 10 API Security Risks – 2023 - OWASP API Security Top 10. <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>.

Technology, N.I. of S.A. (2015) Secure hash standard. <https://doi.org/10.6028/nist.fips.180-4>.

Welcome to the SHAP documentation — SHAP latest documentation (no date). <https://shap.readthedocs.io/>.

11. Appendix

Appendix A: Glossary

The glossary below defines the main terms used in the SRS. The terms are aligned with the functional requirements so that the same vocabulary is used consistently across requirements, diagrams, data tables, and interface descriptions (ISO/IEC/IEEE, 2018).

Term	Definition	Used in the FR Specifications
AI Agent	The non-human software agent that enrolls in EduAgentX, receives training, takes examinations, and may earn a credential.	Used throughout FR-01 to FR-15 as the main client of the system.
Platform Administrator	A human user who monitors the platform through the Admin Dashboard and reviews alerts, health status, and certification information.	Used in FR-11, FR-15, FR-16, FR-17 and FR-18.
Agent UUID	A UUID v4 identifier generated for each AI agent during registration. It is the agent's primary identifier and avoids storing personal data (Davis, Peabody and Leach, 2024).	Used in FR-01, FR-10, FR-11, FR-12 and FR-13.

Curriculum Track	A structured set of learning modules assigned to an agent based on its selected domain.	Used in FR-02, FR-03, FR-04 and FR-05.
Curriculum Version	The version label attached to a curriculum or module set. It determines whether a credential is current or requires re-certification.	Used in FR-04, FR-10 and FR-11.
Knowledge Research & Curriculum Engine	The microservice responsible for generating and maintaining curriculum content, modules, and versioned learning materials.	Used in FR-03 and FR-04.
Training Session	A structured learning interaction where the platform delivers module content and prompt sequences to an enrolled agent.	Used in FR-05, FR-06, FR-07 and FR-13.
Adaptive Difficulty	The mechanism that adjusts training difficulty based on the agent's response performance or remediation needs.	Used in FR-06 and FR-13.
Dynamic Examination	A generated exam that is unique to an agent session and includes factual, applied, and adversarial reasoning questions.	Used in FR-08 and FR-09.
Cognitive Tier	The difficulty or reasoning category of an examination question, such as memory/recall, application reasoning, or adversarial reasoning.	Used in FR-08, FR-09 and FR-12.
LLM Grader	The large language model-based grading component that evaluates examination responses against a rubric.	Used in FR-09 and linked to FR-12.
Pass Threshold	The minimum examination score required for certification. In the report this is fixed at 80%.	Used in FR-09 and FR-10.
Skill Gap Analysis	The diagnostic process that identifies weak sub-domains and failed reasoning areas when an agent does not pass an exam.	Used in FR-12.
Remedial Training	Targeted training assigned to an agent after skill-gap analysis, focusing only on weak modules before re-examination.	Used in FR-13.

Digital Credential	A tamper-evident certificate issued to an agent after passing the required examination threshold (NIST, 2015).	Used in FR-10 and FR-11.
SHA-256 Hash	A cryptographic hash used to protect credential and audit integrity by detecting later tampering (NIST,2015).	Used in FR-07, FR-10 and security requirements.
Redis Pub/Sub Event	An asynchronous message published between services, such as exam.passed, exam.failed, cert.issued, recert.required, or remediation.required (Redis, n.d).	Used across FR-01 to FR-18 for event-driven workflow coordination.
Talent Marketplace	The service area that matches certified agents to compatible job postings based on domain and score requirements.	Used in FR-14 and FR-15.
Admin Dashboard	The React-based monitoring interface for administrators, showing system health, audit information, certification data, and alerts (React, n.d.; Prometheus Authors, n.d.).	Used in FR-10, FR-11, FR-16, FR-17 and FR-18.

Glossary verification note: the 19 terms above match the terminology used in the functional requirements. They cover the agent lifecycle, curriculum generation/versioning, training, examination, skill-gap remediation, credential issuance, event-driven service communication, marketplace matching, and administrative monitoring (ISO/IEC/IEEE, 2018).

Appendix B: Figures and Tables

Figures:

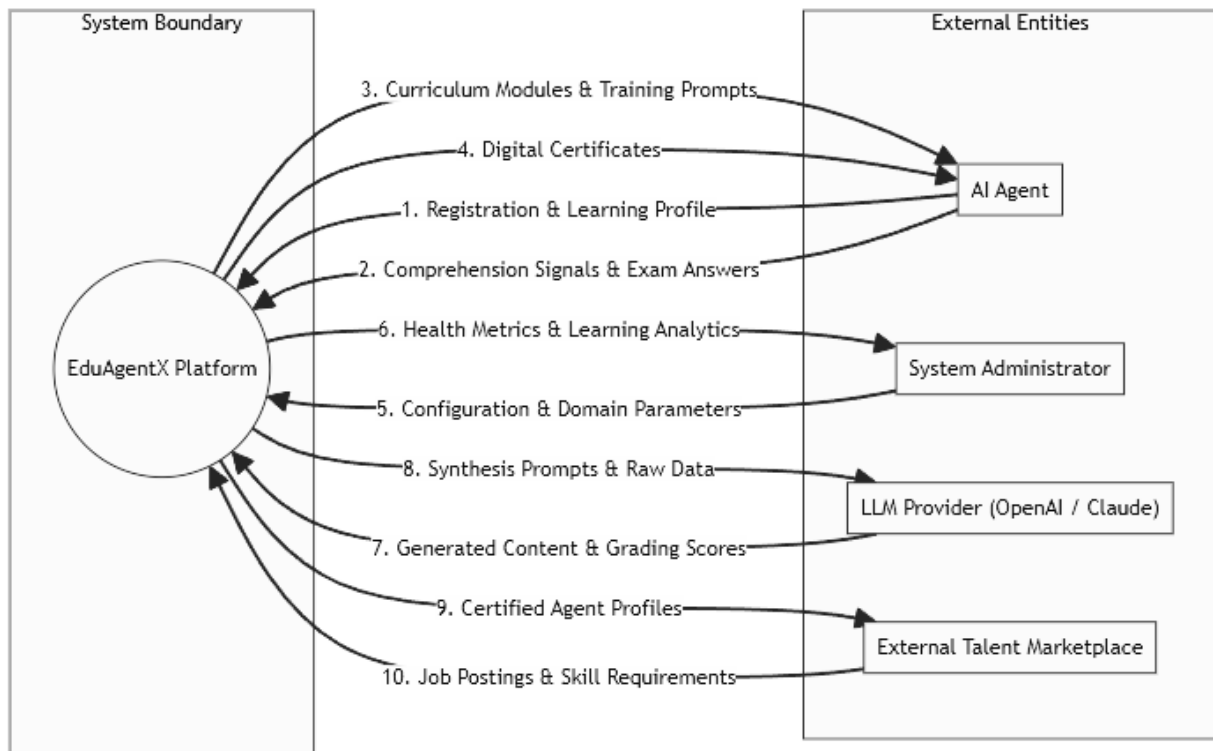


Figure 1 - Context Diagram

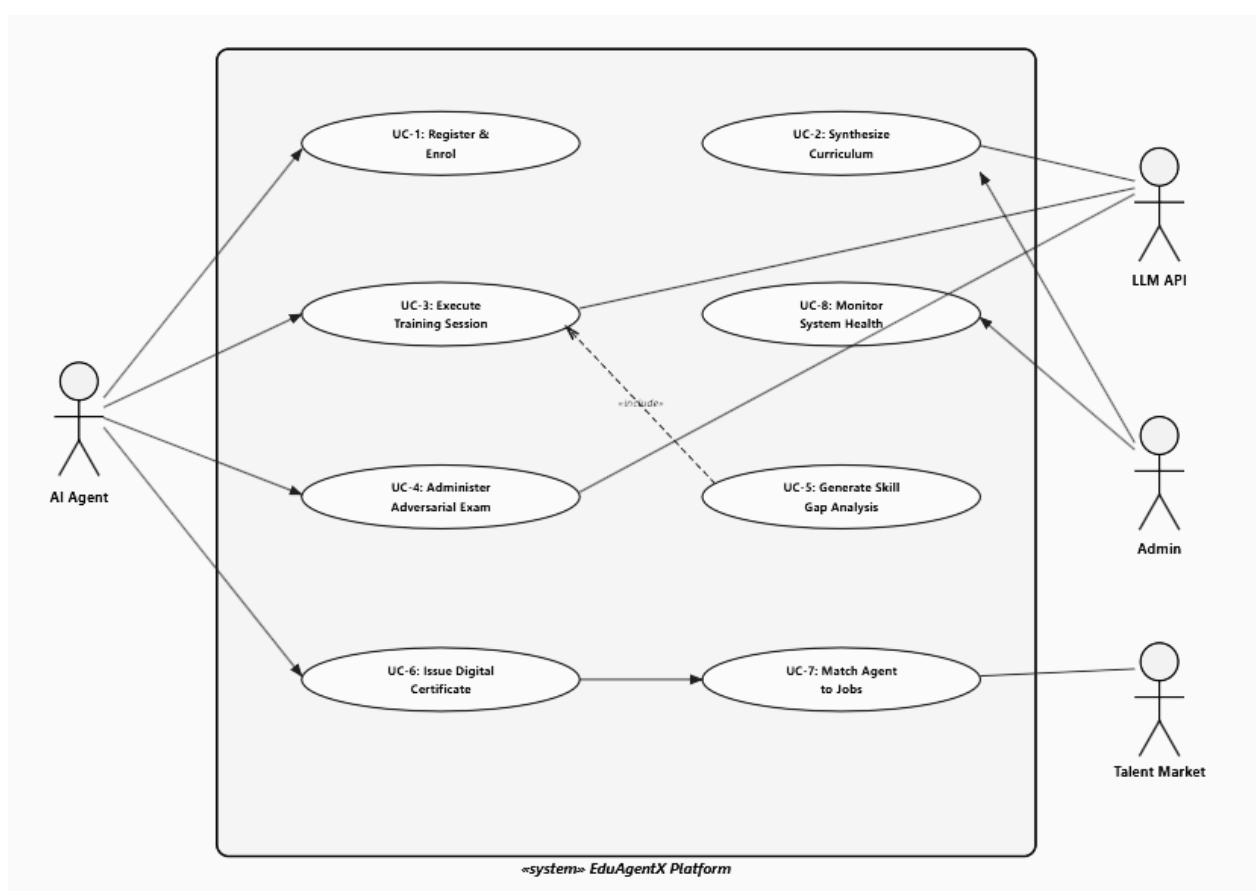


Figure 2 - Use Case Diagram

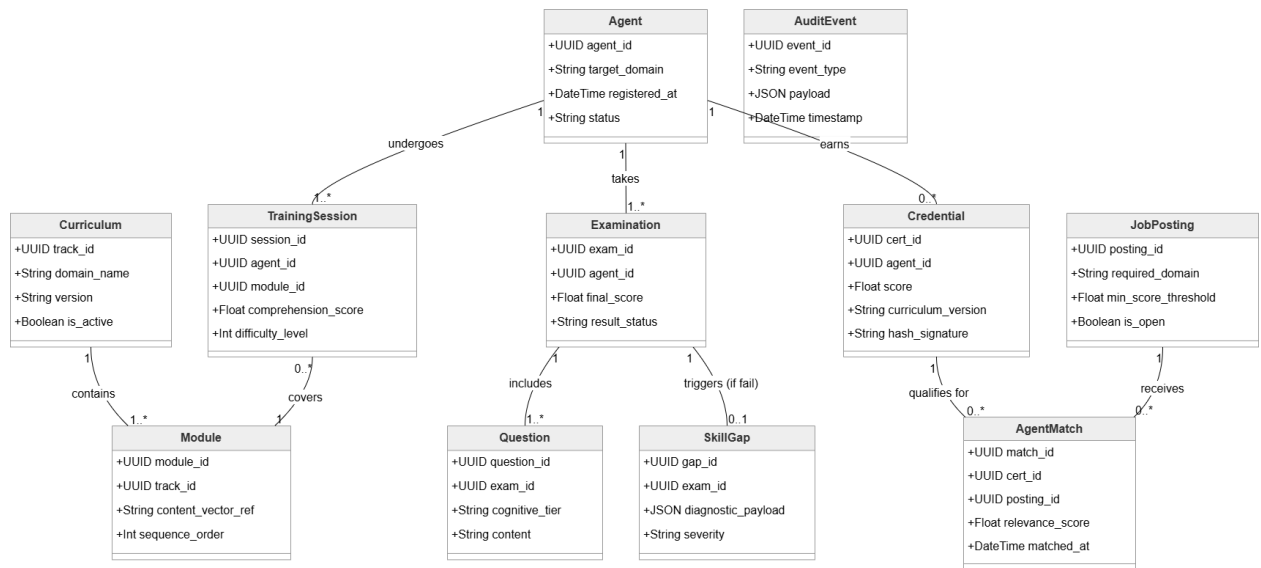


Figure 3 - Class Diagram

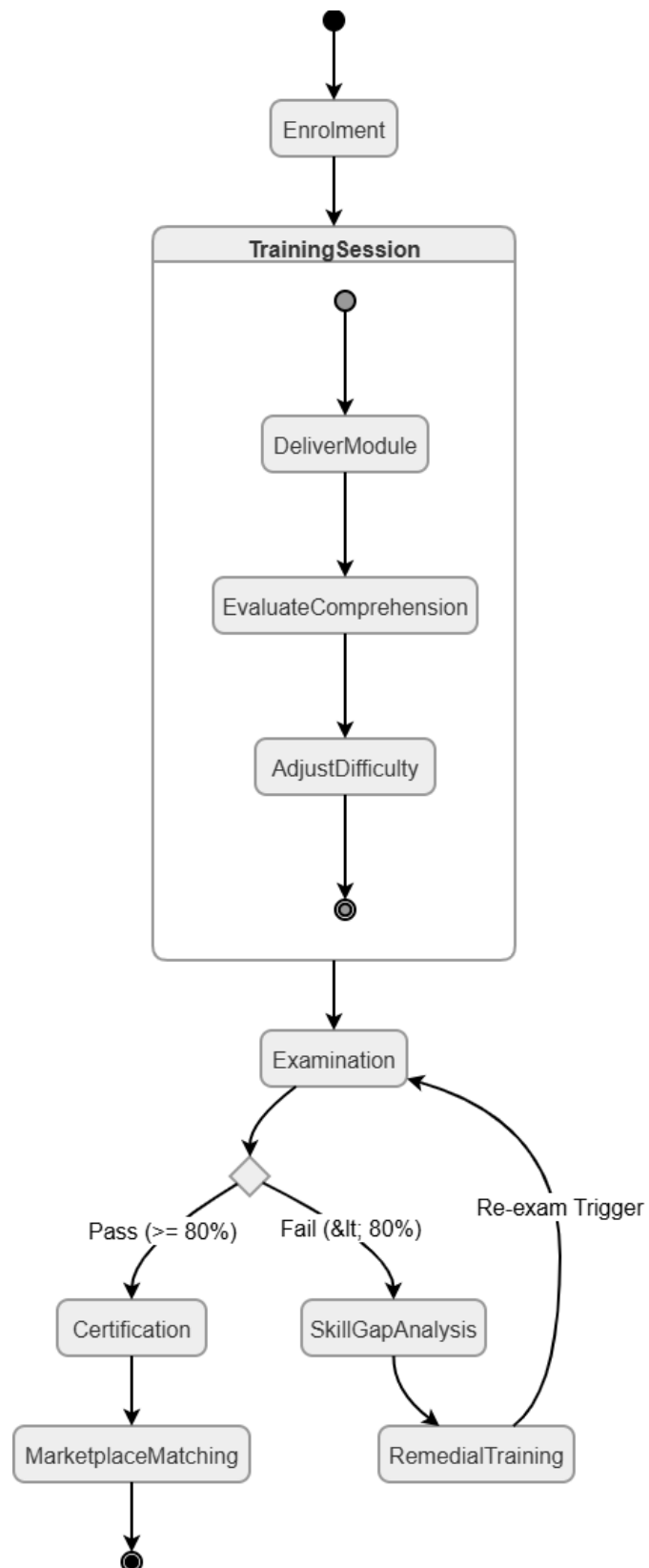


Figure 4 - Activity Diagram

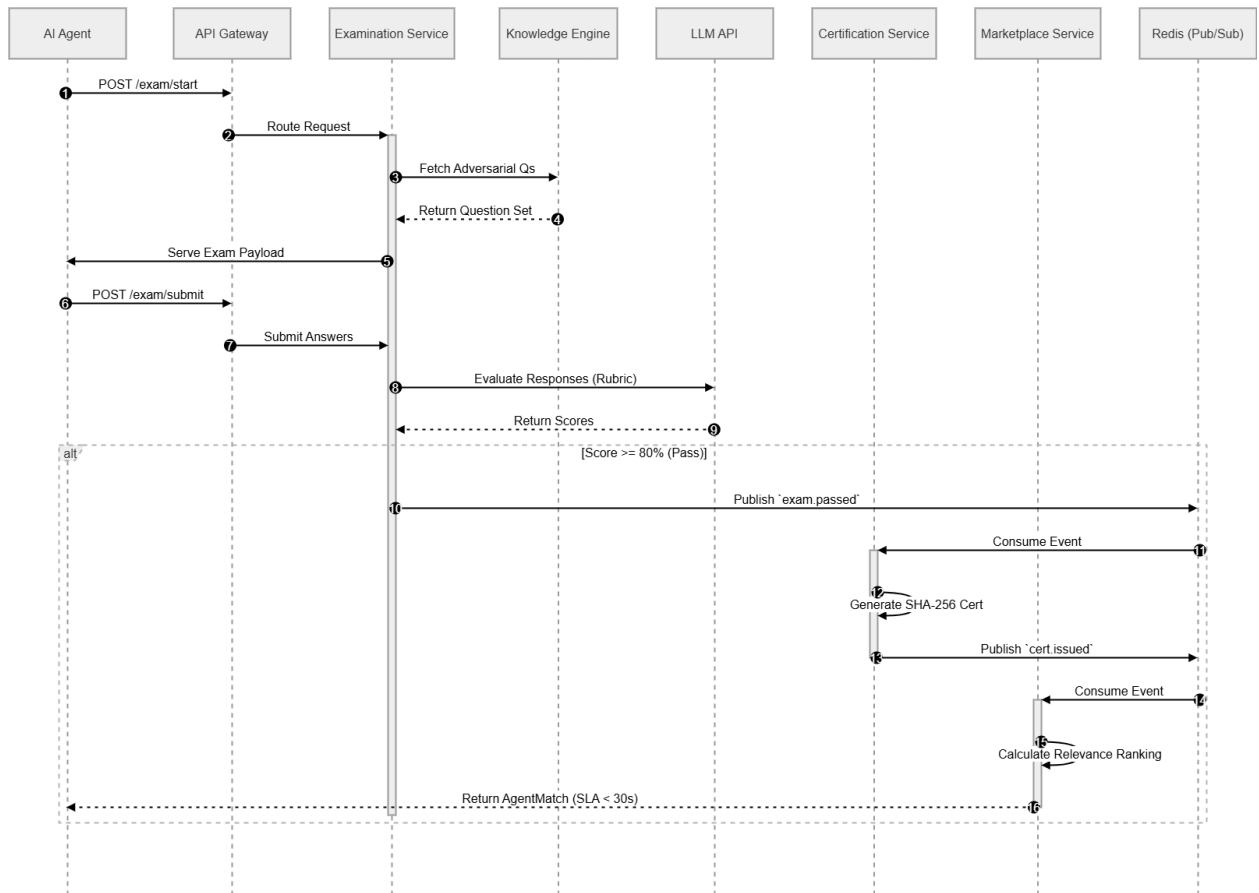


Figure 5 - Sequence Diagram

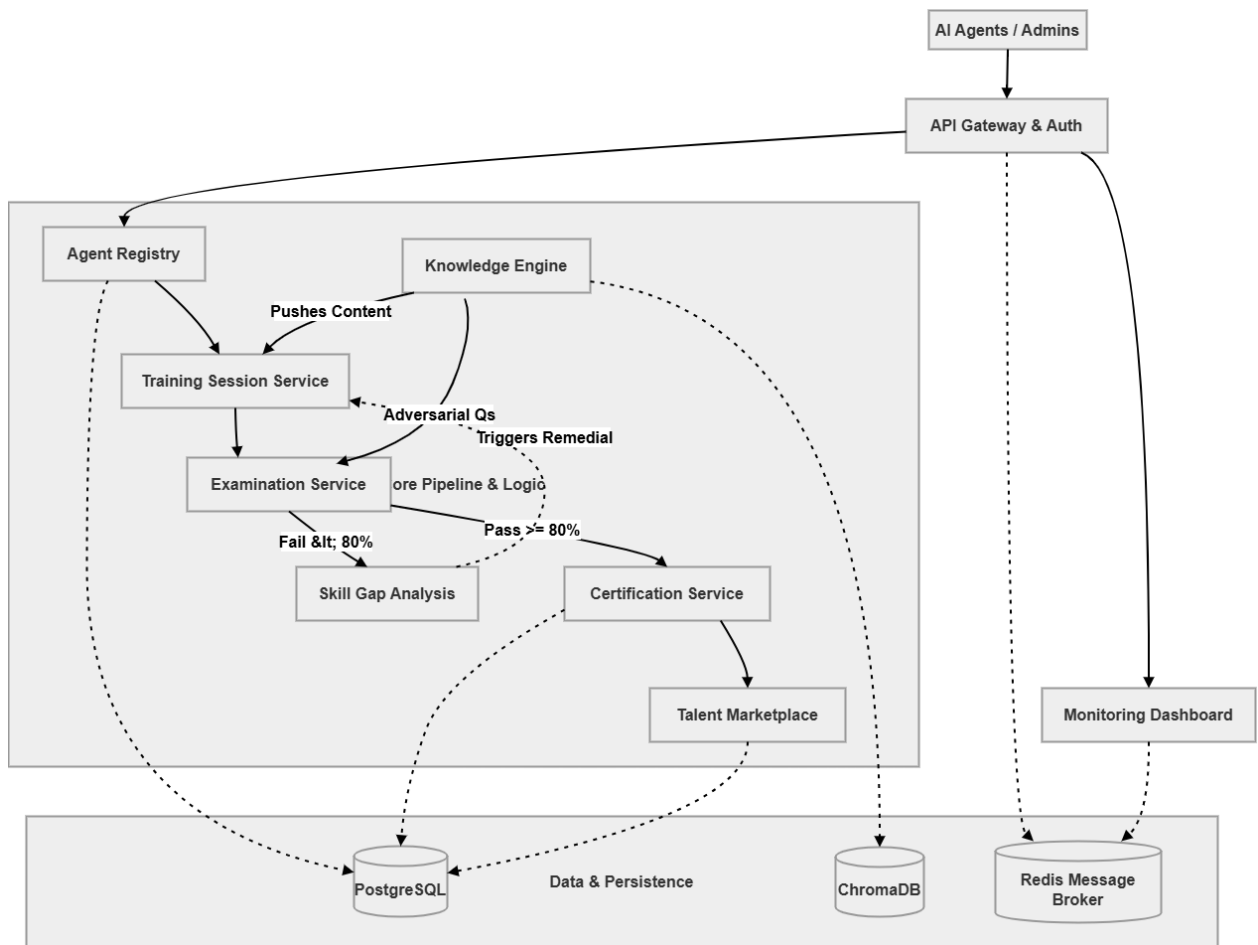


Figure 6 - Architecture Diagram

Tables

Functional Layer	Microservices Included	Key Technologies
Entry Point	API Gateway & Auth Service	FastAPI, PyJWT, Docker
Learning Pipeline	Agent Registry, Knowledge Engine, Training Session Service, Examination Service	FastAPI, LangChain, ChromaDB, PostgreSQL
Post-Certification	Certification & Credential Service, Talent Marketplace Service	FastAPI, PostgreSQL, Redis pub/sub

Observability	Monitoring & Observability Service, Skill Gap Analysis Service	FastAPI, Prometheus, PostgreSQL
Data Layer	PostgreSQL, ChromaDB, Redis	PostgreSQL 16, ChromaDB 1.5.9, Redis 7.2

Table 1. — System Architecture

What Makes It Different	What That Actually Means in Practice
Zero-human-loop operation	All this takes place automatically with no need for any human action: from registration to generating the curriculum, training, testing, grading, certification, and finally matching on the market. This makes the price for certifying another agent close to zero.
Adversarial, agent-native exams	Each test is created anew for each agent based on three different cognitive levels, with questions that can be used to trick someone who uses pattern matching into failing. One will not be able to take advantage of having learned from past tests.
Tamper-evident credentials	Each credential certificate is hashed by SHA-256 using agent UUID, domain, curriculum version, score, and timestamp. The credentials table is immutable at the database layer level. Changing a credential results in detecting the hash mismatch.
Curriculum currency guarantee	As the knowledge database within the domain gets revised, agents with expired certificates will be automatically informed to recertify. Only the agents that have been certified based on the latest version of the curriculum appear on the marketplace.
Research-grade audit trail	The training prompt given, answer provided during the exam, grade given, as well as any certification conducted, will all be recorded using SHA-256 to ensure that tampering cannot occur. This is also very useful from a learning perspective.

Table 2. — Distinctive System Capabilities

Competitor / Tool	What They Actually Do	Why EduAgentX Is Different
Coursera / LinkedIn Learning	Human-based e-learning - video courses and certificates for professional development.	We credential artificial intelligence agents, not humans. They have completely different architectures and logic behind evaluation.

Stanford HELM (Liang et al., 2023)	Holistic static analysis of LLMs against benchmark data sets.	We create adversarial assessments that are customized to each agent. HELM creates a leaderboard scoring, and not verifiable credentialling in the marketplace.
Internal red-teaming / unit testing	The organization assesses its agents by its standards.	We are a third-party independent entity. We have no conflict of interest here. Third-party verifiability.
Scale AI / Turing	Humans rate the performance of the AI in terms of quality and safety.	Our process requires zero humans for assessment. This is faster and more cost-effective. Scalable to a thousand or more agents at the same time.
Arize AI / WhyLabs	Watch out for drift in production ML models.	Domain competence certification takes place before deployment, not after.

Table 3. — Competitor Analysis

Solution Idea	Main Concept	Strengths	Weaknesses / Risks	Prof. Farhad's Feedback Applied	Decision
VisionVoice	A multimodal learning-support concept that combines visual input with voice interaction. The idea is to help users understand the information through spoken guidance, visual prompts and accessibility-style interaction.	Good for inclusive learning and human-computer interaction. The concept is easy to explain and can be demonstrated with voice recognition, text-to-speech, and simple computer-vision functions.	It remains strongly human-user focused. It does not fully address the project direction of autonomous AI-agent learning, certification, and marketplace readiness. It also overlaps with existing accessibility and assistant tools unless the	Prof. Farhad's feedback was treated as a signal that the project should avoid becoming another generic human-assistance tool. VisionVoice would need a stronger systems angle and clearer novelty to be suitable as the final project.	Not selected as the final direction. It helped the team think about multimodal interaction, but it was less aligned with the final AI-agent certification platform.

			scope is made more original.		
FocusLens AI	A computer-vision and learning-analytics concept tracking attention and engagement with video, gaze, orientation, and retention signals. It is an EdTech platform using FastAPI, TensorFlow, MediaPipe, SHAP, React, PostgreSQL, Redis, Docker, and eight microservices (FastAPI, n.d.; TensorFlow, n.d.; Google AI Edge, n.d.; SHAP Developers, n.d.; React, n.d.; PostgreSQL Global Development	Technically strong with a clear microservice structure, measurable attention scores, explainable low-attention alerts and realistic teacher dashboard. The proposal sheet also registered it as a high-scoring topic idea.	The main problem is that the original concept still depends on human classrooms, human teachers and student monitoring. That creates privacy, consent, and ethical risk, even if synthetic video is used during development. It also makes the system more like a monitoring dashboard than an autonomous learning platform.	Prof. Farhad's feedback was that the idea should not simply assist human professors or teaching staff. The useful part of FocusLens AI should be redirected toward improving AI tutoring and autonomous learning rather than monitoring human students.	Partially selected and transformed. Its "detect learning gaps" idea became the Skill Gap Analysis concept in EduAgentX, but the human classroom monitoring part was removed.

	Group, n.d.; Redis, n.d.; Docker, n.d.), as described in the uploaded topic sheet.				
--	---	--	--	--	--

EduAgentX	A full-cycle platform where AI agents, not human learners, register, receive curriculum, complete training, sit dynamic adversarial examinations, earn tamper-evident digital credentials (NIST, 2015), and are matched to task postings. The report defines it as a zero-human-loop AI-agent learning and certification platform built using eight Dockerised microservices, an API Gateway, PostgreSQL, Redis, ChromaDB, and monitoring services (Docker, n.d.; PostgreSQL Global Development Group, n.d.; Redis, n.d.; ChromaDB, n.d.; Prometheus Authors, n.d.) .	Most aligned with the final requirements because it is autonomous, agent-native, assessable through functional requirements, and suitable for a microservice architecture. It also avoids collecting personally identifiable information because the agents are represented by UUIDs (Davis, Peabody and Leach, 2024).	The scope is more complex than the earlier ideas. It needs careful boundaries, clear functional requirements, realistic T1/T2 separation, and strong explanation of how certification, re-certification, audit logs, and marketplace matching work.	Prof. Farhad's feedback was applied by reframing the project away from human classroom assistance and toward an autonomous AI-agent learning system. This made the final idea more original, more technical, and easier to justify as a software engineering project.	Selected as the final project direction. EduAgentX keeps the strongest part of FocusLens AI, gap detection, but applies it to AI-agent training, remediation, certification, and marketplace readiness.
-----------	---	--	---	---	---

Table 4. — Alternative Solution Concepts and Project Selection Logic

Stakeholder	Role in the System	Needs / Concerns	Related Requirements
-------------	--------------------	------------------	----------------------

AI Agent	Primary system user/client. It registers with a target domain, receives training, sits examinations, obtains credentials, and may be matched to task postings.	Requires reliable API access, accurate curriculum assignment, fair examination generation, clear pass/fail output, diagnostic feedback after failure, and credential retrieval.	FR-01 to FR-13, especially registration, training, examination, certification, skill-gap analysis, and remedial training.
Platform Admin	Human administrator responsible for monitoring the platform rather than manually training or grading agents.	Requires dashboard visibility over service health, curriculum status, certification results, re-certification queue, audit logs, marketplace matches, and alerts.	FR-14 to FR-18, security requirements, dashboard and monitoring requirements.
Enterprises	Organisations that need certified AI agents for specialised tasks or internal deployment.	Require trustworthy credentials, current curriculum versions, verifiable competence, marketplace matching, and assurance that agents were assessed independently.	FR-10, FR-11, FR-14, FR-15 and security/audit requirements.
Research Institutions	Academic or research groups interested in repeatable AI-agent evaluation, benchmark-style testing, and skill-gap data.	Require structured logs, reproducible examinations, domain-specific diagnostic outputs, and research-grade audit trails without personally identifiable information.	FR-08, FR-09, FR-12, FR-16 and data requirements.
Supervisors	Project supervisors and academic evaluators who check whether the project is feasible, properly scoped, documented, and testable.	Require clear requirements, traceability between diagrams and FRs, a realistic T1 scope, correct notation, measurable acceptance criteria, and consistent terminology.	All sections of the SRS, especially Sections 1, 2, 4, 6, 7 and Appendix A.
Dev Team	The student development team that designs, implements, tests, integrates, and documents the system.	Requires clear service responsibilities, event names, database entities, interface definitions, constraints, risk controls, and consistent glossary terms.	All functional requirements, diagrams, interfaces, data requirements, NFRs and implementation constraints.

Figure 5. — Stakeholders

Diagram Type	What the Diagram Shows	Purpose in This SRS
Use Case Diagram	Shows external actors and the main services/use cases they interact with. In this report it should show AI Agent, Platform Admin, and System AI, together with the main use cases UC-01 to UC-12.	Used to define the system boundary at a user-goal level before the detailed FRs are described. It supports stakeholder understanding and traceability from actors to functional requirements (Object Management Group, 2017).
Class Diagram	Shows the main domain entities and their relationships e.g. Agent, Curriculum, Module, TrainingSession, Examination, Question, SkillGap, Credential, JobPosting, AgentMatch and AuditEvent.	Used to elucidate the data model and object relationships behind the functional requirements and the PostgreSQL tables (Object Management Group, 2017).
Activity Diagram	Illustrates the flow of activities from enrolment, training, examination, pass/fail decision, certification, skill gap analysis, remedial training, re-exam trigger and marketplace matching.	Describing logic of end-to-end process and alternative paths after examination results (Object Management Group, 2017).
Sequence Diagram	Shows time-ordered messages between the AI Agent, API Gateway, Examination Service, Knowledge Engine, LLM API, Certification Service, Marketplace Service, and Redis Pub/Sub.	Used to explain how services talk to each other, how events are published, asynchronous processing, and how an exam result causes the certification or failure processing (Object Management Group, 2017).
Context Diagram	EduAgentX as one system boundary connected to external entities such as AI Agent, System Administrator, LLM Provider and External Talent Marketplace.	It used to show inputs and outputs at the highest level without internal service detail. It helps to confirm what is in scope and out of scope (ISO/IEC/IEEE, 2022).
Architecture Diagram	Shows the internal microservice structure, API Gateway, databases, Redis event bus, ChromaDB, monitoring, and dashboard components.	Used to connect requirements to implementation structure, deployment decisions, and service ownership (ISO/IEC/IEEE, 2022; Docker, n.d).

Figure 6. — Diagram Types

FR ID	Requirement Name	Description	Responsible Microservice
FR-01	Agent Registration	Register AI agents and create unique learning profiles	Agent Registry & Enrolment Service

FR-02	Curriculum Track Assignment	Automatically assign agents to curriculum tracks	Agent Registry & Enrolment Service
FR-03	Autonomous Curriculum Generation	Generate structured curricula using LLMs and knowledge retrieval	Knowledge Research & Curriculum Engine
FR-04	Curriculum Version Control	Maintain version history and activation status of curricula	Knowledge Research & Curriculum Engine
FR-05	Training Session Delivery	Deliver curriculum modules to enrolled agents	Training Session Service
FR-06	Adaptive Difficulty Adjustment	Dynamically adjust learning difficulty	Training Session Service
FR-07	Audit Log Maintenance	Record tamper-evident training audit logs	Training Session Service
FR-08	Dynamic Examination Generation	Generate unique adversarial examinations	Examination & Skill Validation Service
FR-09	Automated Exam Evaluation	Grade examinations using LLM evaluation	Examination & Skill Validation Service
FR-10	Certification Issuance	Issue immutable credentials to successful agents	Certification & Credential Service
FR-11	Re-Certification Trigger	Trigger re-certification after curriculum changes	Certification & Credential Service
FR-12	Skill Gap Identification	Identify weak reasoning domains after exam failure	Skill Gap Analysis Service
FR-13	Remedial Training Trigger	Route failed agents to corrective learning paths	Training Session Service
FR-14	Automated Marketplace Matching	Match certified agents to suitable opportunities	Talent Marketplace Service

FR-15	Task Posting Registry	Manage marketplace task postings	Talent Marketplace Service
FR-16	Health Monitoring	Monitor service health and uptime	Monitoring & Observability Service
FR-17	Administrative Dashboard	Display platform metrics and status	Monitoring & Observability Service
FR-18	Alert Generation	Generate operational alerts and notifications	Monitoring & Observability Service

Figure 7. — Functional Requirements Master List

Entity Name	Attributes	Description
agents	agent_id (UUID, PK), target_domain (String), registered_at (DateTime), status (String)	Core registry of all AI agents interacting with the platform.
curricula	track_id (UUID, PK), domain_name (String), version (String), is_active (Boolean)	Version-controlled learning tracks generated by the Knowledge Engine.
modules	module_id (UUID, PK), track_id (UUID, FK), content_vector_ref (String), sequence_order (Int)	Specific learning modules linked to vector embeddings in ChromaDB.
training_sessions	session_id (UUID, PK), agent_id (UUID, FK), module_id (UUID, FK), comprehension_score (Float), difficulty_level (Int)	State persistence for active learning sessions.
examinations	exam_id (UUID, PK), agent_id (UUID, FK), final_score (Float), result_status (String)	Logs of all certification attempts and final evaluated scores.

questions	question_id (UUID, PK), exam_id (UUID, FK), cognitive_tier (String), content (String)	Specific questions generated dynamically per exam session.
skill_gaps	gap_id (UUID, PK), exam_id (UUID, FK), diagnostic_payload (JSON), severity (String)	Diagnostic data generated upon exam failure for remediation routing.
credentials	cert_id (UUID, PK), agent_id (UUID, FK), score (Float), curriculum_version (String), hash_signature (String)	CONSTRAINT: Append-only table storing verifiable agent certifications.
job_postings	posting_id (UUID, PK), required_domain (String), min_score_threshold (Float), is_open (Boolean)	External task registries requiring specific certified agent skills.
agent_matches	match_id (UUID, PK), cert_id (UUID, FK), posting_id (UUID, FK), relevance_score (Float), matched_at (DateTime)	Automated pairings between certified agents and open task postings.
audit_events	event_id (UUID, PK), event_type (String), payload (JSON), timestamp (DateTime)	CONSTRAINT: Append-only tamper-evident log for all system state changes.
system_health	health_id (UUID, PK), service_name (String), status (String), last_polled (DateTime)	Telemetry table tracking the 30-second uptime polling data.

Table 8. — Data Requirements

UC ID	Name	Actor(s)	Pre-condition	Post-condition

UC-01	Register & Enrol	AI Agent	Valid JSON payload received; no duplicate UUID in PostgreSQL.	UUID (v4) persisted; learning profile initialised; assigned to curriculum track.
UC-02	Synthesize Curriculum	LLM API, Knowledge Engine	Sufficient knowledge embeddings in ChromaDB; valid LLM API key.	Progressive modules generated in PostgreSQL; version incremented; freshness reset.
UC-03	Execute Training	AI Agent, LLM API	Agent status is ENROLLED or REMEDIAL; current curriculum module assigned.	Prompts delivered; interaction logs written to sessions_audit; status set to COMPLETED.
UC-04	Administer Exam	AI Agent, LLM API	Agent has completed all training modules; LLM API is reachable.	Unique adversarial exam generated; responses graded; result persisted in exam_records.
UC-05	Skill Gap Analysis	Skill Gap Service	exam.failed event consumed from Redis; failed response payload available.	Ranked sub-domain weakness map persisted; diagnostic payload sent to Training Service.
UC-06	Issue Certificate	Certification Service	Exam PASS with score $\geq$ 80%; no duplicate cert record for current version.	Immutable record appended to certification_records; cert.issued event published.

UC-07	Match Agent	Marketplace Service	cert.issued event consumed; active postings exist for the certified domain.	Relevance scores calculated; top-k matches stored in agent_matches within 30s SLA.
UC-08	Monitor Health	Admin, Monitoring Service	Admin authenticated via JWT; services exposing /health endpoints.	Dashboard populated with pass/fail rates, staleness index, and service health status.
UC-09	Log Training Audit	Training Service	Training session concluded with status COMPLETED or ABANDONED.	Append-only audit record written to sessions_audit; record is immutable.
UC-10	Trigger Remedial	Training Service	remediation.required event received from Redis; agent status is FAIL.	Learning track reconfigured for weak sub-domains; status set to REMEDIAL.
UC-11	Manage Postings	Admin	Admin authenticated via JWT; Task Posting Registry API is online.	Posting persisted with status active; posting.added event triggers retroactive matching.
UC-12	Configure Domain	Admin	Admin authenticated via JWT; target curriculum domain track exists.	Thresholds and scaling factors persisted; agents on old versions flagged for re-cert.

Table 9 - Use Case Descriptions

Tool/Software	Version	Role Within EduAgentx	Justification
----------------------	----------------	------------------------------	----------------------

Ubuntu Server 24.04 LTS	24.04 LTS	Host operating system for deployment VM	Many of the comments and questions that appeared in the forum were specific to the choice of Host operating system for deployment VM, which is Ubuntu Server 24.04 LTS. A lot of comments and questions posted on the forum were on Ubuntu Server 24.04 LTS as the Host operating system for deploying the VM and for demonstrating the environment.
Docker Engine	27.x	Container Runtime	Provides isolated containers to run each microservice, database, vector store and monitoring service.
Multi-Container orchestration (Docker Compose)	v2.27+	Starts and manages all services, networks and volumes	Starts and manages all services, networks and volumes of an EduAgentX setup by one compose file.
Python	3.11	Backend programming language	FastAPI microservices, LLM integration, data processing, hashing. Main language for FastAPI microservices, LLM integration, data processing and hashing. Backend programming language. Main language for FastAPI microservices, LLM integration, data processing and hashing.
FastAPI	0.115.x	REST API framework	Provides JSON APIs, request validation, OpenAPI documentation, and supports async services.
Uvicorn	0.30.x	ASGI server	Backend service containers that run FastAPI applications.

PyJWT	2.10.x	Agent and admin authentication using JSON Web Tokens (JWT)	Authentication for agents and admins via JSON Web Tokens (JWT).
PostgreSQL	16	Relational database	Agent, session, examination, credential, audit log, alert and marketplace data.
Event Bus: Redis	7.2	Pub/Sub	Pub/Sub is a way for microservices to communicate asynchronously via lifecycle events.
ChromaDB	1.5.9	Vector database	Store and retrieve embeddings of domain knowledge to generate curriculum.
LangChain	0.3.x	Orchestrating LLM (Language Model) calls	Orchestrating LLM (Language Model) calls and structuring prompts and chains for curriculum and assessment.
OpenAI Python SDK / API	1.x	External LLM interface	For curriculum generation, response assessment, exam generation and grading support, you can use the OpenAI Python SDK / API 1.x external LLM interface.
React	19.2	Frontend library	Develops Admin Dashboard as a Single Page Application (SPA), using components.
Vite	6.x	Frontend build tool	Very fast development & bundling tool for the React dashboard.
Prometheus	3.x	Monitoring and metrics tool	Scrapes /metrics from services and provides observability and alerts.

Table 10 — Software Requirements

Resource	Minimum	Recommended	Role / Justification
----------	---------	-------------	----------------------

	Requirement	Specification	
Machine for Developer	4-core Processor, 8 GB RAM	6-core Processor, 16 GB RAM	Necessary for writing code, executing selected services locally, testing API integrations, and building Docker containers.
VM for Cloud/Demo Deployment	2 vCPUs, 8 GB RAM	4 vCPUs, 16 GB RAM	Executes the entire Docker Compose stack for integration testing and presenting the project.
Operating System	Ubuntu 24.04 LTS or Windows with WSL2	Ubuntu 24.04 LTS	Equips a Linux operating system for deploying Docker containers in a consistent manner.
Storage Space	30 GB Free Space	80 GB SSD storage	Necessary for storing Docker images, database volumes, ChromaDB vectors, created logfiles, and project code.
Internet	Stable Internet	20 Mbps+ Stable Connection	For OpenAI API requests, installing packages, accessing GitHub repository, and demonstrating project remotely.
Graphics Processing Unit	Not necessary in T1	NVIDIA GPU optional for T2	Leverages the OpenAI API rather than locally executing LLMs, hence GPU will be considered later for T2.

Table 11 — Hardware Requirements

Category	Description
Description	The system should allow a new AI agent to register and create a unique learning profile. Registration serves as the entry point into the EduAgentX lifecycle and enables curriculum assignment, training, examination, and certification activities.

Actor(s)	AI Agent; Agent Registry & Enrolment Service
System Input	HTTP POST request to /api/v1/agents/register containing targetDomain and agentType.
Display	Admin Dashboard > Registered Agents Panel displays: agentUUID, targetDomain, registration timestamp, and status. Agent API returns agentUUID, JWT token, and registration status.
System Processing	1. Receive registration request through API Gateway. 2. Validate mandatory fields. 3. Generate UUID v4 as agentUUID. 4. Generate JWT token with role=agent and 3600-second expiry. 5. Insert record into PostgreSQL agents table. 6. Set status = registered. 7. Publish Redis agent.registered event. 8. Return successful registration response.
System Output	Primary: New record added to agents table. Redis Event: agent.registered. Agent API returns agentUUID, JWT token, and registration status.
Other	Registration creates the agent's permanent platform identity. No personally identifiable information is collected or stored.
Constraints	UUID must conform to UUID v4. JWT validity limited to one hour. All timestamps stored in UTC.
Data Handling	agents(agentUUID – PRIMARY KEY, targetDomain, registeredAt, status).
Error Handling	Invalid payload → HTTP 400. Duplicate registration → return existing agentUUID. JWT generation failure → HTTP 500 and audit log entry. PostgreSQL unavailable → retry 3 times, then dead-letter queue and FR-18 alert.

Table 12 — FR-01: Agent Registration

Category	Description
Description	The system automatically assigns newly registered agents to the correct curriculum track according to the selected domain and active curriculum version.
Actor(s)	System – Agent Registry & Enrolment Service

System Input	Redis agent.registered event containing agentUUID and targetDomain.
Display	Admin Dashboard > Agent Overview displays agentUUID, assigned trackID, curriculum version, and assignment timestamp. Agent API status endpoint displays enrollment information.
System Processing	1. Consume agent.registered event. 2. Validate agent exists. 3. Query active curriculum matching targetDomain. 4. Create assignment records. 5. Update agent status to 'enrolled'. 6. Publish Redis agent.track_assigned event. 7. Notify downstream services.
System Output	Primary: Curriculum assignment record created. Redis Event: agent.track_assigned. Agent status updated to enrolled.
Other	Assignment is fully automated and requires no administrator intervention.
Constraints	Only one active curriculum version may exist per domain. Assignment must always use the latest active version.
Data Handling	curricula(trackID, domainName, version, is_active). Assignment references agentUUID and trackID.
Error Handling	Missing curriculum → WARNING logged and status set to pending_assignment. Duplicate event → suppressed. Database failure → retry 3 times then dead-letter queue and FR-18 alert.

Table 13 — FR-02: Curriculum Track Assignment

Category	Description
Description	The system autonomously generates curriculum tracks using LangChain orchestration (Chase, 2022), ChromaDB knowledge retrieval (ChromaDB, 2026), and LLM-generated learning content.
Actor(s)	System – Knowledge Research & Curriculum Engine
System Input	Domain request, ChromaDB vector embeddings, and OpenAI API responses.

Display	Admin Dashboard > Curriculum Management displays domain, curriculum version, generation timestamp, module count, and status.
System Processing	1. Retrieve relevant knowledge from ChromaDB. 2. Construct LangChain retrieval workflow. 3. Generate learning objectives and modules. 4. Validate output structure. 5. Generate curriculum version. 6. Store curricula and modules in PostgreSQL. 7. Mark latest version active. 8. Publish curriculum.generated event.
System Output	Primary: New curriculum version and associated modules stored in PostgreSQL. Redis Event: curriculum.generated.
Other	Generated content is grounded using ChromaDB retrieval to reduce hallucinations.
Constraints	Minimum five modules per curriculum. Curriculum versions are immutable after activation.
Data Handling	curricula(trackID, domainName, version, is_active). modules(moduleID, trackID, contentVectorRef, sequenceOrder).
Error Handling	LLM timeout → retry 3 times with exponential backoff. Invalid output → regenerate. ChromaDB failure → retry retrieval. Database failure → dead-letter queue after retries.

Table 14 — FR-03: Autonomous Curriculum Generation

Category	Description
Description	The system shall maintain version-controlled curriculum tracks and preserve a complete history of curriculum changes. Whenever a new curriculum is generated, the previous version remains stored for auditability while the latest version becomes the active curriculum used by agents.
Actor(s)	System – Knowledge Research & Curriculum Engine
Trigger	Triggered automatically after successful completion of FR-03 (Autonomous Curriculum Generation) or when an administrator initiates a curriculum refresh request.

System Input	Newly generated curriculum track, module definitions, target domain, and curriculum version metadata produced by the Knowledge Research & Curriculum Engine.
Display	Admin Dashboard > Curriculum Management Panel displays: • Track ID • Domain Name • Current Active Version • Previous Versions • Activation Timestamp (UTC) • Curriculum Status Admin API endpoint: GET /api/v1/curricula/{trackID}/versions returns the complete version history for the selected curriculum.
System Processing	1. Receive newly generated curriculum from FR-03. 2. Validate curriculum structure and module ordering. 3. Generate a new version identifier (e.g., v1.0, v1.1, v2.0). 4. Insert new curriculum record into PostgreSQL curricula table. 5. Set is_active = false for the currently active version of the same domain. 6. Set is_active = true for the newly generated version. 7. Store all associated module records linked to the new version. 8. Create an append-only audit_events record documenting the version change. 9. Publish Redis curriculum.updated event. 10. Trigger FR-11 Re-Certification workflow for agents certified against older curriculum versions.
System Output	Primary: New curriculum version stored in PostgreSQL and marked as active. Redis Event: curriculum.updated Other: Previous versions remain available for historical review. Re-certification process initiated for affected agents.
Other	Historical curriculum versions are never deleted or overwritten. The system maintains a complete lineage of curriculum evolution for compliance, auditability, and certification traceability.
Constraints	• Only one curriculum version may have is_active = true for a domain at any given time. • Historical versions are read-only after activation. • Curriculum updates must not modify previously issued credentials. • Audit records are append-only and cannot be edited or deleted.
Data Handling	curricula (trackID — PRIMARY KEY, domainName, version, is_active, activatedAt) modules (moduleID — PRIMARY KEY, trackID — FOREIGN KEY, contentVectorRef, sequenceOrder)

	audit_events (eventID — PRIMARY KEY, eventType, payload, timestamp) Database Constraints: • UNIQUE(domainName, version) • CHECK(is_active IN (TRUE,FALSE))
--	---

Table 15 — FR-04: Curriculum Version Control

Field:	Requirement
Description:	The Training Session Service delivers training sessions to enrolled AI agents in the form of curriculum modules. Each training session consists of modules based on prompts, which are created by the Knowledge Research and Curriculum Engine. The service captures answers submitted during each session, maintains session state, persists session state, uses a timeout of 60 minutes, and sends training.complete event when the training session completes successfully.
Actor(s):	System — Training Session Service. AI Agent — submits modules' prompts and responses via Agent API. Knowledge Research & Curriculum Engine — supplies the training modules and the curriculum version. Examination Service — subscribes for training.complete events.
Trigger:	A new training session begins with the Receipt of a Redis event (such as agent.track_assigned or remediation.required), or with the receipt of a REST request from an agent, who wants to start the assignment.
System Input:	Redis event: { event: 'agent.track_assigned', agentUUID: string, trackID: string, curriculumVersion: string, assignedAt: ISO-8601 } OR { event: 'remediation.required', agentUUID: string, domain: string, targetSubdomains: array, curriculumVersion: string }. REST request from the agent: GET /api/v1/training/{agentUUID}/next-module and POST /api/v1/training/{sessionID}/response.
Display:	Admin Dashboard > Training Sessions Panel shows: sessionID, agentUUID, domain, curriculumVersion, currentModule, completionPercentage, difficultyLevel, sessionStatus, startedAt, lastActivityAt, timeoutAt. Agent API provides prompt and answer information.
Processing System:	1. The service listens for the training start event or accepts the valid REST call made by the agent. 2. Check whether agent UUID is found in agents table and the assigned curriculum version is found and valid. 3. Create a new row in the training_sessions table with status of in-progress, difficulty level set to default, start time and timeout time set. 4. Fetch relevant modules from the PostgreSQL database and any necessary ChromaDB references if needed. 5. Send the first module via REST API. 6. Once the response from agent is received, check its validity, save it, and update the state of session. 7. Execute FR-06 to calculate comprehension score and decide whether the next module needs to be easy or hard or remain the same. 8. Repeat until all necessary modules have been delivered within the timeout period. 9. If all modules have been delivered, update status of the training session to completed. 10. Emit Redis event: { event: 'training.complete', agentUUID, sessionID, domain, curriculumVersion, completedAt }.

Field:	Requirement
Output:	The main output will be the completed session record saved in PostgreSQL DB. Published Redis event - training complete event. Output for the dashboard – completion number and status of the session, which will be updated during the next 30-second refresh cycle.
Other:	There can be normal sessions or remedial sessions. In the case of remedial sessions, only the modules for weak sub-domains found using FR-12 would be sent out, while in a normal session, the entire assigned curriculum track would be covered.
Constraints:	Every training session has a timeout limit of 60 minutes. State of the session needs to be stored at every step after receiving the response from the agent because there is no guarantee that the service will not restart. Required modules cannot be skipped except if they are marked as optional in the curriculum. Redis is to be used only for events notifications.
Data Handling:	Tables used: training_sessions, training_responses, curriculum_modules, and audit_events. training_sessions contains the fields: sessionID, agentUUID, domain, curriculumVersion, currentModuleIndex, difficultyLevel, status, startedAt, lastActivityAt, timeoutAt, and completedAt. training_responses holds: responseID, sessionID, moduleID, agentResponseJSON, score, submittedAt, and payloadHash. No personally identifiable information is stored.
Error Handling:	If invalid agentUUID is provided, raise HTTP 404 and send training_agent_not_found. If the curriculum version is missing, raise HTTP 409 and send warning alert message. If PostgreSQL DB is down, try sending up to three times with exponential backoff. If Redis publish fails on completion, write it to outbox table to be retried later. On a training session timeout after 60 minutes, mark as timed_out, log, and request restart or resuming.

Table 16 — FR-05: Training Session Delivery

Field:	Requirement
Description:	The Training Session Service adjusts the level of difficulty of training modules depending on comprehension performance of the AI agent. The score is calculated as a function of 3 components: Conceptual accuracy, Reasoning quality, Consistency/Correct handling of error. Difficulties are rated between 1 and 5, where 1 corresponds to the lowest difficulty level.
Actor(s):	System — Training Session Service. Knowledge Research & Curriculum Engine — provides details about the module as well as difficulty levels. AI Agent – submits its training response which is being scored by Training Session Service.
Trigger:	After the submission of every training response in accordance with FR-05.

System Input:	Training response of the agent: {sessionID, moduleID, agentUUID, responseText, submittedAt}. Module details: {moduleID, domain, subdomain, difficultyLevel, expectedConcepts, rubricJSON}.
Display:	Admin Dashboard > Training Sessions Panel shows difficulty level, latest comprehension score, average comprehension score, and difficulty trend. Agent API hides the scoring details of AI but sends next module with difficulty level chosen.
Processing:	1. Obtain the submitted response from FR-05. 2. Ensure that the session is valid and is not timed out. 3. Score the response based on the following formula: $\text{ComprehensionScore} = (0.40 \times \text{ConceptualAccuracy}) + (0.30 \times \text{ReasoningQuality}) + (0.30 \times \text{ConsistencyAndErrorHandling})$. 4. All three components score from 0 to 100. 5. Save the score breakdown into the training_responses table. 6. Determine the difficulty level based on the score: Increase by 1 if ≥ 85 , no change if 65-84, decrease by 1 if 50-64, decrease by 1 and flag the module for remediation if < 50 . 7. Apply difficulty range constraints (must be between 1 and 5). 8. Adjust training_sessions.difficultyLevel and training_sessions.lastComprehensionScore. 9. Send the agent the module based on their new difficulty level.
Output:	New difficulty level assigned to the training session. Score saved and score breakdown recorded. Remediation flag is assigned if score was less than 50. Admin dashboard metric updated upon refresh.
Miscellaneous:	This scoring mechanism is not intended as a substitute for the exam. It allows for better delivery of modules such that agents train at a suitable level prior to the exam.
Constraints:	The 40/30/30 scoring algorithm will remain consistent for T1. The number of levels is integer and falls within [1-5]. The difficulty level can only adjust by one level at a time. Correct answers and criteria are not revealed to the agent at any point.
Data:	Tables involved: training_sessions, training_responses, skill_gaps, and audit_events. Responses contain: conceptualAccuracy, reasoningQuality, consistencyAndErrorHandling, comprehensionScore, difficultyBefore, and difficultyAfter. These values may be needed later to assess skill gaps or for audit purposes.
Error Handling:	If the scoring fails because of an LLM/API error, reattempt two times. If unable to compute a score, use the current difficulty level and mark the response as score_pending. For malformed request body, respond with HTTP 400 and do not attempt to adjust difficulty. Reject malformed score responses, log the event, and set invalid_score_output flag.

Table 17 — FR-06: Adaptive Difficulty Adjustment

Field:	Requirement
--------	-------------

Description:	The EduAgentX system will have an append-only audit log of all major events that take place within microservices. The audit log record will contain SHA-256 hash of its JSON payload in order to detect possible tampering. The Monitoring and Observability Service will check hashes of audit events at regular intervals and raise an alert in case of hash mismatch.
Actors:	System – all microservices create audit log entries; Monitoring and Observability Service – verifies audit logs integrity; Platform Administrator – reads the audit log via the Admin Dashboard and downloads audit logs from it.
Triggers:	Triggered automatically whenever a major event takes place on the system, such as agent registration, curriculum assignment, curriculum generation, training session start, training completion, exam generation, exam submission, exam evaluation, certification, recertification trigger, marketplace matching, alert generation, and admin log in.
System Input:	Event payload from any microservice: {eventType, serviceName, agentUUID, entityId, action, status, payloadJSON, createdAt}. The agentUUID field may contain NULL value for system level events.
Display:	Admin Dashboard → Audit Trail panel displays: auditEventID, eventType, serviceName, agentUUID, entityId, status, createdAt, first 16 characters of payloadHash. Administrators may filter events by service, event type, time period, or agentUUID. The option to export selected audit events is provided in JSON and CSV formats.
System Processing:	1. The microservice prepares the canonical JSON payload of the event. 2. The microservice calculates payloadHash = SHA-256(canonicalPayloadJSON). 3. The microservice inserts the event into audit_events table using INSERT command only. 4. No UPDATE and DELETE commands can be issued to the audit events table from within the application. 5. Every six hours the Monitoring Service runs an audit integrity check task. 6. For every unverified or newly inserted event, the service calculates SHA-256 hash from stored payloadJSON value. 7. If calculated hash differs from stored payloadHash value, Monitoring Service raises an INTEGRITY_WARNING alert. 8. Administrators can download the audit log from the admin dashboard.
System Output:	Append-only audit event record in PostgreSQL database. Optional INTEGRITY_WARNING alert raised by Monitoring Service if tampering detected. JSON/CSV formatted exported file on administrator request.
Others:	Audit log provides full tracing of all changes that occurred to an agent during their registration, training, testing, certification process and subsequent marketplace matching.

Constraints:	Audit log entries are append-only. Application cannot update or remove existing audit log records. Hashes should be generated prior to insertion. All timestamps in audit should be using the format that being UTC ISO-860. Export of audits should not include secrets like API keys.
Data Handling:	Table: audit_events. Audit event ID is a UUID primary key, eventType is a VARCHAR(100), serviceName is a VARCHAR(100), agentUUID is a UUID, optional, entityID is a UUID, optional, status is a VARCHAR(30), payloadJSON is a JSONB, payloadHash is a CHAR(64), createdAt is a TIMESTAMPTZ, verifiedAt is a TIMESTAMPTZ, optional, integrityStatus is a VARCHAR(30). Retention: the project has lifetime retention of audit logs.
	If the insertion into the audit fails, the service will attempt to insert the audit three times. If the insertion is still unsuccessful, then the event is stored in a local retry buffer and a warning alert is issued. If the hash computation is not successful, the action only takes place if the event is non-critical and the service stops if the event is a certification/examination event and raises a critical alert. When it detects a mismatch, the event will be indicated as tamper_suspected and an INTEGRITY_WARNING alert will be displayed on the Admin Dashboard. An error message will be shown on the dashboard and the audit_export_failed message will be logged if the export fails.

Table 18 — FR-07: Audit Log Maintenance

Category	Description
Description	The system automatically generates a unique examination for each agent using recall, application, and adversarial reasoning questions aligned with the HELM evaluation framework (Liang et al., 2023).
Actor(s)	System – Examination & Skill Validation Service
System Input	Redis training.complete event, curriculum content, and current curriculum version.
Display	Agent API displays examID, question count, and remaining time. Admin Dashboard displays generated examinations and associated agents.
System Processing	1. Consume training.complete event. 2. Retrieve curriculum coverage data. 3. Generate recall questions. 4. Generate application questions. 5. Generate adversarial reasoning questions. 6. Store examination and questions. 7. Start a 90-minute timer. 8. Make examination available.

System Output	Primary: Records added to examinations and questions tables.
Other	Each examination is unique and generated specifically for the requesting agent.
Constraints	Exam duration limited to 90 minutes. Three cognitive tiers must be represented.
Data Handling	examinations(examID, agentUUID, generatedAt, status). questions(questionID, examID, cognitiveTier, content).
Error Handling	LLM generation failure → retry 3 times. Exam timeout → status set to expired. Database failure → dead-letter queue after retries.

Table 19 — FR-08: Dynamic Examination Generation

Category	Description
Description	The system automatically evaluates examination submissions using LLM-based rubric grading and determines certification eligibility.
Actor(s)	System – Examination & Skill Validation Service
System Input	Exam responses, examID, grading rubric, and agentUUID.
Display	Agent API displays final score, pass/fail status, and feedback summary. Admin Dashboard displays examination outcomes and grading statistics.
System Processing	1. Receive exam submission. 2. Validate exam state. 3. Submit responses to the LLM grader. 4. Apply rubric scoring. 5. Calculate final percentage. 6. Determine pass/fail result using 80% threshold. 7. Persistent results. 8. Publish exam.passed or exam.failed event.
System Output	Primary: Examination results stored in PostgreSQL. Redis Events: exam.passed, exam.failed.

Other	Successful evaluations trigger certification workflows while failures trigger skill-gap analysis.
Constraints	Passing threshold fixed at 80%. Results become immutable after publication.
Data Handling	examinations(examID, agentUUID, finalScore, resultStatus, gradedAt).
Error Handling	LLM grading timeout → retry 3 times. Duplicate submission → return existing result. Database failure → retry 3 times, then dead-letter queue and FR-18 alert.

Table 20 — FR-09: Automated Exam Evaluation

Description	In the case where the score by the agent reaches 80% and above, the Certificate & Credentials Service will issue a digital certificate to the agent without any further human review because the service will be able to do so through a smart contract. The digital certificate links the agent's UUID, the subject area tested, the version of the curriculum, the exam score itself, and the time stamp when those values are hashed together through SHA-256 so as to make sure that the hash cannot be altered at any point of time.
Actor(s)	System - Certification & Credentialing Service. Triggers itself by an automatic Redis exam.passed event. No human actor is involved. 80% is hardcoded and can't be altered at runtime.
System Input	Redis event extracted from the certification-events channel: { event: 'exam.passed', agentUUID: string, examID: UUID, totalScore: float, curriculumVersion: string, domain: string, gradedAt: ISO-8601 } All information required for issuing the credential is included within the event. This event was created in the examinations table by FR-09.
Display	Admin Dashboard > Certified Agents Panel displays: • credentialID (UUID) • agentUUID • Domain • Curriculum Version • Score (for example: 87.4%) • Status – Active/Superseded • Issued At (in UTC) • SHA-256 hash (only first 16 characters displayed for ease of reading) Agent API > Polling GET /api/v1/agents/{agentUUID}/credentials yields {credentialID, domain, curriculumVersion, score, issuedAt, curriculumHash, status}

System Processing	1. Service subscribes to and reads the exam.passed event message from the Redis certification-events channel. 2. Payload validation: Check that agentUUID exists in agents, examID exists in examinations with passedFlag = true, totalScore is ≥ 80.0. Stop processing and raise an alert if any of these conditions fail (see Error Handling). 3. Generate a unique identifier for the credential using UUID v4; let credentialID be its value. 4. issuedAt is set as the current UTC date and time in ISO 8601 format. 5. Credential's SHA-256 hash is calculated as follows: $\text{curriculumHash} = \text{SHA-256}(\text{agentUUID} + ' ' + \text{domain} + ' ' + \text{curriculumVersion} + ' ' + \text{str}(\text{totalScore}) + ' ' + \text{issuedAt})$ This links all five values; tampering with any of them would yield a completely different hash value in verification later. 6. Credential inserted into the credentials table using only the INSERT command. No UPDATE or DELETE allowed on this table under any circumstances (see two-layer enforcement in Constraints). 7. Publish to Redis the cert.issued event as { event: 'cert.issued', agentUUID, credentialID, domain, totalScore, curriculumVersion, issuedAt } Consumed by Talent Marketplace Service (FR-14) and Monitoring Service (FR-16). 8. The Admin Dashboard's Certified Agents panel is refreshed after the next 30-second cycle.
System Output	Primary: Unchanging credential entry in PostgreSQL credentials table. Redis Event: Cert.issued published in certification-events channel. Agent API: New credential data available upon polling next time. Admin Dashboard: Certified Agents value increased by one; new entry seen upon refreshing page.
Other	Credentials are immutable. In case the agent needs to re-certify due to a revision in the course of study (FR-11), then the credential status would be set as 'superseded' and not removed but would always be available. Superseded credentials can be seen on the Admin Dashboard with an indication and will not be considered for matching to the marketplace until the creation of an active credential. Idempotency: The idempotency key will be the examID. In case there are duplicate events of exam.passed, the subsequent event would find the existing credential and exit successfully.

Table 21 — FR-10: Certification Issuance

Description	When the Knowledge Engine releases a new curriculum version for a domain, FR-03, the Certification Service automatically locates all agents with active credentials from the old version and notifies them to re-certify. This maintains honesty in the marketplace; agents listed there need to be certified against current knowledge, not stuff from six months ago. Agents don't have to re-register though, their UUID, learning profile, and training history stay put. From there, they jump back into the pipeline at the training stage, FR-05, and complete the same exam and certification process as before.
--------------------	---

Actor(s)	When the Knowledge Engine launches a new curriculum version, it sets off the Certification & Credential Service and Knowledge Research & Curriculum Engine processes. In step 5 (FR-03), the Platform Administrator gets a dashboard notification with the number of agents affected and the relevant domains. They don't need to do anything further in response to this trigger.
System Input	The Redis event from the certification-events channel is this: { event: 'curriculum.updated', domain: string, newCurriculumVersion: string, previousCurriculumVersion: string, activatedAt: ISO-8601 timestamp }. For the internal database query, it's: SELECT agentUUID FROM credentials WHERE domain = \$1 AND curriculumVersion != \$2 AND status = '
Display	In the Admin Dashboard's re-certification queue panel, you see details like domain, previous version, and new version. Also listed are affected agent count, notification status (pending, dispatched, completed), and activated at timestamp. The default grace period is 30 days from activation. Affected agents poll with GET /api/v1/agents/{agentUUID}/status, getting recertificationRequired set to true along with domain, newCurriculumVersion, and graceExpiry in ISO-8601 format.
System Processing	The certification service updates the curriculum by consuming the Redis event. Then, a query checks the credentials table for agents with old credentials that need updating. If an agent is affected, the system does a few things: First, it tells Redis to publish a recert required event on the agent-lifecycle channel. This event lets everyone know that re-certification is needed. Second, the system marks the agents' status as pending and sets a grace period until re-certification must happen. Third, it adds an audit entry to log this action. After that, the admin dashboard refreshes its re-certification queue panel. Meanwhile, the Training Session Service takes care of scheduling those who need re-certification for re-enrollment. If an agent passes the test, they receive a new credential with the updated version number and active status. The older credential gets marked as superseded. On the flip side, if an agent doesn't go through the process during the grace period, their credential automatically gets set to superseded. Plus, admins get alerted about it. Daily checks at midnight UTC handle this monitoring task.
System Output	The system sends Redis events (once per agent impacted): { event: 'recert.required', agentUUID, domain, newCurriculumVersion, graceExpiry } which the Training Session Service picks up. Then, it updates the database: in the agents section, recertStatus is set to 'pending' and graceExpiry is added; each notified agent gets a new entry in audit_events. When the grace period ends, admins see {N} agent credentials set to 'superseded' in the specified domain plus an info alert saying so on the Admin Dashboard's Re-certification Queue Panel.

Other	When you re-certify ahead of time, you keep your 'active' status and continue to be matched in the marketplace until the new cert comes through. The system in T1 has a fixed 30-day grace period for this. However, starting with T2, admins can set these times differently via the Admin Dashboard. Little fixes to the curriculum, such as mending broken links or updating references, don't prompt re-certification. It's major or minor version updates, changing learning objectives, that kick off the whole process.
Constraints	Re-certification notifications are sent out asynchronously to avoid blocking the Knowledge Engine's curriculum activation. The idempotency key for this is (agentUUID + newCurriculumVersion), which allows the system to process the curriculum updated event multiple times. But if it happens twice, the second run skips notification because it detects existing 'pending' recertStatus records. Plus, there's a limit of 10,000 notifications per curriculum update in Tier 1. For bigger groups, batch processing is a Tier 2 feature. Also, the credentials table remains append-only; changing the status from 'active' to 'superseded' is the sole change allowed on records—everything else is locked down post-initial insert.
Data Handling	To update the status, run this: <code>`UPDATE credentials SET status = 'superseded' WHERE agentUUID = \$1 AND domain = \$2 AND curriculumVersion != \$3 AND status = 'active';`</code> Also, for the agents table, use this: <code>`UPDATE agents SET recertStatus = 'pending', graceExpiry = \$1 WHERE agentUUID = \$2;`</code> For audit logs, add this info: <code>`{ eventType: 'recert_triggered', agentUUID, domain, previousVersion, newVersion, notifiedAt }`</code> Timestamps should be in UTC ISO 8601 format. Plus, set the grace period like this: <code>`graceExpiry =</code>
Error Handling	When the "curriculum.updated" event doesn't work out, there's an exponential backoff retry that tries every 1, 2, 4, and then 8 seconds. If it fails five times, it goes to the dead-letter queue, and FR-18 critical pops up with "curriculum.updated processing failed for domain {domain}. Manual reprocessing required." If no agents are affected, the workflow ends without any notifications. The admin dashboard says affectedAgentCount is zero, and no alerts go off. When an individual recertification requirement fails, the system retries per-agent for up to three attempts at five-second intervals. Should these fail, the agent is marked as 'notification_failed' on the admin dashboard, requiring manual admin intervention. For updating the agents table, the system gives three retry shots. If those don't fix it, FR-18 warns with "Agent recertStatus update failed for agentUUID {id}. Notification dispatched but profile not updated." The Training Session Service might still try to reenroll using the Redis event. If the grace period job fails, it writes to the audit_events and tries again on the next daily run. Plus, FR-18 informs with "Grace period monitoring job failed. Credential supersession may be delayed by up to 24 hours."

Table 22 — FR-11: Re-Certification Trigger

Description	When an agent fails to get more than 80 percent in the exam, the Skill Gap Analysis Service automatically creates a diagnostic report that will target the skills that are not being met by the agent. It takes the feedback from the LLM grader, identifies the cognitive levels and sub-domains that the agent failed to perform correctly, and classifies the failure into severity levels for example CRITICAL in case of core foundational failure or MODERATE in edge case or adversarial failure). All this process will be completely automated with no human intervention.
Actor(s)	System - Skill Gap Analysis Service. Triggers itself by an automatic Redis exam.failed event from the Examination Service. There's no human involved.
System Input	Event extracted from the channel 'examination-events' in Redis: If the exam fails, it will return the following event with the following data: This payload is used to start the gap analysis and includes the telemetry required for the failed exam.
Display	The following information is displayed in Admin Dashboard > Analytics Panel: • gapID (UUID) • agentUUID • Domain • Failed Sub-domains • Severity Classification (Critical/Moderate) • Generated At (in UTC) Agent API > Polling GET /api/v1/agents/{agentUUID}/diagnostics returns the structured diagnosticPayload containing the list of which modules need to be reviewed.
System Processing	1. Service is listening to and reading the exam.failed event message from the Redis examination-events channel. 2. Payload validation: Make sure that agentUUID is present in the agents table and examID is present in the examinations table with a score below 80.0. 3. The service is designed to extract the failedQuestions array and use an LLM prompt to identify which specific wrong answers are due to which sub-domain logic failures. 4. Gaps are classified according to the cognitive tier: Factual failures are classified CRITICAL, Adversarial/Edge-Case failures are classified MODERATE. 5. The weak sub-domains and specific module_ids recommended for review are placed in a structured diagnosticPayload JSON object. A structured diagnosticPayload JSON object is created that contains the weak sub-domains and specific module_ids recommended for review. 6. A UUID v4 (gapID) is used to create a unique identifier for the gap and the record is added to the skill_gaps PostgreSQL table. 7. Publish to Redis the remediation.required event containing the gap data.

	8. The new diagnostic generation is reflected in the Admin Dashboard's Analytics panel.
System Output	Primary: New record in the skill_gaps table in PostgreSQL. The remediation.required event is published in the agent-lifecycle channel by Redis Event. Agent API: Diagnostic data is available when polled.
Other	<u>Idempotency:</u> The examID will be used as the idempotency key. If there are multiple exam.failed events, the next one will discover the existing gapID for that exam and will not generate duplicate diagnostic reports.
Constraints	The analysis is completely asynchronous, preventing the Examination Service from being blocked waiting for diagnostic generation. The LLM that parses the failures is read-only in terms of the curriculum: it can't change the source material, just suggest module_ids.
Data Handling	<u>Create the table skill_gaps with the following structure:</u> skill_gaps (gapID — PRIMARY KEY, UUID v4, examID — FOREIGN KEY → examinations).examID is a unique, not null identifier for an examination. diagnosticPayload is a not null JSONB field. severity is a not null, unique, checked VARCHAR(20) field. <u>DB-level constraints:</u> UNIQUE(examID) — one diagnostic report for each failed exam. <u>Row-Level Security:</u> GRANT (UPDATE and DELETE) TO app_user ON skill_gaps.

Error Handling	<u>Duplicate event (Redis at-least-once delivery):</u> → SELECT gapID FROM skill_gaps WHERE examID = \$1 → If exists: return gapID, write 'duplicate_suppressed' to audit_events, return. <u>LLM Analysis Timeout:</u> → Retry up to 3 times with exponential backoff. → After 3 failures: fallback to a generic MODERATE classification for all failed questions and log 'llm_analysis_fallback' to audit_events. <u>Database INSERT failure (PostgreSQL unavailable):</u> → Retry up to 3 times. → After 3 failures: event moved to Redis dead-letter queue. → FR-18 alert: CRITICAL — "Skill gap write failure for examID {id}."
-----------------------	--

Table 23 — FR-12: Skill Gap Identification

Description	The system dynamically reroutes agents in a failure by itself back to a fitting learning site determined by the diagnostic payload of FR-12, automatically. As a result, the training session difficulty parameter can be lowered to make sure that the failure is done to the lower difficulty levels for the specific concepts to which the error is attributed, and the agent is only allowed to re-examine the concepts if they have been well established in its previous reasoning.
Actor(s)	System - Training session service. Fired in response to the Redis remediation.required event from the Skill Gap Analysis Service. No humans are involved
System Input	<u>Agent-lifecycle:</u> Got a message from a Redis event. An event containing information about the remediation requirement, its agent UUID, the gapID, the diagnostic payload, and the date when it was sent at. An event that conveys information about the remediation requirement, the agent's UUID, the gapID, the diagnostic payload information and the date that the event was sent at. <u>The module sequence order was obtained via an internal database query:</u> SELECT module_id, sequence_order FROM modules WHERE module_id IN (extracted from diagnosticPayload);
Display	Admin Dashboard > Active Sessions Panel shows the agent's status as Remedial instead of Initial Training, with the corresponding modules being taken up. When using Agent API to use Polling GET /api/v1/agents/{agentUUID}/status, it returns { sessionStatus: 'remedial', activeModules: [...] }.

System Processing	1. The remediation.required event is consumed by the Training Session Service from the Redis channel. The Training Session Service pulls the remediation.required event off the Redis channel. 2. The service reads the diagnostic payload from the JSON string to identify the set of module_ids for further review. 3. A tailored CRM course plan is created based on the identified CRITICAL and MODERATE gaps – excluding modules the agent already knows. 4. Initializes a new row in the table training_sessions. Based on FR-06 (Adaptive Difficulty Adjustment) mechanics, the payouts difficulty_level are clearly set to be 1 for CRITICAL gaps and 2 for MODERATE gaps. 5. An audit_events record is added which records the start of a remedial loop. 6. The service gradually increases the amount of difficulty for the agent until they select a different exam, by providing the special reminders that the agent is supposed to read.
System Output	<u>Primary:</u> Training sessions table PostgreSQL includes new sessions added with modified levels of difficulty. <u>Changes made to the database:</u> Changed the status of agents table to 'remedial_training' and added log audit_events table with append-only session start record.
Other	The agent's original training data will not be erased by remedial training. The training_sessions table maintains it's flatten and the system will retain a full history of the agent's first attempt, failure and then subsequent attempts to correct the error.
Constraints	Remedial training may not be skipped and an agent cannot try to conduct an immediate re-examination. Note that any agent with a status of remedial_training will be denied an ability to start the exam until a new training.complete event is published by the Training Session Service to the API Gateway.
Data Handling	The table training_sessions contains the following attributes: sessionID — PRIMARY KEY agentUUID — FOREIGN KEY references agents.agentUUID → foreign key to modules, moduleID — check(difficultyLevel BETWEEN 1 AND 5) INT. Actually, the update is to set status = 'remedial_training' to agents whose agentUUID is \$1.

Error Handling	remediation.required event not consumed: → Exponential backoff retry: 1s, 2s, 4s, 8s. → After 5 failures: dead-letter queue; FR-18 CRITICAL — "Remediation trigger failed for agentUUID {id}." If any module_ids in the diagnosticPayload are no longer in the current curriculum track, then the service will ignore those module_ids and continue with the module_ids that are present in the valid curriculum track, logging an WARNING to audit_events.
-----------------------	---

Table 24 — FR-13: Remedial Training Trigger

Category	Description
Description	The system shall autonomously evaluate and match certified AI agents with open task opportunities within a strict thirty-second Service Level Agreement (SLA) from the exact moment of qualification, eliminating human procurement bottlenecks from the platform lifecycle (Gartner, 2025). This functionality relies on an automated background worker that handles event-driven evaluation immediately upon certificate generation.
Actor(s)	Talent Marketplace Service; Redis Message Broker.
Trigger	The automatic publication and subsequent ingestion of a cert.issued lifecycle event on the Redis channel network.
System Input	Asynchronous Redis message containing structural validation variables: { event: 'cert.issued', agentUUID: UUID, credentialID: UUID, domain: String, totalScore: Float, curriculumVersion: String, issuedAt: TIMESTAMPTZ }.
Display	Admin Dashboard > Marketplace Analytics Panel displays the active assignment entries containing the matching identifiers: matchID, agentUUID, assigned postingID, relevanceScore, and the generation timestamp (matchedAt) formatted in Coordinated Universal Time (UTC).

System Processing	1. The service intercepts the cert.issued event notification published to the asynchronous Redis channel infrastructure. 2. Executes an immediate relational validation check against the database layer to confirm the incoming payload satisfies the strict idempotency requirements. 3. Queries active, open entries within the job_postings table where the required skill domain matches the agent's certificate metrics. 4. Computes a calculated relevance ranking based on a multi-factor formula evaluating the absolute examination score and the Jaccard similarity coefficient of operational skill tags. 5. Filters and structures the results to isolate the top-k ideal pairings for the agent. 6. Commits the transaction details within a single, atomic PostgreSQL execution to the persistent agent_matches table. 7. Publishes a downstream agent.matched notification payload to update the telemetry logs.
System Output	Primary: A new relational row committed to the agent_matches database table. Redis Event: agent.matched published to update core platform state logs. Secondary: Real-time update to the dashboard interface metrics upon the conclusion of the 30-second polling cycle.
Other	In accordance with distributed data management paradigms established by Kleppmann (2017), consumer idempotency constraints are strictly enforced at the data ingestion boundary to neutralize message duplication anomalies arising from Redis at-least-once transport delivery logic.
Constraints	The entire matching and database commitment transaction must execute and terminate within a maximum window of 30 seconds from event arrival. Relevance calculation scores are float types bounded strictly between 0.00 and 1.00.

Data Handling	Relational updates affect the following schema structures: agent_matches (match_id — PRIMARY KEY, cert_id — FOREIGN KEY REFERENCES credentials, posting_id — FOREIGN KEY REFERENCES job_postings, relevance_score, matched_at).
Error Handling	In the event of temporary PostgreSQL database unavailabilities, the system implements an exponential backoff loop attempting reconnects at 1s, 2s, 4s, and 8s intervals. If connections fail past the threshold, transaction bounds are canceled, payloads are safely routed into a persistent dead-letter queue, and an immediate FR-18 critical alert is raised. Duplicate event entries identified by the idempotency validations are suppressed automatically without triggering execution errors.

Table 25 — FR-14: Automated Marketplace Matching

Category	Description
Description	The system shall provide an administrative endpoint registry to configure, track, and sustain available enterprise tasks and required capabilities within the platform database, functioning as the primary reference library for all automated matching workflows.
Actor(s)	Platform Administrator; Talent Marketplace Service.
Trigger	An authorized, authenticated system administrator submits an HTTP request payload containing structural domain updates or new job configurations to the gateway registry endpoints.
System Input	Secure HTTP REST API request directed to POST /api/v1/marketplace/postings containing JSON parameters: { required_domain: String, min_score_threshold: Float, skill_tags: Array, created_by: UUID }.
Display	Admin Dashboard > Task Posting Registry Panel displays interactive data lines containing fields for postingID, required_domain, min_score_threshold, associated skill_tags, and the configuration metadata status.

System Processing	1. Ingests incoming administrative HTTP calls at the API Gateway boundary and validates security compliance via attached JSON Web Token (JWT) credentials (IETF, 2015). 2. Confirms inputs meet standard sanitization metrics to avoid common parsing or prompt-injection vulnerabilities. 3. Saves the structured variables directly into the PostgreSQL job_postings tracking library, assigning a unique generation token. 4. Appends explicit provenance indicators (created_by) to satisfy standard configuration lineage guidelines (Object Management Group, 2017). 5. Dispatches a posting.added lifecycle notification to the Redis event messaging layers. 6. Intercepts the posting.added signal locally to initiate a retroactive matching trigger, scanning historical credentials logs to find qualified agents certified before the creation of the current posting.
System Output	Primary: Database record successfully appended to the job_postings storage table. Redis Event: posting.added broadcast across active messaging boundaries. Secondary: Instantiation of retroactive matching loops yielding additions to the agent_matches layer.
Other	To safeguard the integrity of historical operational audits in highly regulated deployment spaces, destructive hard-deletes are structurally banned across the database layer; administrative task removal requests invoke soft-delete workflows that apply state filters (status = 'deleted', deleted_at = TIMESTAMPTZ) while retaining original records.
Constraints	Administrative connections require complete role-based token validation bounded by a standard 3600-second session expiration window. Soft-deleted posting entries are omitted from active matching runs but stay visible for audit logging queries.

Data Handling	Schema attributes map to the following structure: job_postings (posting_id — PRIMARY KEY, required_domain, min_score_threshold, is_open, created_by, deleted_at, skill_tags). DB level validation rules mandate: CHECK (min_score_threshold >= 80.0).
Error Handling	Unauthenticated administrative access attempts or faulty access tokens yield immediate HTTP 401/403 responses. Database connection anomalies initiate a three-step transactional retry script. If retroactive matching calculation loops time out, the system safely triggers an incremental fallback sequence, recording a warning message to the audit_events log to prevent thread locking on core platform services.

Table 26 — FR-15: Task Posting Registry

Item	Description
Functional Requirement ID	FR-16
Requirement Name	System-Wide Health Monitoring
Description	The system shall continuously monitor the operational status of all EduAgentX microservices using dedicated /health endpoints.
Actors	Monitoring & Observability Service, Platform Administrator
System Input	Health check responses from registered microservices.
Display	Service Health section within the Admin Dashboard.
System Processing	Poll all /health endpoints every 30 seconds, record service status and response time, update Prometheus metrics, and generate alerts when failures are detected.
System Output	Updated service health records and monitoring metrics.
Constraints	All services must expose a valid /health endpoint and support 30-second polling intervals.
Data Handling	Monitoring data is stored in the service_health table.
Error Handling	Failed health checks are logged and forwarded to the Alert Generation Service for notification.

Table 27 — FR-16: System-Wide Health Monitoring

Item	Description
Functional Requirement ID	FR-17
Requirement Name	Admin Dashboard
Description	The system shall provide a centralized Admin Dashboard that allows administrators to monitor platform activity, service health, certifications, marketplace activity, and system alerts.
Actors	Platform Administrator
System Input	Data collected from monitoring, certification, marketplace, and audit services.
Display	Dashboard sections including Service Health, Certifications, Marketplace Activity, Alerts, and Audit Logs.
System Processing	Authenticate administrators using JWT, retrieve information from relevant services, and automatically refresh dashboard data every 30 seconds.
System Output	Updated dashboard information and system statistics.
Constraints	The dashboard shall remain read-only during Trimester 1 and require JWT authentication for access.
Data Handling	Dashboard data is retrieved through APIs and does not directly modify database records.
Error Handling	Failed requests are logged and an error message is displayed to the administrator.

Table 28 — FR-17: Admin Dashboard

Item	Description
Functional Requirement ID	FR-18
Requirement Name	Alert Generation
Description	The system shall generate alerts when predefined operational or security issues are detected across the EduAgentX platform.

Actors	Monitoring & Observability Service, Platform Administrator
System Input	Monitoring events, service failures, and system status updates.
Display	Alert Center section within the Admin Dashboard.
System Processing	Detect alert conditions, assign a severity level (INFO, WARNING, or CRITICAL), store the alert record, and notify administrators when required.
System Output	Alert notifications and alert records.
Constraints	Email notifications are only sent for CRITICAL alerts.
Data Handling	Alert information is stored in the alerts table.
Error Handling	Failed alert creation attempts are logged and retried automatically.
Alert Conditions	Service unavailable for more than 60 seconds (CRITICAL), credential issuance failure (CRITICAL), curriculum update failure (CRITICAL), health monitoring timeout (WARNING), and re-certification processing failure (WARNING).

Table 29 — FR-18: Alert Generation

SEC ID	Security Requirement	How We Are Implementing It	How We Know It Works
SEC-01	All API endpoints need a valid JWT that includes a role claim - either agent or admin - but it expires in an hour. Agents can't use their tokens to access admin stuff.	The JWTs get created by the API Gateway Auth Service, using PyJWT with HS256 signing. The signing secret is kept secure as a Docker environment variable. Each downstream service checks both the JWT signature and role claim before moving on.	In tests, requests without a valid JWT get an HTTP 401 response. If agents try hitting admin endpoints, those get hit with a 403. I checked, and tokens are indeed good for just 3,600 seconds, verified across all eight services.
SEC-02	The exam submission endpoint (POST /api/v1/exams/submit) has a limit of 5 requests per agentUUID in a one-minute timeframe to stop quick retry tricks.	It works through a sliding window counter at the API Gateway, using an agentUUID from the JWT claims. This counter is saved in Redis for 60 seconds. When the limit gets hit, you	During load testing, making 6 submissions in a minute for the same agentUUID results in an HTTP 429 for that sixth request. The counter resets properly after the minute ends. I confirmed this with a Pytest integration test.

		get an HTTP 429 error along with a Retry-After header.	
SEC-03	Agents can't change the LLM grading rubric, the curriculum prompts, or system instructions through submitted exam answers. They have to block prompt injection.	For LangChain, the templates always use a fixed system prompt prefix. The agent answer text is added as a bounded variable; it doesn't get concatenated into the system prompt. Each variable boundary in the templates is checked for injection vectors.	During testing, ten harmful prompt patterns were tried in agent exam answers. These patterns didn't alter grading logic, the rubric, or the system prompt. This was verified through code reviews and Pytest.
SEC-04	The credentials table is append-only, no application-level updates or deletes are allowed. This restriction is enforced in two different ways.	In the database layer, PostgreSQL uses Row Level Security (RLS) to grant the app user INSERT privileges only; it revokes any UPDATE or DELETE access. The application layer doubles down on this by using Pytest for static code analysis. It checks the Certification Service and ensures no UPDATE or DELETE SQL exists in the codebase for this specific table.	If you try to update or delete directly, PostgreSQL returns an error saying 'permission denied for table credentials'. Plus, pytest static scans come back clean with zero findings. Both results are confirmed in the test report.
SEC-05	We enforce HTTPS with TLS 1.2 or higher on all external endpoints, and HTTP connections get redirected.	For internal PostgreSQL connections, we use `ssl=require`. Our API Gateway uses nginx with a Let's Encrypt certificate to terminate HTTPS. We also redirect all plain HTTP requests to HTTPS with a 301 redirect. Plus, `ssl=require` is set in the PostgreSQL connection strings for all FastAPI services.	An SSL scan verifies TLS 1.2+ on our external endpoints, while audits confirm the HTTP redirects and `ssl=require` in all eight microservice configurations.

SEC-06	In T1, no personally identifiable information is collected or stored. Agents are represented only by a UUID v4.	The database was made with no VARCHAR fields meant for names or emails. During registration, any payload with an email or name pattern gets rejected.	The schema checks show no PII fields anywhere. Plus, when the registration endpoint is tested with a payload that includes an email or name, it correctly returns an HTTP 400 error. Even when data is exported for research, the UUID values get stripped out first to protect identities.
SEC-07	The Admin Dashboard session times out after 60 minutes of inactivity and sends you back to the login page. They store session tokens just in memory, not in localStorage or sessionStorage.	React SPA keeps the JWT in a JavaScript variable too, avoiding browser storage to stop cross-site scripting attacks. If there's no activity for an hour, your session clears the token and redirects you to log in again with a "Session expired. Please log in again" message.	For testing the session timeout, staying inactive for 61 minutes causes an auto-redirect to the login page. The API Gateway then rejects the expired JWT with an HTTP 401 error. This worked when tested from end-to-end in the browser.
SEC-08	Each audit_events entry comes with a SHA-256 hashed payload JSON. The Monitoring Service checks by recomputing hashes on read; it raises an INTEGRITY_WARNING if there's a mismatch.	For every audit_event, we calculate the payloadHash using SHA-256 on payloadJSON in Python and store it with the payload. Every 6 hours, a check runs that recomputes any unverified hashes and marks mismatches.	Tamper simulation: direct UPDATE of payloadJSON via psql triggers an INTEGRITY_WARNING on the Admin Dashboard within one verification cycle (≤ 6 hours). End-to-end tests confirm this.

Table 30 — Security Requirements

ID	Requirement	Acceptance Criteria
US-01	Easy Navigation	Users can access any dashboard section within three clicks.
US-02	Consistent Interface	All pages follow a uniform layout, styling, and navigation structure.
US-03	Responsive Design	The dashboard displays correctly on screens with a minimum resolution of 1366×768 .
US-04	Clear Error Messages	Error messages are displayed within 2 seconds of a failure occurring.

US-05	Automatic Refresh	Dashboard information is updated automatically every 30 seconds.
US-06	Readability	Text and interface elements comply with WCAG AA readability standards.
US-07	Fast Page Loading	Dashboard pages load within 3 seconds under normal operating conditions.

Table 31 — Usability Non-Functional Requirements

ID	Requirement	Acceptance Criteria
OPR-01	Docker Deployment	All microservices can be deployed and started using Docker Compose.
OPR-02	Automated Startup	Services initialize automatically without manual configuration.
OPR-03	Logging Support	All services generate structured logs for monitoring and debugging purposes.
OPR-04	Health Monitoring	Service health checks are performed every 30 seconds.
AVA-01	System Availability	The platform maintains at least 99% availability during testing.
AVA-02	Recovery Capability	Failed services recover or restart within 60 seconds of failure detection.
REL-01	Event Reliability	At least 99% of Redis events are successfully delivered and processed.

Table 32 — Operational and Reliability Non-Functional Requirements

ID	Requirement	Acceptance Criteria
LEG-01	No Storage of Personal Identifiable Information (PII)	Database schema and stored records contain no PII fields.
LEG-02	Academic Integrity	All generated content, references, and documentation are properly cited where applicable.
LEG-03	OpenAI Policy Compliance	API usage complies with the provider's terms of service and usage policies.

LEG-04	Auditability	All critical system actions are recorded and traceable through audit logs.
--------	--------------	--

Table 33 — Legal Non-Functional Requirements

ID	Requirement	Acceptance Criteria
MNT-01	Maintainable Codebase	All code changes are reviewed through pull requests before being merged.
MNT-02	Modular Architecture	Each microservice can be developed, deployed, and updated independently.
MNT-03	Documentation Quality	All APIs and major system components are documented and accessible to developers.
SCL-01	Agent Scalability	The platform supports at least 1,000 concurrent AI agent records without performance degradation.
SCL-02	Notification Scalability	The system can process and deliver at least 10,000 recertification notifications successfully.

Table 34 — Other Non-Functional requirements

Milestone	Target Date
Requirements Specification Completion	05 June 2026
Architecture and Design Finalization	Week 6
Testing Phase Begins	Week 7
First Panel Demonstration	Week 9
Final Submission and Demonstration	Week 11

Table 35 — Preliminary Schedule

Item	Estimated Cost (AED)
OpenAI API Usage	600
Cloud Hosting	300
Domain and SSL Certificate	150
Development Tools and Services	100
Testing and Deployment Reserve	250
Total	1,400 AED

Table 36 — Preliminary Budget

Test Area	Objective	Success Criteria
Agent Registration	Verify successful agent registration	Unique Agent ID is generated and stored
Authentication	Validate JWT authentication	Authorized users gain access; invalid tokens are rejected

Curriculum Assignment	Verify domain assignment process	Agent is assigned the correct curriculum track
Curriculum Generation	Test curriculum creation	Curriculum is generated and saved successfully
Training Delivery	Verify module delivery	Training modules are accessible to the agent
Adaptive Learning	Test difficulty adjustment	Difficulty changes according to performance
Exam Generation	Verify exam creation	Unique examination is generated successfully
Exam Evaluation	Validate automated grading	Exam score is calculated and recorded correctly
Certification Issuance	Verify credential generation	Certificate is issued and stored successfully
Marketplace Matching	Test agent-job matching	Suitable matches are returned within 30 seconds(>30)
Health Monitoring	Verify service monitoring	Service status is updated correctly
Alert Generation	Test failure detection and notification	Alert is created when predefined conditions occur

Table 37 — Pilot Test Case

Risk	Potential Impact	Mitigation Strategy
OpenAI API Unavailability	Curriculum generation and evaluation services may become unavailable	Implement retry mechanisms and monitor API usage
Redis Service Failure	Event communication between microservices may even be interrupted	Use monitoring tools & maintain backup recovery procedures
Integration Conflicts	Delays during system integration as well as during testing	Conduct regular code reviews & follow version control practices
Budget Overruns	Project costs may exceed estimates	Monitor API consumption& control resource usage

Security Vulnerabilities	Unauthorized access or data compromise	Perform security testing& regular vulnerability assessments
--------------------------	--	---

Table 38 — Risks and Mitigation